



北京航空航天大学
BEIHANG UNIVERSITY

微机原理及接口技术

实验指导书

(2024版)

《微机原理及接口技术实验》课程教学团队 编

2024 年 10 月

目录

概述/前言/课程介绍	1
基本要求与注意事项	2
一、安全操作守则	2
二、预习要求	2
三、出勤要求	3
第一部分 实验系统操作指导	4
1 仪器设备介绍	4
2 仪器设备使用方法	5
2.1 实验箱结构	5
3 实验系统软件介绍	8
3.1 程序的编辑和编译	8
3.2 程序的调试和运行	11
4 课前预习一	14
注意事项	15
5 课前预习二	16
第二部分微机原理及接口技术实验指导	18
实验一 并行接口实验	18
一、实验目的	18
二、实验内容	18
三、实验设备	18
四、预习要求	18
五、实验原理	19
六、实验步骤	20

七、注意事项	21
八、实验报告要求	22
九、课后思考题	22
十、参考资料	23
实验二 数字音频实验（定时器与并口综合设计实验）	24
一、实验目的	24
二、实验内容	24
三、实验设备	24
四、预习要求	24
五、实验原理	24
图 2.2.3 程序流程图	28
六、实验步骤	28
七、注意事项	29
八、实验报告要求	29
九、课后思考题	30
十、参考资料	30
实验三 数/模与模/数转换	31
一、实验目的	31
二、实验内容	31
三、实验设备	32
四、预习要求	32
五、实验原理	33
六、实验步骤	37
七、注意事项	37
八、实验报告要求	38
九、课后思考题	38
十、参考资料	39
实验四 综合设计实验（自主设计型实验）	40

一、实验目的	40
二、实验内容	40
三、实验设备	41
四、预习要求	41
五、注意事项	42
六、实验报告要求	42
七、参考资料	43
第三部分 设计型实验（虚拟仿真实验）	44
实验五 最小系统设计与仿真实验（虚拟仿真实验）	44
一、实验目的	44
二、实验内容	44
三、实验设备	47
四、预习要求	47
五、实验原理	47
六、实验步骤	49
七、注意事项	49
八、实验报告要求	49
九、课后思考题	50
十、参考资料	50
实验六 并行接口设计实验（虚拟仿真实验）	51
一、实验目的	51
三、实验内容	51
三、实验设备	51
四、预习要求	51
五、实验原理	52
六、实验步骤	52
七、注意事项	52
八、实验报告要求	53

九、课后思考题	53
十、参考资料	53
实验七 定时器与数字音频设计实验（虚拟仿真实验）	54
一、实验目的	54
二、实验内容	54
图 3.7.1 电路原理图	54
三、实验设备	55
四、预习要求	55
五、实验原理	55
六、实验步骤	57
七、注意事项	57
八、实验报告要求	57
九、课后思考题	58
十、参考资料	58
第四部分 ARM（STM32F103R6）实验	59
一、ARM（STM32F103R6）基本概念及编程简介	59
二、ARM/STM32F103 实验	70
实验八 通用输入输出（GPIO）/流水灯实验	70
一、实验目的	70
四、实验内容	70
三、实验设备	72
四、预习要求	72
五、实验原理	72
六、实验步骤	77
七、注意事项	86
八、实验报告要求	86
九、课后思考题	86



十、参考资料	86
实验九 SysTick 时钟/数码管显示实验	87
一、实验目的	87
五、 实验内容	87
三、实验设备	89
四、预习要求	89
五、实验原理	90
六、实验步骤	90
七、注意事项	91
九、课后思考题	91
十、参考资料	91

概述/前言/课程介绍

《微机原理及接口技术实验》是自动化、控制工程以及电子电气等工科专业教育中的一门核心专业基础实验课。本实验由微机原理部分及接口部分组成，微机原理部分以微机指令系统，汇编程序设计为主要内容，程序设计面向 16 位/32 位系统指令，接口部分面向几种常用的微机接口芯片如 8254、8255、0832、0809 开展实验，要求学生学习和理解相关硬件电路，面向 16 位地址/数据系统编程，实现对接口芯片的控制并完成具体的实验要求。

本实验课共 16 学时，包括 4 个实验单元，分别为：

- 1 并行接口实验
- 2 综合设计实验（定时器与并行接口设计型实验）
- 3 数模、模数转换实验
- 4 硬件综合实验（自主设计型实验）

此外，还提供了基于 EDA 的虚拟仿真实验、ARM 扩展实验等内容。学生通过实验学习，掌握微机组成原理、工作过程、软件设计及应用、以及接口芯片的输入输出控制，建立微机工作的整体概念。使学生在了解基本的微机应用系统基础上，具有进行软件和硬件开发的基本能力。

本实验课培养学生融合所学的电路电子技术、工程基础和专业知识的基本概念，通过软硬件结合的实验方案，提高学生的实际操作能力和综合设计能力。

通过本实验课程的学习，学生能够通过软硬件结合的实验项目直观了解微型计算机的基本工作原理，掌握 8086 汇编语言常用指令的使用及编程方法，加深对微机中各种控制接口技术的认识，掌握常用接口芯片的编程方式，熟练运用汇编语言编程控制接口与外设进行数据通讯；能够基于科学原理，根据工程需求分析，设计软硬件解决方案，完成软件设计与硬件电路设计；能够通过软硬件实验训练，培养发现并解决工程实践中出现的问题的能力，具备现场故障诊断维修的基本工程师素养。

基本要求与注意事项

一、安全操作守则

1. 首次进入实验室参加实验的学生应认真听取实验指导教师对于安全内容的介绍。
2. 实验室总电源由指导教师负责，学生不得擅自接触。
3. 不得携带任何食品进入实验室，水杯、饮料瓶等容器统一放在远离计算机和实验设备的指定地点，禁止放置在实验操作台上，以防发生危险。
4. 学生进行实验时，独立完成的实验线路连接或改接，须经指导教师检查无误并提醒注意事项后，方可接通电源。
5. 严禁带电接线、拆线、接触带电裸露部位及电机旋转部件。
6. 各种仪表、设备在使用前应先确认其所在电路的额定工作状态，选择合适的量程。若认为仪表、设备存在问题或发生故障，应报告指导教师，不得自行排除故障。
7. 实验中发生故障时，必须立即切断电源并保护现场，同时报告指导教师。待查明原因并排除故障后，才可继续进行实验。
8. 实验室内禁止打闹、大声喧哗、乱扔废物以及其它不文明行为。
9. 实验开始后，学生不得远离实验装置或做与实验无关的事。
10. 实验完毕后应首先切断电源，再经指导教师检查实验数据后方可拆除实验线路，并将实验仪表、用线摆放整齐。
11. 实验过程中如发生事故，应立即关断电源，保持现场，报告指导老师。若不按要求操作导致实验设备损坏，应根据损坏情况赔偿。
12. 因计算机内装有防病毒卡，文件只能存在非系统盘，最好存在 U 盘内，切忌存在桌面上，否则关机即消失。
13. 未经老师许可，严禁向实验室计算机内安装任何软件。

二、预习要求

上实验课前，要求仔细阅读实验指导书，完成本次实验相关的预习思考题，准备本次实验的相关程序。

三、出勤要求

本实验课共 16 学时，4 次实验课。学生必须按照学校教务实验系统的选课时间参加实验课并在签到表中签到。特殊情况不能按时上课，必须准备书面假条，由学院教务盖章后，在实验开始前交给实验老师、协调补课时间。

实验课迟到 10 分钟以上不得进入实验室，实验完成后必须经实验老师检查确认后方可离开实验室，以上情况无正当理由均按照无故缺席实验课处理。

未请假缺席一次实验，当次成绩为零。

注意：

选择实验 4 “题目七 加密/解密算法实现”的同学，应当提前 4 周开始，并在完成实验 3 的同时向实验指导老师汇报加解密算法的原理和编程方案。

第一部分 实验系统操作指导

1 仪器设备介绍

《微机原理及接口技术实验》用到的主要实验设备包括计算机和 TPC-ZK-II 实验箱，软件编程和调试运行使用 TPC-ZK-II 集成开发环境，实验系统组成如图 1.1 所示，实验系统依托 PC 主机 CPU 微处理器，主机与实验系统用 USB 通线相连，通过 USB 口通信完成微机接口硬件实验；实验系统配有集成软件，具有编辑、编译、链接、运行及调试功能；实验系统自备电源，具有电源短路保护确保系统安全；实验时采用自锁紧单股导线及排线。



图 1.1 《微机原理及接口技术实验》设备组成

1.1 TPC-ZI-II 实验箱系统硬件

实验箱硬件布局示意图如图 1.2 所示

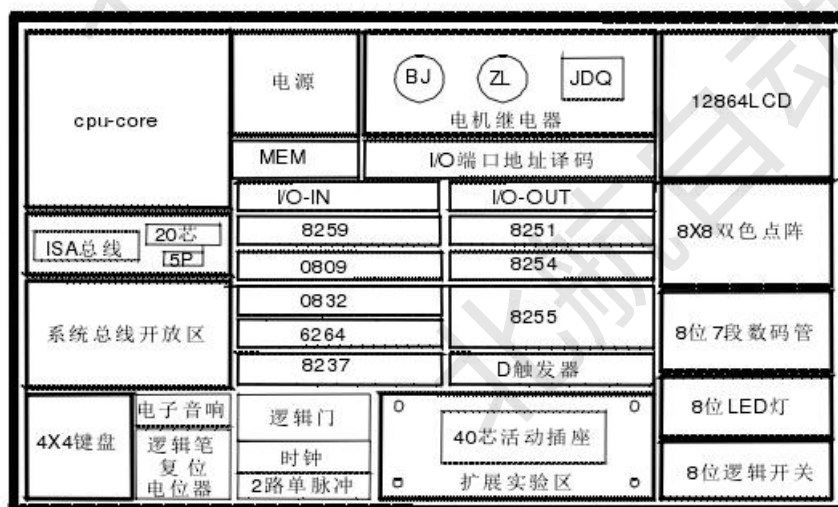


图 1.2 TPC-ZK-II 实验箱

2 仪器设备使用方法

2.1 实验箱结构

实验箱简介和使用说明



图 1.3 TPC-ZK-II 实验箱

- 1) 电源：主机电源打开后，再打开实验箱上的 K1，K2 开关。
- 2) 插孔：采用“自锁紧”插孔和排线插座。
- 3) 总线区：引出数据总线 D7~D0；地址总线 A9~A0；读、写信号 IOR、IOW；中断请求信号 IRQ；DMA 请求信号 DRQ1；DMA 响应信号、DACK1；及 AEN 信号，供实验选用。
- 4) 编程环境：按实验电路接线后，双击计算机桌面上的 TPC-ZK-II 集成开发环境图标进入程序开发。
- 5) 实验系统 I/O 端口地址译码电路

如图 1.4 所示，地址空间：280H~2BFH 共分 8 条译码输出线：Y0~Y7，其地址分别是 280H~287H；288H~28FH；290H~297H；298H~29FH；2A0H~2A7H；2A8H~

2AFH; 2B0H~2B7H; 2B8H~2BFH。

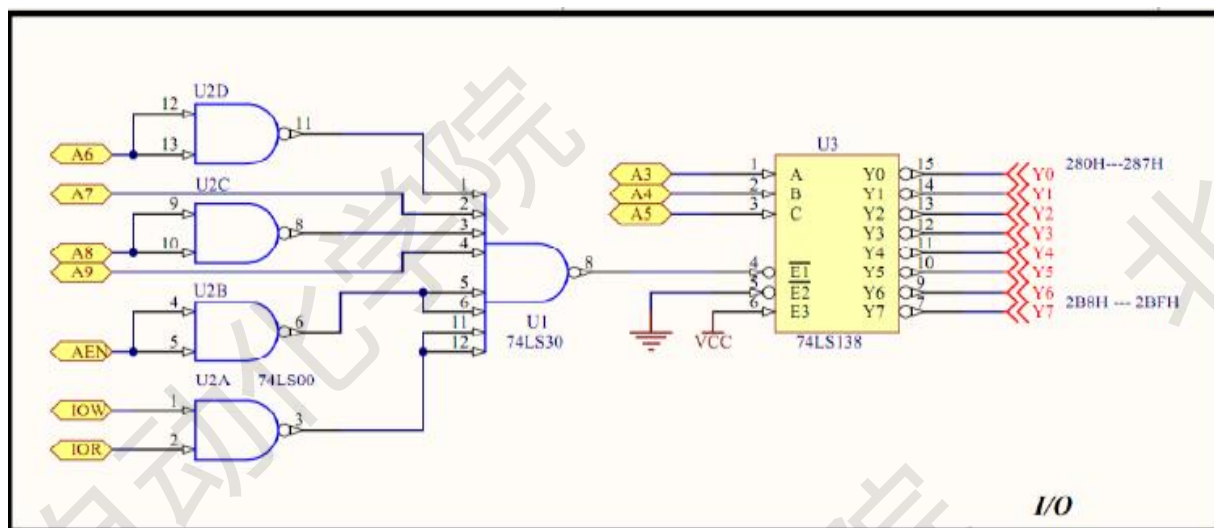


图 1.4 I/O 端译码电路原理图

6) 时钟电路

实验箱上可以输出 1MHZ、2MHZ 两种信号，供 A/D 转换器、定时器/计数器、串行接口实验使用。

7) 逻辑电平开关电路

如图 1.5，实验台右下方设有 8 个开关 K7~K0，开关拨到“1”位置时开关断开，输出高电平，向下到“0”位置时开关接通输出低电平，电路中串接了保护电阻，使接口电路不直接同+5V、GND 相连，防止误操作、误编程损坏集成电路。

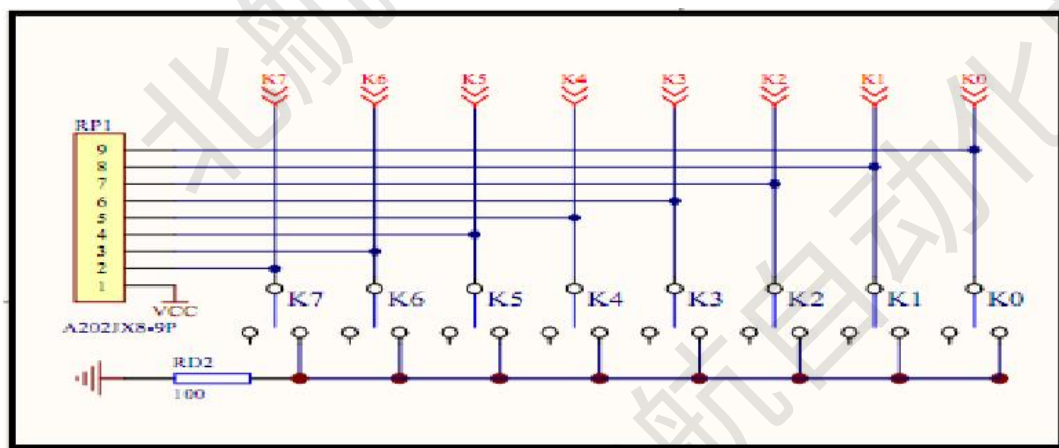


图 1.5 逻辑电平开关电路

8) LED 显示电路

如图 1.6，实验台上设有 8 个发光二极管及驱动电路（输入端 L7~L0），当输入信号为“1”时发光，为“0”时灭。

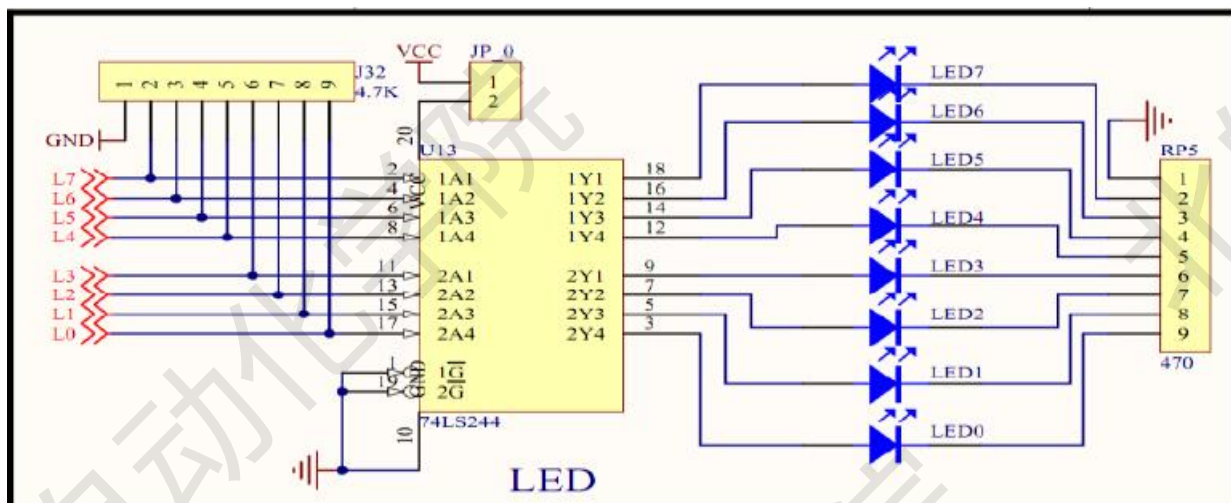


图 1.6 LED 显示电路

9) 七段数码管显示电路

两组共 8 个七段数码管，段码输入端：a、b、c、d、e、f、g、dp(JP3)，位码输入端：s1~s7(JP4)。

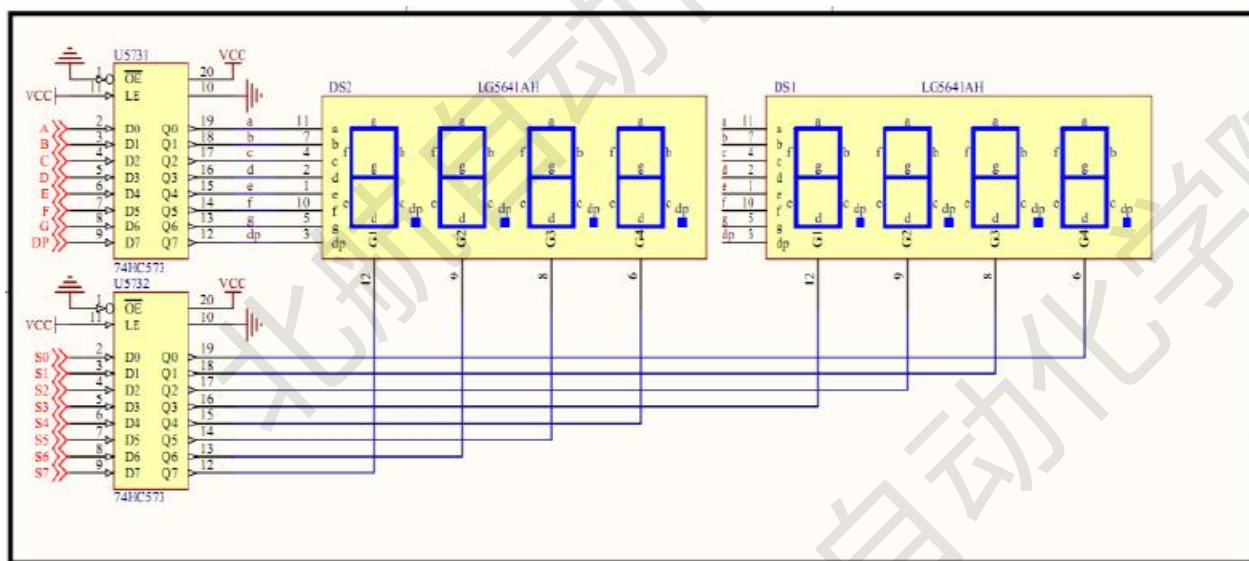


图 1.7 数码管显示电路

10) 接口集成电路

实验箱上有微机原理硬件实验最常用接口电路芯片，包括：可编程定时器/计数器（8254）、可编程并行接口（8255）、数/模转换器（DAC0832）、模/数转换器（ADC0809），这里芯片与 CPU 相连的引线除片选信号 CS 外都已连好，与外界连接的关键引脚在芯片周围用“自锁紧”插座和排线插座引出，供实验时使用。

11) 数字电路实验区

实验箱上有一块数字电路实验区，设有三种基本门电路（与、或、非）及 D 触发器在接口实验或数字电路实验时直接使用。

3 实验系统软件介绍

TPC-ZK-II 集成开发环境是 TPC-ZK-II 实验系统配套的软件，提供了用户程序的编辑和编译，调试和运行，实验项目的查看和演示，实验项目的添加等功能。该软件基于 windows2000/XP/2003/WIN7 环境。

双击计算机桌面上的 TPC-ZK-II 集成开发环境图标进入软件环境。主界面如图所示：



图 1.8 TPC-ZK-II 集成开发环境主界面

3.1 程序的编辑和编译

1) 新建一个源程序

在当前运行环境下，选择菜单栏中的“文件”菜单，菜单下拉后选择“新建”，或是在工具栏中单击“新建”，会弹出“新建”窗口，“新建”窗口如图 1.9 所示：

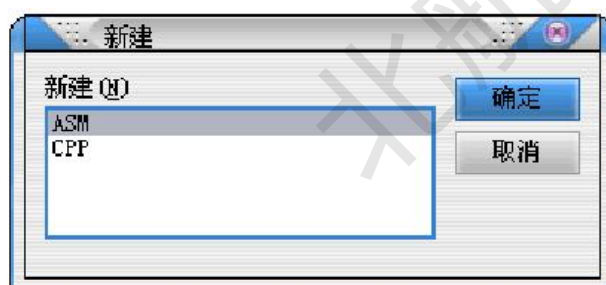


图 1.9 新建一个源程序

选择新建表单中的“ASM”，点击“确定”即可新建对应的汇编语言程序，点击“取消”则取消新建源文件操作。

2) 打开一个源程序

当前运行环境下，选择菜单栏中的“文件”菜单，菜单下拉后选择“打开”，或是在工具栏中单击“打开”，会弹出“打开”文件选择窗口，“打开”窗口如图 1.10 所示：

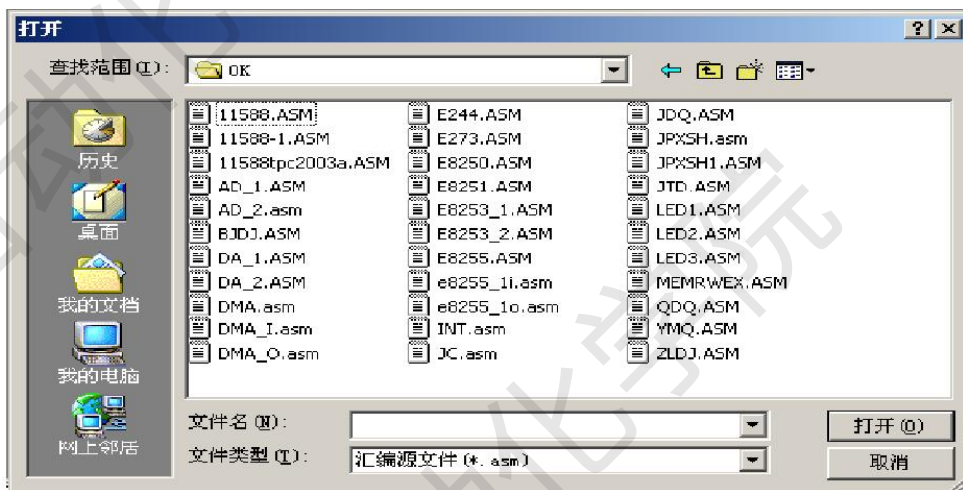


图 1.10 打开一个源程序

在窗口中“文件类型”下拉菜单中选择“ASM 文档 (*.asm)”一项，程序即显示当前目录下所有的 asm 文档，单击要选择的文件，选中的文件名会显示在“文件名”中，单击“打开”则打开当前选中的文档显示在文档显示区域。点击“取消”则取消新建源文件操作。

3) 编辑源程序

软件提供了基本的编辑功能，并实现了实时的语法高亮，各项操作说明如下：

撤消

当前运行环境下，选择菜单栏中的“编辑”菜单，菜单下拉后选择“撤消”，或是在工具栏中单击“撤消”，即可撤消上一步剪切或粘贴操作。

剪切

当前运行环境下，选择菜单栏中的“编辑”菜单，菜单下拉后选择“剪切”，或是在工具栏中单击“剪切”，即可将文档显示区域中选中的内容剪切到剪贴板。

复制

当前运行环境下，选择菜单栏中的“编辑”菜单，菜单下拉后选择“复制”，或是在工具栏中单击“复制”，即可将文档显示区域中选中的内容复制到剪贴板。

粘贴

当前运行环境下，选择菜单栏中的“编辑”菜单，菜单下拉后选择“粘贴”，或是在工具栏中单击“粘贴”，即可将剪贴板中当前内容粘贴到文档显示区域光标所在处。

全选

当前运行环境下，选择菜单栏中的“编辑”菜单，菜单下拉后选择“全选”，即可将文档区域中所有内容选中。

查找

当前运行环境下，选择菜单栏中的“编辑”菜单，菜单下拉后选择“查找”，弹出查找对话框，在查找内容一栏中输入需要查找的内容，可选择“全字匹配”与“区分大小写”的查找方式，单击查找下一个程序则在文档显示区域中搜索与查找内容匹配的字符串，找到第一个后则高亮显示，点击查找下一个则继续搜索下一个匹配字符串，点击“取消”退出查找操作。

查找下一个

当前运行环境下，选择菜单栏中的“编辑”菜单，菜单下拉后选择“替换”，即可在当前文档显示区域查找下一个查找对话框中输入的字符串，找到后高亮显示。

替换

当前运行环境下，选择菜单栏中的“编辑”菜单，菜单下拉后选择“替换”，弹出替换对话框。在查找内容一栏中输入需要查找的内容，可选择“全字匹配”与“区分大小写”的查找方式，在“替换为”一栏中输入需要替换的内容，单击“查找下一个”程序则在文档显示区域中搜索与查找内容匹配的字符串，找到第一个后则高亮显示，单击“替换”将匹配的字符串替换，也可单击“全部替换”将当前文档显示区域中所有与查找内容匹配的字符串全部替换。单击“查找下一个”则继续搜索下一个匹配字符串。也可单击“取消”退出查找操作。

4) 保存源程序

当前运行环境下，选择菜单栏中的“文件”菜单，菜单下拉后选择“保存”，如果是无标题文档，需在提示下输入文档的名称及选择保存的路径，单击确定后保存；否则程序自动保存当前文档显示区域中显示的文档。或者选择菜单栏中的“文件”菜单，菜单下拉后选择“另存为”，并在提示下输入文档的名称及选择保存的路径，单击确定后保存。

5) 编译源程序

编译调试窗口

在当前运行环境下，选择菜单栏中的“查看”菜单，单击编译调试窗口选项或是单击工具栏中“显示/隐藏编译调试窗口”按钮则可对状态栏的显示进行操作。若当前环境显示编译调试窗口，则单击编译调试窗口选项即可隐藏该窗口，编译调试窗口选项前选中标记将消失；若当前隐藏编译调试窗口，则单击编译调试窗口选项即可显示该窗口，编译调试窗口选项前选中标记将显示。

ASM 编译

汇编

在 ASM 运行环境下，选择菜单栏中的“ASM 编译”菜单，选择汇编选项则程序对当前 ASM 源文件进行汇编，编译调试窗口中输出汇编结果，若程序有错，则详细报告错误信息。

汇编+链接

在当前运行环境下，选择菜单栏中的“ASM 编译”菜单，选择汇编+链接选项则程序对当前 ASM 源文件进行汇编与链接，编译调试窗口中输出汇编与链接的结果，若程序汇编或链接有错，则详细报告错误信息。

汇编+链接+运行

在当前运行环境下，选择菜单栏中的“ASM 编译”菜单，选择汇编+链接+运行选项则程序对当前 ASM 源文件进行汇编与链接，编译调试窗口中输出汇编与链接的结果，若程序汇编或链接有错，则详细报告错误信息。若汇编与链接成功，程序自动运行。

3.2 程序的调试和运行

1) ASM 程序的调试

寄存器窗口

在当前运行环境下，选择工作区的“寄存器”菜单，寄存器窗口即可显示。寄存器窗口中显示主要的寄存器名称及其在当前程序中的对应值，若值为红色，即表示当前寄存器的值。调试时，单步执行，寄存器会随每次单步运行改变其输出值，同样以红色显示。

开始调试

程序的编译和链接成功之后，调试工具将会显示，也可以在“项目”中选择“开始/结束调试”，即可开始进行程序的调试。编译选项选择如图：

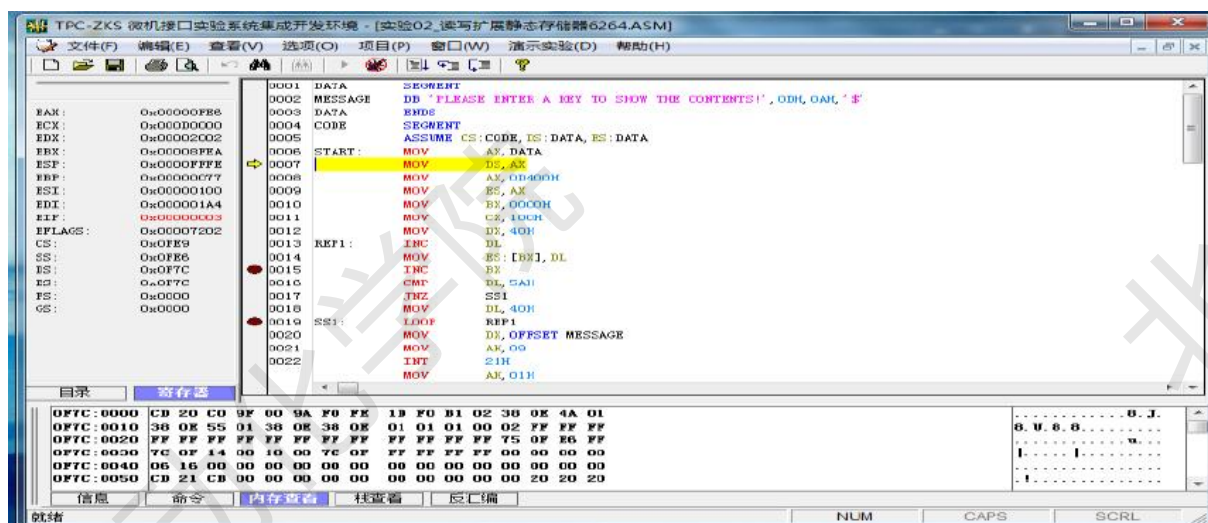


图 1.11 TPC-ZK-II 程序调试状态界面

在 ASM 程序正常链接之后，选择菜单栏中的“开始/结束调试”菜单，选择开始调试选项，则对源程序进行反汇编，进入 ASM 的调试状态，并在寄存器窗口中显示主要的寄存器的当前值。

设置/清除断点

在 ASM 的调试状态下，对程序代码所在某一行前的灰色列条单击鼠标，即对此行前设置了断点，如果清楚断点，只需要再在此行前的灰色列条上的断点单击鼠标，此断点标记将被清除。黄色箭头所指的行为当前单步执行到的所在行。

连续运行

在 ASM 的调试状态下，选择“项目”菜单栏中的“连续运行”菜单或 F5，则程序连续运行，直至碰到断点或程序运行结束。

单步

在 ASM 的调试状态下，选择“项目”菜单栏中的“单步执行”菜单或 F1，则程序往后运行一条语句。

退出调试

在 ASM 的调试状态下，选择“项目”菜单栏中的“开始/结束调试”菜单，程序则退出 ASM 的调试状态。

命令调试

集成开发环境可以进行命令的调试，如图 1.12:

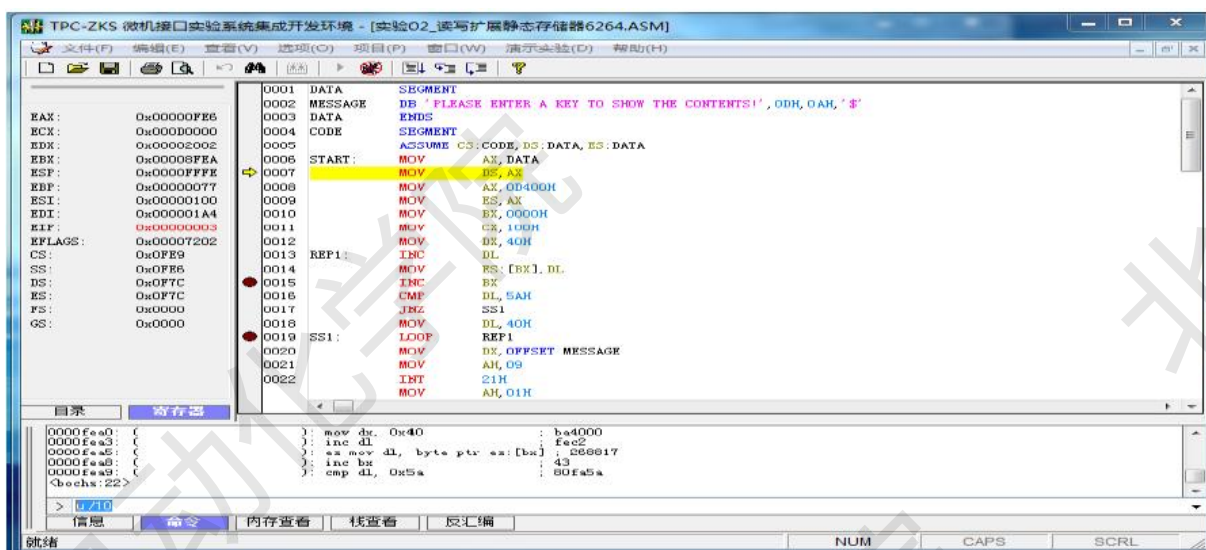


图 1.12 TPC-ZK-II 命令调试

调试时，输出窗口可以输出编译信息、命令信息、内存查看信息、栈查看信息等。

4 课前预习一

根据给出的汇编语言源程序，在 TPC-ZK-II 集成环境中输入后，进行编译和链接，得到可执行文件。然后即可使用 TPC-ZK-II 集成环境的调试功能进行单步执行。

所谓**调试**，是将编制的程序投入实际运行前，用手动或编译程序等方法进行测试，修正语法错误和逻辑错误的过程，是保证计算机信息系统正确性的必不可少的步骤。单步执行是调试程序的一种方式，可以一步步跟踪程序的运行，查看每一条指令对 CPU 寄存器和存储单元的影响，从而理解典型 8086 指令的执行过程，对指令执行和系统运行建立感性认识，理解 8086CPU 的基本结构和组成原理，实践所学习的 8086 汇编语言的常用指令。

在 TPC-ZK-II 集成环境中对图 1.13 给出的程序进行编译、链接、进入到调试程序状态，单步执行程序或者设置断点执行程序，观察、记录以下数据（截图并文字说明）并回答问题。

- 1) 生成 LIST 文件，观察各项数据、指令经过编译后，对应的偏移地址和机器码。查找并记录标号 (3)、(12) 处指令的偏移地址和机器码。
- 2) 标号 (1)~(2) 之间为程序初始化部分。执行 (1) 之前和执行 (2) 之后，CPU 内部寄存器哪些发生了变化，其内容是什么？数据段和堆栈段的起始地址是多少，内容是什么（内容可截图）？
- 3) 标号 (3) (4) 处的指令各自执行完后，AX 的内容是多少？(4) 执行完后，标志寄存器 PSW (FR) 中各项标志的值是多少，为什么？标号 (5) 处的指令执行完后，PSW (FR) 中各项标志的值是否发生改变，DT4 内容是多少？
- 4) 标号 (6) 的指令执行前后，AL 和 BL 中的值各为多少？标号 (6) 的指令执行后，标志寄存器 PSW (FR) 中各项标志的值是多少？标号 (7) 处的指令执行完后，DT1 内容是多少？
- 5) 标号 (9) 和 (10) 处两条指令，会执行哪个指令，为什么？指令执行后堆栈、SS、SP 会发生什么变化？执行 (11) 处的 POP 指令后，堆栈、SS、SP、DX 有哪些变化？
- 6) 执行标号 (12) (14) 处的 CALL 和 RET 指令，对堆栈、SS、SP 有何影响？
- 7) 标号 (13) 处的指令功能是什么，执行前后 DX 的内容有何变化？
- 8) 执行 INT 21H（标号 (15) 处的指令），堆栈、SS、SP 有何变化？

9) 若将(8)后面的 JA NEXT1 改为 JG NEXT1, 程序执行会发生什么变化, 为什么?

<pre> DATA SEGMENT DT1 DB 37 DT2 DB 64H DT3 DW 0ABCDH DT4 DW 8765H DT5 EQU 0A7H DATA ENDS STKS SEGMENT STACK DW 100 DUP(0) STKS ENDS CODE SEGMENT ASSUME CS:CODE, DS:DATA, SS:STKS MAIN PROC FAR ;初始化及数据段、堆栈段观察记录 MOV AX, STKS (1) MOV SS, AX MOV SP, 100*2 MOV AX, DATA MOV DS, AX (2) ;加减法、PSW 实验 MOV AX, DT3 (3) ADD AX, DT4 (4) MOV DT4, AX (5) </pre>	<pre> MOV AL, DT1 MOV BL, DT2 SUB AL, BL (6) MOV DT1, AL (7) ;比较、条件转移、CALL 指令以及堆栈实验 CMP AX, DT3 (8) JA NEXT1 PUSH AX (9) JMP NEXT2 NEXT1: PUSH DT3 (10) NEXT2: POP DX (11) CALL SPRC (12) MOV AH, 4CH INT 21H (15) MAIN ENDP SPRC PROC NEAR MOV CL, 3 ROL DX, CL (13) RET (14) SPRC ENDP CODE ENDS END MAIN </pre>
---	---

图 1.13 预习作业使用的汇编程序

注意事项

1. 由于没有连接实验箱, 所以 TPC-ZK-II 集成开发环境对程序进行编译连接后, 运行时 would 报错。需要在弹出的报错对话框中选择 “continue and don’t ask again” 才能继续执行。

2. TPC-ZK-II 集成开发环境的一些注意事项:

- 1) 不要在程序编辑框内缩放字体，否则会造成调试时光标和实际运行的指令不对应。
- 2) 代码段段名只能用 CODE，否则无法正常调试。
- 3) INT 21H 的 2 号功能调用会改变 AL 的值，在使用前后需要把 AX 进行出入栈保护。
- 4) 标志寄存器的 OF 位显示结果不正确。
- 5) 数据段定义表名不能够用 TABLE。
- 6) 源程序第一条可执行指令前使用标号"START:",源程序最后使用伪指令"END START"。

5 课前预习二

按照以下 1~7 的要求完成练习。有关程序编写、运行和截图的内容需要使用 TPC-ZK-II 集成环境来完成。

- 1) 设有数据如下：

```
DT1  DB  1,2,'AB'  
COUNT EQU $-DT1  
DT2  DW  1,2,'AB'  
DT3  DW  DT2  
DT4  DW  $-DT2  
SN    DB  8 DUP(?)  
NAM   DB  30 DUP(?)
```

试定义数据段，将上述数据定义在数据段中。

- 2) 编写汇编程序，完成 DS 初始化，使用 INT21H 的 2 号功能调用将 DT1 后面定义的数字 1 和 'A' 显示到计算机屏幕上（即在显示器上显示 1 和 A，截图）。
- 3) 在 TPC-ZK-II 集成环境的调试状态下执行程序，截图数据段，画出 DT1~DT4 定义的各项数据在内存的存储顺序。同时标明各个地址单元的偏移地址：
- 4) 回答以下问题：
 - ① COUNT 是否在数据段中有对应的存储单元？
 - ② DT1 和 DT2 有何不同，是什么原因造成这种不同？

③ DT4 的值是多少，有何含义？如果自己计算，如何计算？

- 5) 完善程序，使得程序能够从键盘输入自己的学号和姓名拼音，分别存储到 SN 和 NAM 中。要求：使用 INT 21H 的 1 号功能从键盘输入自己的学号；使用 INT 21H 的 0AH 功能调用，输入自己姓名拼音（可对 NAM 的定义进行适当修改）
- 6) 完善程序，使用 INT21H 的 2 号功能输出显示 SN 中的学号，使用 09 号功能显示 NAME 中的姓名拼音（显示结果截图）

7) 思考题

1. 返回 DOS 语句的作用是什么，如果缺失会有什么后果？
2. 在定义了多个逻辑段的程序中，同时有效的逻辑段最多有几个？
3. 在程序中，哪一类逻辑段必不可少？

第二部分微机原理及接口技术实验指导

实验一 并行接口实验

一、实验目的

1. 学习和理解接口芯片的地址译码电路,掌握 IO 译码电路设计;
2. 熟练掌握并行接口芯片 8255A 的基本原理、初始化设置、工作方式 0 的应用;
3. 学习和掌握数码管动态显示数字的原理;
4. 对比 EDA 软件设计与虚拟仿真和实验箱实物实验的异同。

二、实验内容

1. 实验内容 1 (**必选**): 使用实验箱搭建电路,编写程序从键盘输入两个数字并显示在实验箱的两个七段码数码管上。
2. 实验内容 2 (**选做**): 完成“实验六 并行接口设计实验”,使用 Proteus 软件设计并行接口电路及其地址译码电路、编写程序、运行和仿真,与实验内容 1 进行对比。
3. 实验内容 3 (**选做**): 在实验内容 2 的基础上,搭建 4*4 数字键盘,编写程序通过 8255A 采集键盘输入,并在数码管上显示。

*注: 若选做实验内容 2、3, 则与实验内容 1 和合并写一份实验报告提交。

三、实验设备

1. 计算机 (Windows 7 32 位操作系)
2. TPC-ZK-II 集成开发环境 (第一部分第 3 节 实验系统软件介绍)
3. TPC-ZK-II 实验箱

四、预习要求

1. 学习《微机原理及接口技术》中可编程并行接口芯片 8255A 相关内容;
2. 编制程序流程图, 画出硬件原理图;
3. 预习要求
 - 1) 计算采用共阴极七段数码管显示的 0 到 9 的字形码 (十六进制数表示);

- 2) 编写键盘输入数字程序/子程序,要求在输入过程中检查是否是数字键,不是则不显示键入数字并重新输入;
- 3) 编写显示器显示键入数字程序/子程序,要求当输入正确的数字后能显示键入数字;
- 4) 编写延时程序/子程序,适当设置延时时间,保证两位数码管正确、稳定、可靠地显示键入的两位数字。

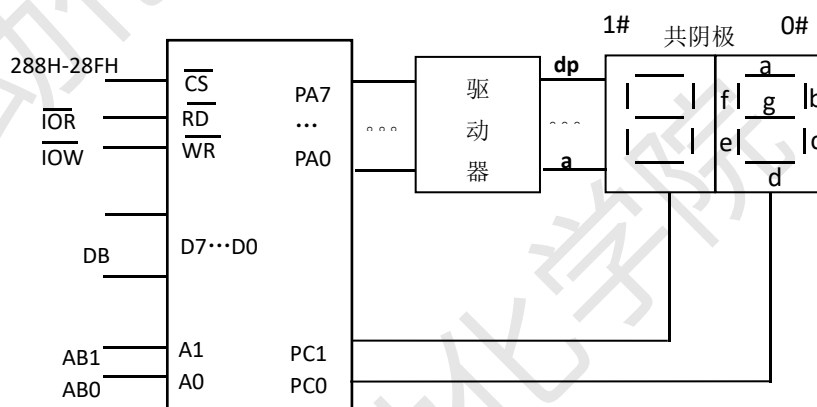


图 2.1.1 实验原理图

五、实验原理

电路原理图如图 2.4.1 所示。地址译码器输出 (288H~28FH) 片选信号, 连接 8255A 的 CS 引脚。

8255A 的 PA0~PA6 分别连接数码管的 LED 灯 a~g 的阳极; PC0 连接 0#数码管的 LED 灯 a~g 的阴极, PC1 连接 1#数码管的 LED 灯 a~g 的阴极。

实验箱上的七段数码管为共阴型, 阴极输入端加低电平, 选中的数码管亮。

8255A 的 PA 端口引脚、PC 端口的 PC0、PC1 即可控制数码管中 LED 灯的亮和灭。例如, 设 PA1、PA2 输出高电平 (逻辑 1), 其它 PA 端口引脚输出低电平 (逻辑 0), 若 PC0 输出低电平 (逻辑 0), 则 0#数码管的 LED 灯 b、c 点亮, 显示数字 “1”; 若此时 PC1 输出高电平 (逻辑 1), 则 1#数码管不显示。

设 PA0、PA1、PA2 输出高电平 (逻辑 1), 其它 PA 端口引脚输出低电平 (逻辑 0), 若 PC1 输出低电平 (逻辑 0), 则 1#数码管的 LED 灯 a、b、c 点亮, 显示数字 “7”; 若此时 PC0 输出高电平 (逻辑 1), 则 0#数码管不显示。

当 PA 端口输出两个不同的数值编码, 同时 PC0、PC1 交替输出 0 和 1, 则两

个数码管交替显示不同的数字。当交替显示的频率非常快的时候，仿佛两个数码管同时显示两个数字。

七段数码管的字型代码表如图 2.4.2 所示：

显示字形	g	f	e	d	c	b	a	段码
0	0	1	1	1	1	1	1	3FH
1	0	0	0	0	1	1	0	06H
2	1	0	1	1	0	1	1	5BH
3	1	0	0	1	1	1	1	4FH
4	1	1	0	0	1	1	0	66H
5	1	1	0	1	1	0	1	6DH
6	1	1	1	1	1	0	1	7DH
7	0	0	0	0	1	1	1	07H
8	1	1	1	1	1	1	1	7FH
9	1	1	0	1	1	1	1	6FH
A	1	1	1	0	1	1	1	77H
B	1	1	1	1	1	0	0	7CH
C	0	1	1	1	0	0	1	39H
D	1	0	1	1	1	1	0	5EH
E	1	1	1	1	0	0	1	79H
F	1	1	1	0	0	0	1	71H

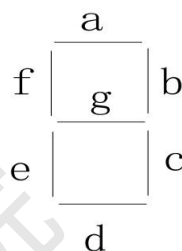


图 2.1.2 字型代码（七段码）表

编程提示：

- 1) 在数据段中存放 0 到 9 的字形码；
- 2) 从微机键盘输入 2 个数字的 ASCII 码，在输入过程中检查如非数字键则重新输入；
- 3) 然后将输入的 ASCII 码变成相应的数字，再利用换码指令 XLAT 查表得到相应的字形码；
- 4) 将字形码送到 8255 输出口所接的数码管上显示。

六、实验步骤

1. 硬件连接

连接地址译码器输出与 8255 的片选信号，以及 8255 的与数码管的连线，如图 2.2.3。

按图 2.4.3 连接电路，将 8255 的 A 口 PA0~PA6 分别与七段数码管的段码驱动输入端 a~g 相连（使用排线），位码驱动输入端 S1，S0 接 8255 C 口的 PC1，PC0。片选信号 CS 接地址译码器输出的 288H~28FH 端。

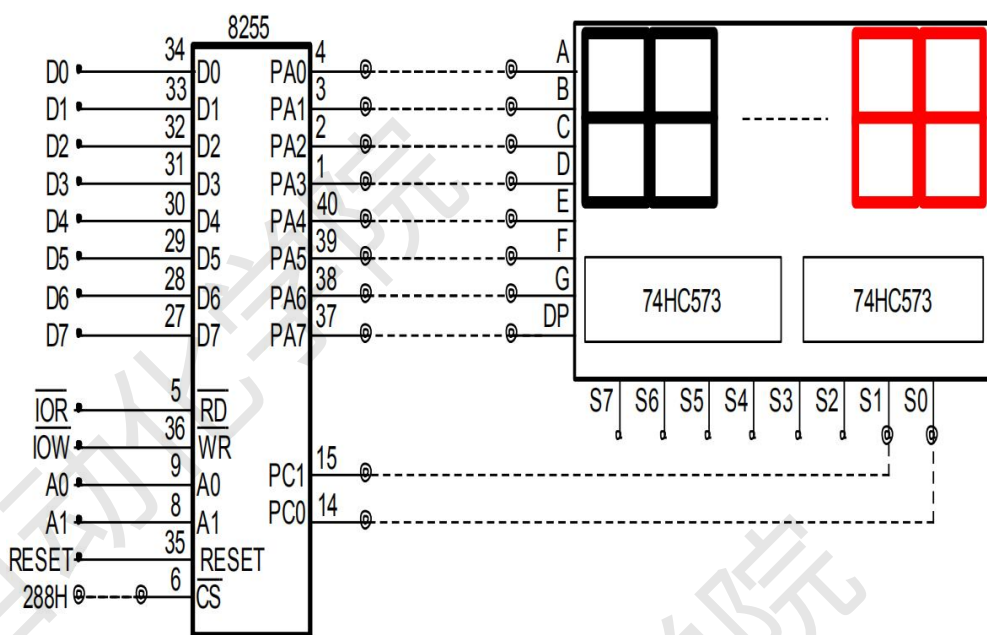


图 2.1.3 8255A 连线图

PA0 ——a, PA1 ——b, PA2 ——c, PA3 ——d,
 PA4 ——e, PA5 ——f, PA6 ——g, dP ——PA7
 PC0 ——S0 , PC1 ——S1

2. 连接实验箱与计算机；
3. 请老师检查硬件连接；
4. 硬件检查通过后，开启计算机；
5. 打开运行 TPC-ZK-II 集成开发环境；

在 TPC-ZK-II 集成开发环境运行界面下建立或打开已编写好汇编语言源程序文件，对程序进行编译、连接、运行。

6. 运行程序，从键盘输入两位数字，观察数码管上的数据显示。

若数码管结果显示正确，则保存实验程序和实验结果，让老师检查，回答老师提出的关于实验的相关问题，通过后关闭电脑、收拾实验箱后即可离开实验室；

七、注意事项

1. 实验要求的所有连线连接好并让老师检查确认后，方可用 USB 线连接实验箱和计算机并打开实验箱电源开关。
2. 实验箱的电源开关有两个，分别位于右侧和上方；
3. 计算机必须连接实验箱后，才能够在 TPC-ZK-II 集成开发环境中完成编译连接运

行整个过程。没有连接实验箱的计算机 TPC-ZK-II 集成开发环境中只能对程序进行编译连接，运行时会报错。需要按照下面方法运行程序：

计算机任务栏左侧开始-运行-输入 `cmd`-打开 `command` 窗口，指明之前保存源程序文件的文件夹路径，输入可执行文件名，回车即可运行可执行文件查看实验结果是否正确。

4. 因为公共实验室的计算机都装有防病毒卡，所以请将程序保存在非系统盘内，否则计算机意外关机或重启后所作的工作都将丢失。

5. 爱护实验设备，在接有实验箱的实验过程中，一定按实验要求操作，严禁带电接线或拆线。

6. 上实验课前,要求准备上机实验程序，结合课堂讲授内容阅读实验指导；实验中要求记录实验过程和结果，按时写出实验报告。

7. 实验完毕后 ,实验结果经老师确认后方可拆线，并将实验箱及接线按原样整理好。

8. 实验过程中如发生事故，应立即关断电源，保持现场，报告指导老师。若不按要求操作，损坏了实验设备，根据损坏情况赔偿。

9. 实验中注意安全，保持环境整洁、安静。

八、实验报告要求

实验报告中应包括以下内容：

- 1.实验目的
- 2.实验原理
- 3.实验内容与实验过程描述
- 4.实验结果分析与实验结论
- 5.收获、体会及建议
- 6.预习思考题及实验验证
- 7.实验后思考题
- 8.实验源程序

实验报告模板可参照附件 1。

九、课后思考题

1. DOS 功能调用中，键盘输入字符的方式有几种，有何不同？每种方式各适合于何种应用场合？

2. 数码管显示的原理是什么？若采用共阳极数码管，应如何修改程序？

3. 两位数码管同时显示不同数字的机理是什么？对延时时间有何要求？

4. 是否可以用 8255 的 B 口 PB0~PB6 分别与七段数码管的段码驱动输入端 a~g 相连，若可以，应如何修改程序？

十、参考资料

1. 微型计算机原理与接口技术，周荷琴，吴秀清，中国科技大学出版社。

2. 微机原理及接口技术实验指导书，《微机原理及接口技术》课程教学团队，北京航空航天大学。

实验二 数字音频实验（定时器与并口综合设计实验）

一、实验目的

1. 学习使用两种及以上接口芯片实现复杂功能
2. 学习产生数字音频的简单方法

二、实验内容

1. 实验内容 1（**必选**） 编程使计算机的数字键 1、2、3、4、5、6、7 作为电子琴按键，按下即发出相应音阶。
2. 实验内容 2（**必选**） 编写程序，运行时发出一段旋律。
3. 实验内容 3（**选做**） 完成“实验七 定时器与数字音频设计实验”，在 Proteus 软件中搭建电路原理图，编写程序并仿真运行。

三、实验设备

1. 计算机（Windows 7 32 位操作系统）
2. TPC-ZK-II 集成开发环境
3. TPC-ZK-II 实验箱

四、预习要求

1. 复习并行接口芯片 8255A 和定时器 8253/8254 芯片工作原理、工作模式特点和编程设置。
2. 实验课之前自主准备和完成实验所用的汇编程序，以便实验课上调试和运行。
3. 预习思考题
 - 1) 电路原理图中与门的作用是什么，能否去掉该与门，直接将 8254 输出的音频信号加载到扬声器？

五、实验原理

1. 电路原理图

实验电路如图所示。

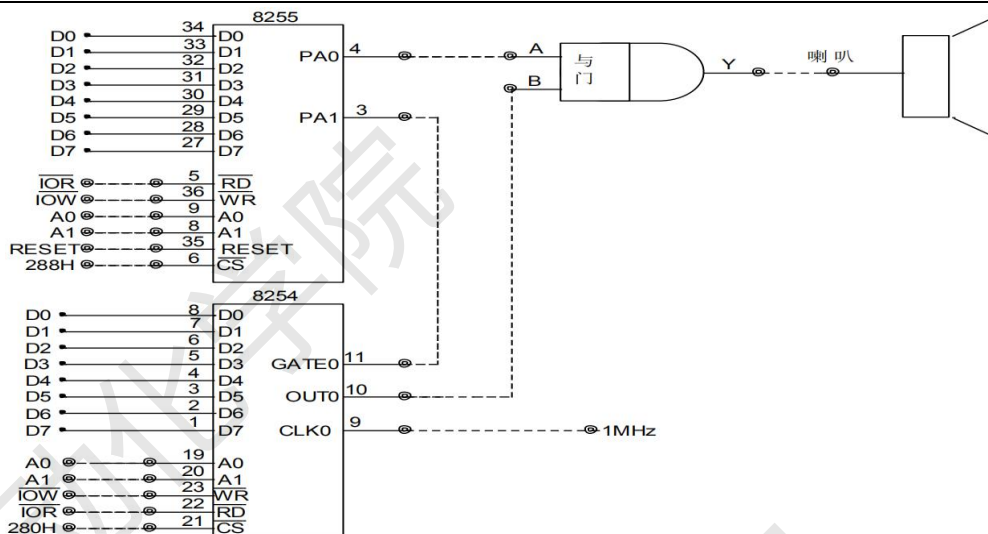


图 2.2.1 电路原理图

接线:

8255/CS 接 I/O 地址译码/Y1(288H--28FH)

8255/PA0 接 与门/A

8254/CLK0 接 时钟/1MHz

8254/CS 接 I/O 地址译码/Y1(280H--287H)

8254/OUT0 接 与门/B

8254/GATE0 接 8255/PA1

与门/Y 接 喇叭

2. 定时器 Intel8253/8254

可编程定时器 Intel8254 是 8253 的增强型，引脚完全兼容，功能几乎完全相同。

区别在于:

1) 8253 最大输入时钟频率是 2MHz，而 8254 是 5MHz，8254-2 是 10Mhz

2) 8254 有读回 (readback) 功能，可以同时锁存 1~3 个计数器的计数值及状态值，供 CPU 读取，而 8253 每次只能锁存和读取一个通道的计数器，而且不能读取状态值。

Intel8253/8254 是一种可编程计数器/定时器芯片 (可编程间隔定时器, PIT: Programmable Interval Timer)。8253/8254 内部有 3 个独立的 16 位计数器通道，通过对它进行编程，每个计数器通道都可以按 6 种不同的方式工作，并且可以按 2 进制或 10 进制格式进行计数，最高计数频率 2MHZ。8253/8254 还可以作为方波发生器、分频器、程控单脉冲发生器等等。

(1) 工作方式设置/控制字

在 8253/8254 工作之前，必须对它进行初始化编程，也就是向 8253/8254 的控制字寄存器写入一个控制字和向计数器赋计数初值。控制字的功能是：选择计数器，确定对计数器的读/写格式，选择计数器的工作方式以及确定计数的数制。8253/8254 控制字的格式如图所示。

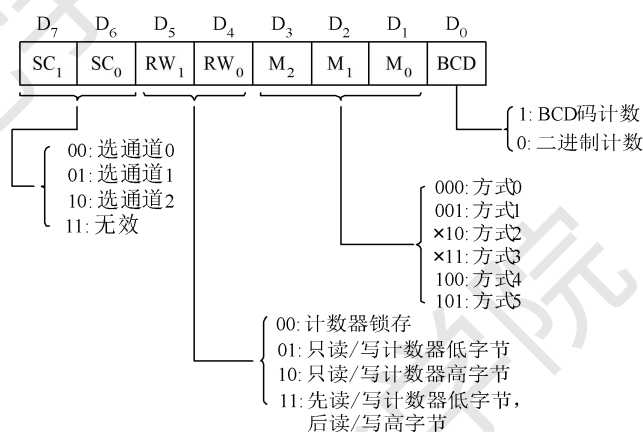


图 2.2.2 8253/ 8254 控制字

(2) 读写操作

1) 8253/8254 的写操作包括写控制字和写计数初值两项内容。具体要求：

- 各计数器的控制字都写到同一地址单元，而各计数初值写到各自的地址单元中。
- 对于每个计数器，必须先写控制字，后写计数初值。因为后者的格式是由前者决定的。
- 写入的计数初值必须符合控制字决定的格式。16 位数据应先写低 8 位，再写高 8 位。
- 当给多于一个的计数器写入控制字和计数初值时，其顺序有一定的灵活性，只要遵循上述要求即可。

2) 8253/8254 的读操作所得到的是当前计数值，通常用于实时检测、实时显示和数据处理。在进行读操作时需要注意以下几点：

- 读操作是通过访问对应于各计数器的地址单元来实现。
- 每个计数器的读操作必须按照控制字确定的格式。如果是 16 位计数，读操作要进行两次，先读低 8 位，后读高 8 位。

当计数器为 16 位时，为了避免在两次读出过程中计数值的变化，要求先将计数值锁存。锁存计数值的常用方法是使用计数器锁存命令：控制字的 D7D6 两位为所要锁存的计数值，D5D4 两位置为 00。8253/8254 的每个计数器都有一个输出锁存器（16 位），平时它的值跟随计数值而变化。当向计数器写入锁存命令后，现行计数值被锁存（计数器仍能继续计数）。这样 CPU 读取的就是锁存器中的值。当 CPU 读取了计数值或对计数器重新编程以后，锁存状态被解除，输出锁存器的值又随计数值变化。

3. 音阶频率

利用 8255A 的 PA0 口输出控制信号给与门，用来控制扬声器开关状态。再利用设置不同的计数值，使 8254 产生不同频率的方波，输出给扬声器产生不同频率的音调，达到类似音阶的高低音变换。音乐的每个音阶的标成频率值：

音阶	1	2	3	4	5	6	7	i
低频率 (Hz)	262	294	330	347	392	440	494	524
中频率 (Hz)	524	588	660	698	784	880	988	1048
高频率 (Hz)	1048	1175	1318	1397	1568	1760	1976	

当
键
盘

按下数字按键时，程序计算相应的频率和 8254 时间常数，并对 8254 计数初值进行赋值，控制 8255A 使得 8254 输出频率到扬声器，发出数字键对应的声调。

在扩展实验中，需要事先定义所播放旋律的音阶、节拍，其中节拍可以通过不同的时间延迟程序来实现，也可以通过定时器其它通道的计数输出给 8255A，通过查询 8255A 引脚状态来实现。

4. 编程提示

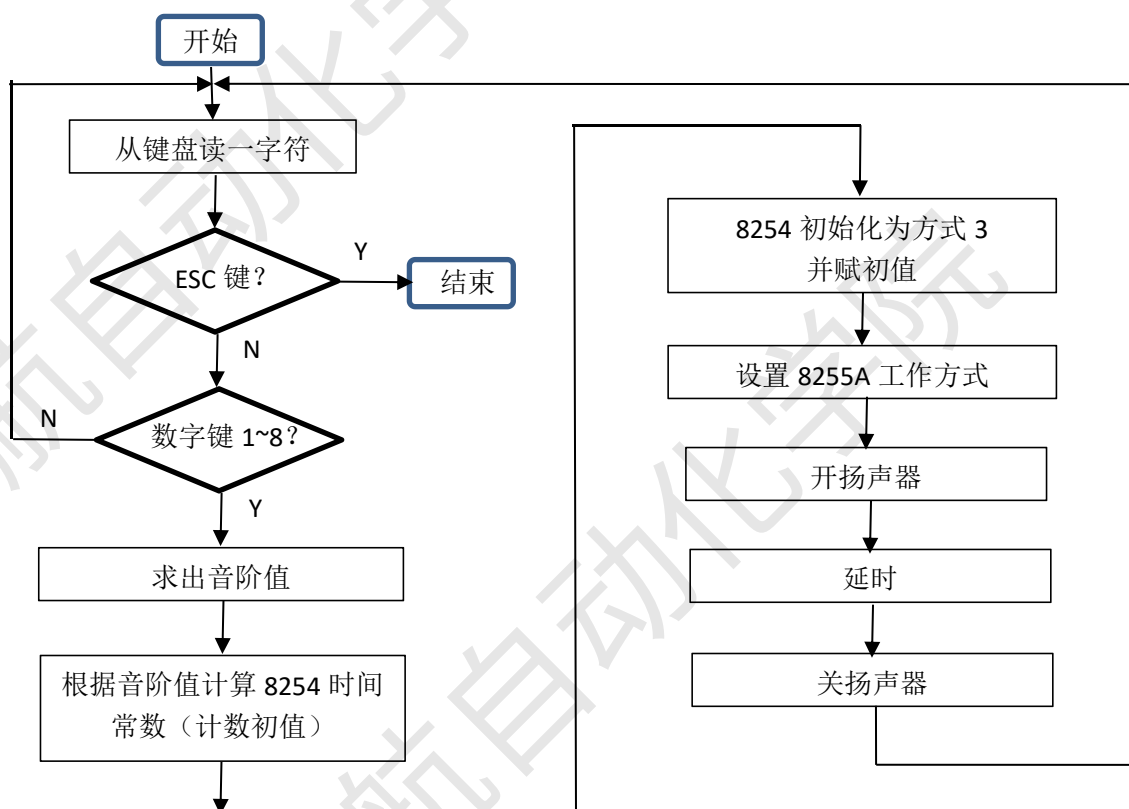


图 2.2.3 程序流程图

六、实验步骤

1. 打开计算机，进入 windows XP 操作系统。
2. 领取实验箱，检查实验箱是否插槽、电路、电源插座是否完好。
3. 将实验箱中的电位计输出端 U1（或者 U2）连至 ADC0809 的 IN0 端；
4. 将 8254 的 CS 端连接到实验箱中的 280H~287H 端口；
5. 用 USB 线将实验箱和计算机连接；
6. 连接实验箱电源；
7. 检查上述连接无误后，打开电源开关；
8. 运行 TPC-ZK-II 集成开发环境；

9. 在 TPC-ZK-II 集成开发环境中打开事先编写的汇编程序；
10. 编译、运行程序。如果出现错误则进行调试；
11. 显示运行结果。

七、注意事项

1. 实验要求的所有连线连接好并让老师检查确认后，方可用 USB 线连接实验箱和计算机并打开实验箱电源开关。

2. 实验箱的电源开关有两个，分别位于右侧和上方；

3. 计算机必须连接实验箱后，才能够在 TPC-ZK-II 集成开发环境中完成编译连接运行整个过程。没有连接实验箱的计算机 TPC-ZK-II 集成开发环境中只能对程序进行编译连接，运行时会报错。需要按照下面方法运行程序：

计算机任务栏左侧开始-运行-输入 cmd-打开 **command** 窗口，指明之前保存源程序文件的文件夹路径，输入可执行文件名，回车即可运行可执行文件查看实验结果是否正确。

4. 因为公共实验室的计算机都装有防病毒卡，所以请将程序保存在非系统盘内，否则计算机意外关机或重启后所作的工作都将丢失。

5. 爱护实验设备，在接有实验箱的实验过程中，一定按实验要求操作，严禁带电接线或拆线。

6. 上实验课前,要求准备上机实验程序，结合课堂讲授内容阅读实验指导；实验中要求记录实验过程和结果，按时写出实验报告。

7. 实验完毕后 ,实验结果经老师确认后方可拆线，并将实验箱及接线按原样整理好。

8. 实验过程中如发生事故，应立即关断电源，保持现场，报告指导老师。若不按要求操作，损坏了实验设备，根据损坏情况赔偿。

9. 实验中注意安全，保持环境整洁、安静。

八、实验报告要求

实验报告中应包括以下内容：

- 1.实验目的
- 2.实验原理

3.实验内容与实验过程描述

4.实验结果分析与实验结论

5.收获、体会及建议

6.预习思考题及实验验证

7.实验后思考题

8.实验源程序

实验报告模板可参照附件 1。

九、课后思考题

1. 本实验所使用的程序，是否有必要每次向扬声器输出音频的时候都设置一遍 8255A、8254 的工作方式，为什么？
2. 程序中的延时功能，除了使用软件方式之外，也可以利用硬件来实现。试提出硬件实现的思路。

十、参考资料

1. 微型计算机原理与接口技术，周荷琴，吴秀清，中国科技大学出版社。
2. 微机原理及接口技术实验指导书，《微机原理及接口技术》课程教学团队，北京航空航天大学。



实验三 数/模与模/数转换

一、实验目的

1. 通过实验了解数模转换的原理，使用典型的数/模转换芯片 DAC0832，在单缓冲工作方式下，通过编写汇编程序来控制数/模转换，理解不同输出电路连接方式下电压输出值（单极性、双极性）与输出数字量之间的对应关系。；掌握 $\overline{ILE}$ ， $\overline{CS}$ 以及 $\overline{WR1}$ 信号对输入寄存器的控制作用，掌握 $\overline{WR2}$ 和 $\overline{XFER}$ 信号对 DAC 寄存器的控制作用；
2. 建立模数转换的感性认识，加深对连续模拟量转换为离散数字量的理解；掌握 ADC0809 的转换使用方法，包括模数转换的启动、等待转换结束、读取转换结果整个过程；编写汇编程序对 AD 转换后的数字量进行处理并显示，提高微机原理与接口技术课程的综合应用能力。
3. 通过实验加深理解过程控制系统中计算机与外界进行数据交互的过程。

二、实验内容

1. 实验内容一（必选）：数/模转换

方波和正弦波是应用中常见的两种信号源，实验利用数/模转换芯片 DAC0832 产生方波及正弦波。

DAC0832 采用单缓冲方式，输出端具有单双极性两种方式。编写程序，在数据段中存放好方波和正弦波的数字量（正弦波要求 20 个值），执行程序将数据段中的数字量送到 DAC0832 的输出端产生方波和正弦波。

DAC0832 对应的端口地址为 290H~297H。

2. 实验内容二（必选）：模/数转换

编写程序对 ADC0809 的 IN0 通道的模拟量进行模数转换。包括：ADC 输出端口号定义，逻辑段定义，程序初始化，启动 ADC0809，延时子程序，转换数据读取；

将模数转换得到的数字以合适的方式显示在微机屏幕上。显示的数字可以是以下一种或多种方式：16 进制数（必选）、二进制、十进制数或电压值。

程序应当能够连续显示转换后的数字量，可以在以下几种方法中选择其中一种：

1) 间隔显示。每隔一段时间即触发一次 AD 转换, 读取转换后的数据并进行显示。

2) 使用键盘按键来控制下一次的显示。例如: 每按一次空格键, 就启动一次 AD 转换并将转换后的数据显示在屏幕上, 当按下 ESC 键后程序退出。

3) 每隔一段时间启动 AD 转换, 读取转换后的数据。如果读取的数据与上次相同则不显示, 如果不同则显示。

3. 实验内容三 (选做):

(1) 在 Proteus 中使用 DAC0832 设计搭建 D/A 电路, 编写程序输出三角波、锯齿波、方波和正弦波, 并在数字示波器中显示;

(2) 在 Proteus 中使用 ADC0808 设计搭建 A/D 电路, 使用可调电阻输出 0~5V 电压, 经过 AD 转换后将电压值显示在数码管上。

三、实验设备

1. 计算机 (Windows 7 32 位操作系统)

2. TPC-ZK-II 集成开发环境

3. TPC-ZK-II 实验箱

四、预习要求

1. 了解数模转换器 DAC0832 的基本工作原理; 了解 DAC0832 在单缓冲方式下, 程序控制输出数字量的方法; 掌握不同的电压输出端输出电压与输出数字量的关系。

2. 了解模数 ADC0809 的工作原理, 内部结构和引脚定义。重点掌握 ADC0809 启动 AD 转换、等待转换、转换结束整个过程中 ALE、START、ADDA、ADDB、ADDC、EOC、OE 等引脚信号的时序要求。

3. 复习 IN 指令和 OUT 指令的使用方法。尤其需要重点掌握执行 IN 指令和 OUT 对 8086CPU 引脚的影响、及其时序图。

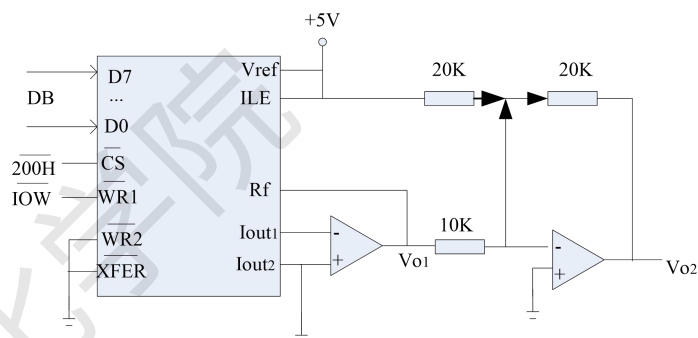
3. 掌握字符 '0' ~ '9'、'A' ~ 'F'、ESC 和空格的 ASCII 码值。

4. 编制程序流程图, 写出程序。

5. 预习思考题

1) 对于 8 位 D/A 转换器, 假设要转换的 8 位数字量为 $D_7 D_6 D_5 D_4 D_3 D_2 D_1 D_0$, 请

写出单极性端的输出电压 V_{O1} 与数字量的对应关系？双极性端的输出 V_{O2} 电压为多少？



2) 显示 ASCII 码字符的 DOS 功能调用有哪些，它们的区别是什么？

2) 如果试图将一个字节的二进制数分别以二进制数和十六进制方式显示在计算机屏幕上，需要做哪些处理？（提示：使用移位或循环移位指令）

3) 如何将一个字节的二进制数转换为十进制数显示出来？（提示：可以使用除法指令）

五、实验原理

（一）数模转换

1. 实验电路连线

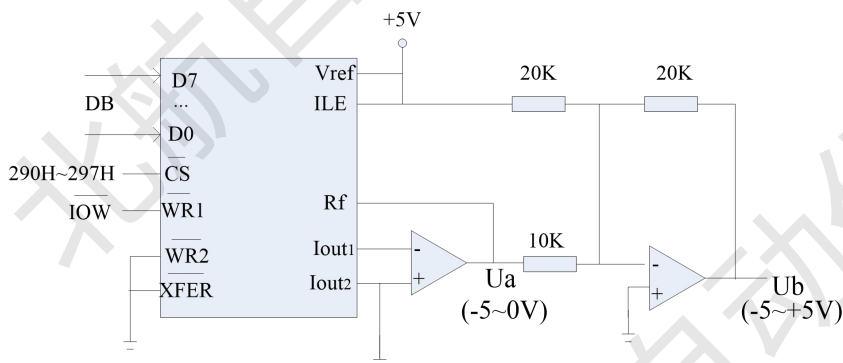


图 2.3.1 数/模转换实验电路连线图

图 2.3.1 为数/模转换实验电路连线图，DAC0832 采用单缓冲方式，具有单双极性输出端（图中的 U_a 为单极性、 U_b 为双极性）。

片选段 $\overline{CS}$ 连接地址译码器对应 290H-297H 的输出端，当地址信号为 290H-297H 时，DAC0832 的 $\overline{CS}$ 端被选通，为低电平有效。

2. 编程提示

1) 产生方波信号的数字量;

要产生方波输出信号,在数据段中存放方波的数字量,要使方波输出在 8 位 DA 的数字量范围 (0~255) 内。取两个点分别作为方波的高电平和低电平输出。若在 Ua 端产生方波,则高电平取数字量 0 (0V),低电平取数字量 255 (-5V);若在 Ub 端产生方波,则高电平取数字量 255 (+5V),低电平取数字量 0 (-5V)。

2) 产生正弦信号的数字量;

要产生正弦波的输出信号,需要从一个周期中取多个点输出。下面以 20 个点为例来说明。

假设 $Y = \sin n \frac{2\pi}{20}$, $n=0-19$, 为了使输出在 8 位 DA 的数字量范围 (0~255) 内,对 Y 进行放大和上移。设 $Y = 80H + 80H \times \sin n \frac{2\pi}{20} = 80H (1 + \sin n \frac{2\pi}{20})$, 列出 $n=0-19$ 时 Y 的输出情况 (超过 FFH, 则取值 255)。

N	0	1	2	3	4	5	6	7	8	9
Y	128	168	203	232	250	255	250	232	203	168
N	10	11	12	13	14	15	16	17	18	19
Y	128	88	53	24	6	0	6	24	53	88

1) 把要输出的数据放在数据段中;

2) 循环输出数据,以便显示方波或者正弦波;程序中可通过检测键盘按键等方法退出循环结束程序。

（二）模数转换

1. 实验硬件连线

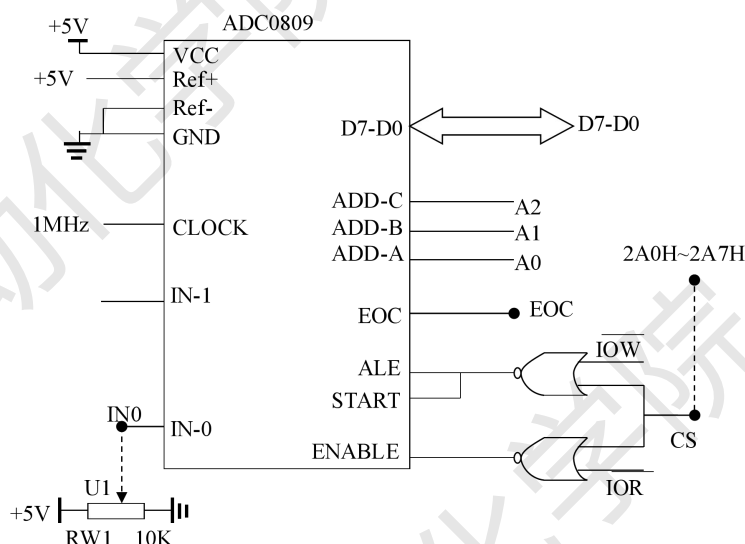


图 2.7.1 硬件连接图

本实验的硬件连接线较为简单，除了电源线、与电脑连接的 USB 线外，只需要连接两处：

- 1) ADC0809 的模拟量输入端 IN0 与 电位计 RW1 的电压输出端 U1（位于实验箱左下方，小键盘旁边）
- 2) ADC0809 的片选信号 CS 与 计算机端口控制端 2A0H~2A7H

2. 编程提示

1) ADC0809 转换过程

启动转换和输入转换后的数据，都是将端口号写入 DX 后直接使用 OUT 指令和 IN 指令即可。

等待转换结束有两种方法。一种是查询 ADC0809 的 EOC 状态，另一种是长时间延时。由于实验硬件连线不支持 EOC 状态查询，因此本实验采用后一种方法，通过编写和调用延时子程序 DELAY 等待 ADC0809 转换结束。

2) 数据显示算法

数据显示允许采用多种方式。

模数转换后的数据直接显示其 16 进制数值，原理如下：

转换后的数据为 8 位的数字量，需要针对高 4 位和低 4 位分别进行显示，另外也需要针对 0~9 和 A~F 做出不同的处理。

首先显示高 4 位。将模数转换后的数据逻辑右移 4 位，只保留高 4 位。然后判断此高 4 位是否大于 9。如果大于 9，则表明其数值在 A~F 之间，需要加上 37H，转换为 ASCII 码值进行输出；否则加上 30H，转换为 ASCII 码值进行输出。

然后显示低 4 位。将模数转换后的数据和 0FH 进行与操作，只保留低 4 位。按上述方法显示低 4 位。

3) 循环方法：使用键盘按键来控制循环

由于电压值需要反复读取和输出，因此程序需要循环操作。为了避免循环过快，导致数据显示快速刷新无法看清，因此需要对循环的速度和节奏加以控制。最简单的方法是使用 INT 21H 的 1 号功能调用。

每当一轮数据采集和显示结束后，调用 INT 21H 的 1 号功能，程序暂停等待键盘的输入。

如果输入的按键是 ESC 键则退出程序。否则重新开始一轮模数转换操作。

4) 延时程序

延时程序主要用于启动 ADC0809 转换后，等待转换结束。延时的原理是，每条指令的执行都需要时间，当反复执行某些指令时即可占用相当多的 CPU 时间，从而达到延时的目的。

为了加长延时时间，可以使用嵌套循环结构。

5) 其它数据显方法（可选）

✧ 显示 10 进制数

直接显示十进制数值的原理如下：

将待显示的字节数据（0~255）使用 DIV 指令除以 100，商（保存在 AL 中）即为十进制的百位数；余数（保存在 AH 中）复制到 AL 寄存器，AH 清零，再次使用 DIV 指令除以 10，得到的商即为十位数，余数为个位数。

然后将得到的百位、十位、个位数分别加上 30H，转换为 ASCII 码值进行显示输出。

✧ 显示电压值（精确到小数点后两位）

电压值显示算法的原理如下：采集到的数据 X 为 8 位自然二进制编码，范围 0~255，

对应电压为 0~+5V。

因此，根据除法计算规则，需要显示的个位数为： $I = X * 5 / 255$ ；此外还需要保持余数 $J = X * 5 \% 255$ ，以便计算小数点后面的电压数据。

小数点后面第一位数据： $K = J * 10 / 255$ ；另有余数 $M = J * 10 \% 255$

小数点后面第二位数据： $N = M * 10 / 255$

✧ 显示二进制数

模数转换后的数据显示二进制数值的原理如下：

显示数据的二进制位顺序是从左到右，即先显示高位，再显示低位。

将待显示的数据进行逻辑左移。最高位被移出后进入到了标志寄存器的 CF 位中，因此判断 CF 位，如果为 1 则跳转到显示字符‘1’的代码处，负责显示字符‘0’。

由于数据共有 8 位，因此上述移位、判断和显示操作需要循环 8 次。

✧ 其它循环方法

- ① 按一定时间间隔显示采集到的数据。使用延时程序，精确调整参数，使得延时时间在可接受的范围内。

仅显示变化的数据。在此方式下每采集一次数据，都和上次的采集结果 DTIN 进行比较，如果相同则不显示，不同才进行显示。

六、实验步骤

1. 打开计算机，进入 windows XP 操作系统。

1. 先按实验原理图连线，接着用 USB 线将实验箱和计算机连接好，最后开启电源；
2. 按实验电路接线后，双击与之连接的 PC 机上的 TPC-ZK-II 集成开发环境，编译程序；

3. 运行程序，查看结果。

4. 教师检查程序运行结果，随堂提问，根据回答问题情况和实验报告完成情况综合评估给出实验成绩。

七、注意事项

1. 实验要求的所有连线连接好并让老师检查确认后，方可用 USB 线连接实验箱和计算机并打开实验箱电源开关。

2. 实验箱的电源开关有两个，分别位于右侧和上方；
3. 计算机必须连接实验箱后，才能够在 TPC-ZK-II 集成开发环境中完成编译连接运行整个过程。没有连接实验箱的计算机 TPC-ZK-II 集成开发环境中只能对程序进行编译连接，运行时会出现报错。
4. 公共实验室的计算机都装有防病毒卡，所以请将程序保存在非系统盘内，否则计算机意外关机或重启后所作的工作都将丢失。
5. 爱护实验设备，在接有实验箱的实验过程中，一定按实验要求操作，严禁带电接线或拆线。
6. 上实验课前，要求准备上机实验程序，结合课堂讲授内容阅读实验指导；实验中要求记录实验过程和结果，按时写出实验报告。
7. 实验完毕后，实验结果经老师确认后方可拆线，并将实验箱及接线按原样整理好。
8. 实验过程中如发生事故，应立即关断电源，保持现场，报告指导老师。若不按要求操作，损坏了实验设备，根据损坏情况赔偿。
9. 实验中注意安全，保持环境整洁、安静。

八、实验报告要求

实验报告中应包括以下内容：

1. 实验目的
2. 实验原理
3. 实验内容与实验过程描述
4. 实验结果分析与实验结论
5. 收获、体会及建议
6. 预习思考题及实验验证
7. 实验后思考题
8. 实验源程序

实验报告模板可参照附件 1。

九、课后思考题

1. 改变 DAC0832 片选端 CS 的连线方式，要如何修改程序？

2. 若要是 DAC0832 工作在直通和双缓冲方式下，应该如何连线？
3. 如果 Ua 端输出 -3V 到 0V 之间的三角波，Ub 端输出波形如何？
4. 如果通过查询 ADC0809 的 EOC 引脚状态来判断转换是否结束，则硬件应当如何连接，相应的程序如何编写？

十、参考资料

1. 微型计算机原理与接口技术，周荷琴，吴秀清，中国科技大学出版社。
2. 微机原理及接口技术实验指导书，《微机原理及接口技术》课程教学团队，北京航空航天大学。

注意：

选择实验 4 “题目七 加密/解密算法实现”的同学，应当提前 4 周开始，并在完成实验 3 的同时向实验指导老师汇报加解密算法的原理和编程方案。

实验四 综合设计实验（自主设计型实验）

一、实验目的

1. 在完成前置实验，掌握微机系统的基本原理、EDA 软件使用方法、以及实验室硬件资源的基础上，完成本次设计型实验。

3. 提高课程知识的综合运用能力；

2. 培养分析工程问题、提出设计方案、以及设计微机应用系统的能力。

二、实验内容

下面的题目中任选其中一个，分析题目需求，提出设计方案。在实验箱或 Proteus 软件中搭建电路、编写程序、运行（或仿真），对设计方案进行验证。

（1）题目一 监测报警系统。使用 ADC 采集数据，数码管或屏幕显示电压，高于 4.0v 或低于 1.0v 发出告警声。

（2）题目二 电子秒表。用定时器、数码管实现实现电子秒表，具有启动、暂停、继续、结束等功能；

（3）题目三 火箭发射倒计时系统。从键盘或实验箱按钮启动倒计时（10 秒），每秒计数变化一次，并发出一次警报声。计时结束后发出连续滴滴声

（4）题目四 密码锁。小键盘（大键盘）输入 6 个数，同时数码管显示，并与预置的密码对比。密码输入完成后，正确则响一声滴，错则连续滴滴滴三声。连续输错三次则锁死，并反复报警。

（5）题目五 交通灯控制系统。在 Proteus 中模拟十字路口（共 4 个方向）交通灯，每个方向包括一组红黄绿交通灯，编写程序控制交通灯的亮、灭。

扩展一：每组交通灯配一个数字时钟，用于显示当前亮灯的剩余时间；

扩展二：加入手动控制系统，以便对每个方向的交通进行人工控制。

（6）题目六 出租车计价器。用开关 K1 闭合/断开表示计价器开始/停止工作，ADC 转换器对可调电压进行转换，代表当前出租车时速。根据累计里程计算并显示费用，需要考虑出租车等待红灯、堵车时额外增加的耗时费用。

（7）题目七 加密/解密算法实现（需提前 4 周开始）。查找资料，研究 RC4 加密算法的原理，采用汇编编程实现 RC4 加密/解密算法，实现以下功能：从键盘输入明文，

经过加密后在屏幕输出密文（16 进制）；从键盘输入密文（16 进制），经过解密后在屏幕上显示明文。

（8）题目八 电机控制系统。对电位计信号进行数据采集，根据采集的数字量控制电机的转速。可设置一开关量，根据开关的状态决定电机的转向。电机速度和转向可用数码管显示。

（9）题目九 裁判计分器。使用数字键盘输入 5 个裁判的打分并显示，去掉最高分、最低分后，取平均值显示。分数范围 0~99，原始分和平均分都要求显示在数码管上。

（10）题目十 猜数字小游戏。由程序随机产生一个数字（0~9 之间）并发出声音，提醒用户从键盘输入一个数字。如果两个数字相同则播放一段音乐，不同则播放报警声，同时在数码管上显示正确的数字和用户输入的数字。等待用户输入数字的过程中显示流水灯，等待超时则使用灯光或者声音进行提示，并重新产生数字。

（11）题目十一 电话拨号模拟。用 12 个按键模拟电话机的 0~9、*和#按键，没按下一个按键就显示在数码管或 LCD 屏幕上，同时发出一个特定的声音。按下*键可以清除最后输入的数字，按下#键则清除所有输入的数字。当输入 8 数字后发出连续的“嘀-嘀-”声。按键和使用 PC 机键盘、实验箱上的按键、或者 EDA 软件自行搭建按键键盘。

（12）题目十二 自学实验指导书提供的 ARM 系统设计资料，从实验 8、实验 9 中选择一项完成。

*注 1：如果使用 Proteus 完成上述题目，报警声可以使用闪光灯或者其它方式警告代替。

*注 2：可以自行增加题目所实现的内容和功能。

三、实验设备

1. 计算机（Windows 7 32 位操作系统）
2. TPC-ZK-II 集成开发环境
3. TPC-ZK-II 实验箱
4. EDA 软件

四、预习要求

1. 了解实验箱硬件资源；

2. 理解微机原理基础知识；
3. 掌握 EDA 使用方法，能够使用 EDA 软件设计电路原理图。

五、注意事项

1. 实验要求的所有连线连接好并让老师检查确认后，方可用 USB 线连接实验箱和计算机并打开实验箱电源开关。

2. 实验箱的电源开关有两个，分别位于右侧和上方；

3. 计算机必须连接实验箱后，才能够在 TPC-ZK-II 集成开发环境中完成编译连接运行整个过程。没有连接实验箱的计算机 TPC-ZK-II 集成开发环境中只能对程序进行编译连接，运行时会出现报错。需要按照下面方法运行程序：

计算机任务栏左侧开始-运行-输入 cmd-打开 command 窗口，指明之前保存源程序文件的文件夹路径，输入可执行文件名，回车即可运行可执行文件查看实验结果是否正确。

4. 因为公共实验室的计算机都装有防病毒卡，所以请将程序保存在非系统盘内，否则计算机意外关机或重启后所作的工作都将丢失。

5. 爱护实验设备，在接有实验箱的实验过程中，一定按实验要求操作，严禁带电接线或拆线。

6. 上实验课前,要求准备上机实验程序，结合课堂讲授内容阅读实验指导；实验中要求记录实验过程和结果，按时写出实验报告。

7. 实验完毕后 ,实验结果经老师确认后方可拆线，并将实验箱及接线按原样整理好。

8. 实验过程中如发生事故，应立即关断电源，保持现场，报告指导老师。若不按要求操作，损坏了实验设备，根据损坏情况赔偿。

9. 实验中注意安全，保持环境整洁、安静。

六、实验报告要求

实验报告中应包括以下内容：

- 1.系统功能描述
- 2.系统设计方案

- 3 编程思路（或程序框图）
- 4 系统功能展示（照片及文字说明）
- 5.结果分析与结论
- 6.收获、体会及建议
- 7.电路原理图、实验源程序

实验报告模板可参附件 1。

七、参考资料

1. 微机原理及接口技术实验指导书，《微机原理及接口技术》课程教学团队，北京航空航天大学；
2. 微型计算机原理与接口技术，周荷琴，吴秀清，中国科技大学出版社；

第三部分 设计型实验（虚拟仿真实验）

实验五 最小系统设计与仿真实验（虚拟仿真实验）

一、实验目的

1. 学习使用 EDA 软件搭建微型计算机系统电路原理图的方法。
2. 学习使用 EDA 软件进行系统仿真的方法
3. 学习和理解 8086 最小系统的电路组成、总线生成、系统运行过程。

二、实验内容

1. 在 Proteus 软件中搭建电路原理图，完成其中的第 1、2、3 部分。
2. 在电路原理图中添加虚拟示波器或者逻辑分析仪，观察和记录指令执行时的地址总线、数据总线、ALE、BHE、RD、WR、M/IO、DEN、DT/R 等信号的波形
3. 选做内容：搭建电路第 4、5、6 部分
4. 将提供的汇编程序复制到 Proteus 项目中的源程序中（默认是 main.asm），编译、仿真运行。（如果完成了电路第 4、5、6 部分，可以试着鼠标双击电路图中的开关，观察对 LED 灯的影响。）

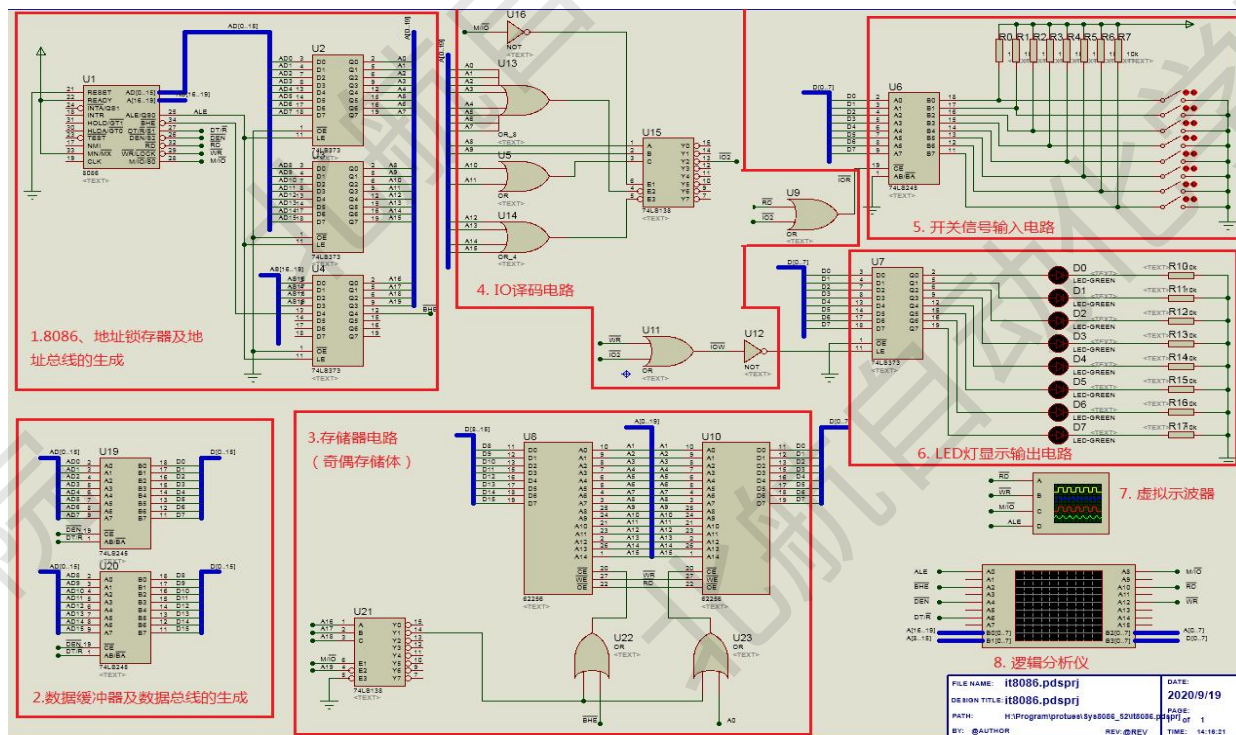


图 3.5.1 8086 最小系统电路原理图

5. 元件列表

元件名称	所属类	所属子类	功能说明
8086	Microprocessor ICs	i86 Family	微处理器
74LS373	TTL 74 series	Flip-Flops & Latches	三态输出的8D透明锁存器
74LS138	TTL 74LS series	Decoders	3线—8线译码器
74LS245	TTL 74LS series	Transceivers	8路同相三态双向总线收发器
62256	Memory ICs	Static RAM	存储器
LED-GREEN	Optoelectrics	LEDs	绿色发光二极管
NOT	Simulator Primitives	Gates	非门
OR	Simulator Primitives	Gates	2输入或门
OR4	Modelling Primitives	Digital(Buffers&Gates)	4输入或门
OR8	Modelling Primitives	Digital(Buffers&Gates)	8输入或门
RES	Resistors		电阻
SWITCH	Switchs & Relays	Switchs	开关

6. 源程序

1) 复制下面的汇编程序并替换 Proteus 项目 main.asm 中原有的代码（用于实现了电路原理图 1、2、3 部分的系统仿真）

CODE SEGMENT

ASSUME CS:CODE

START: MOV AX,1000H

MOV DS,AX

MOV SI, 0

SIM: MOV BYTE PTR [SI], 01H

MOV BYTE PTR [SI+1], 02H

MOV BYTE PTR [SI+2], 03H

MOV BYTE PTR [SI+3], 04H

JMP SIM

CODE ENDS

END START

2) 复制下面的汇编程序并替换 Proteus 项目 main.asm 中原有的代码（针对电路原理图 4、5、6 部分的系统仿真）

```
CODE SEGMENT
ASSUME  CS:CODE
START:  MOV  DX, 200H
        IN   AL, DX
        NOT  AL
        OUT  DX, AL
        JMP  START
CODE ENDS
        END START
```

3) 或者综合前面两种代码，替换 Proteus 项目 main.asm 中原有的代码（用于实现电路原理图 1~6 部分的系统仿真）

```
CODE SEGMENT
ASSUME  CS:CODE
START:  MOV AX, 1000H
        MOV DS, AX
        MOV SI, 0
        ;存储器操作（向内存中存入数据）
SIM:    MOV BYTE PTR [SI], 01H
        MOV BYTE PTR [SI+1], 02H
        MOV BYTE PTR [SI+2], 03H
        MOV BYTE PTR [SI+3], 04H
        ;输入输出操作（检测开关状态，LED 灯显示）
        MOV  DX, 200H
        IN   AL, DX
        NOT  AL
        OUT  DX, AL
```

;跳转到 SIM 处，重复执行前面的操作

JMP SIM

CODE ENDS

END START

三、实验设备

1. 计算机（Windows7/10 32/64 位操作系统）
2. 电子设计自动化（EDA）软件 Proteus

四、预习要求

1. 预习 8086CPU 工作原理，存储器组织结构，最小系统。
2. 安装 Proteus 软件。
3. 预习思考题
 - 1) 为什么 8086 最小工作模式系统需要地址锁存和数据缓冲器
 - 2) 什么是总线

五、实验原理

1. 8086CPU 工作原理及最小系统构成，请阅读参考书中相关部分。
2. Proteus 软件的部分使用事项：主要是 Proteus 软件中示波器、逻辑分析仪的使用，以及数据查看功能（图 9-2~图 9-4）。
 - 1) 图 9-2 说明了示波器和逻辑分析仪如何查找。
 - 2) 图 9-3 是逻辑分析仪使用的方法。注意事项：
 - 每次仿真运行，都要点击图 9-3 标号 7 的 Cpature 按钮，此时按钮变成粉红。然后要等到变绿、再变灰之后才会出现波形。
 - 图 9-3 标号 6 的地方，如果捕捉分辨率过低（即捕捉时间间隔太长），可能导致波形不能显示。
 - 图 9-3 标号 4、5 的地方可以不用管，使用默认配置。

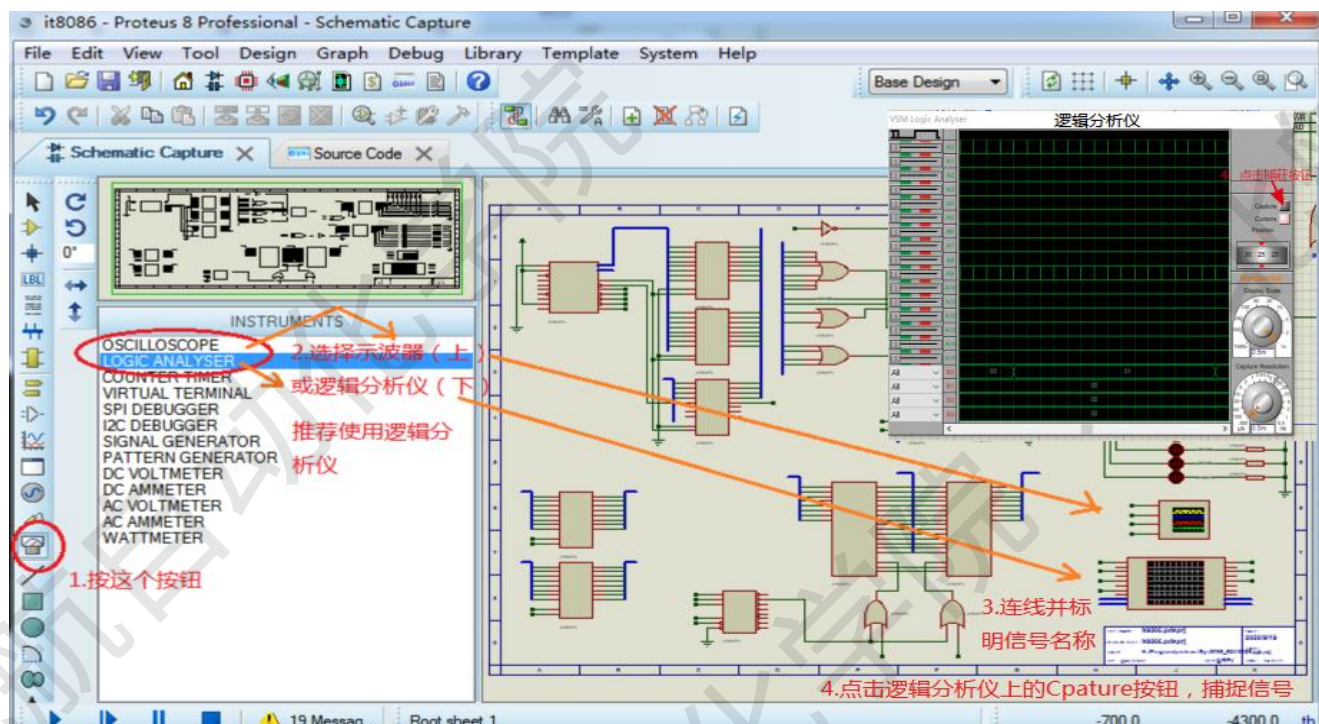


图 3.5.2 Proteus 软件中示波器和逻辑分析仪的使用

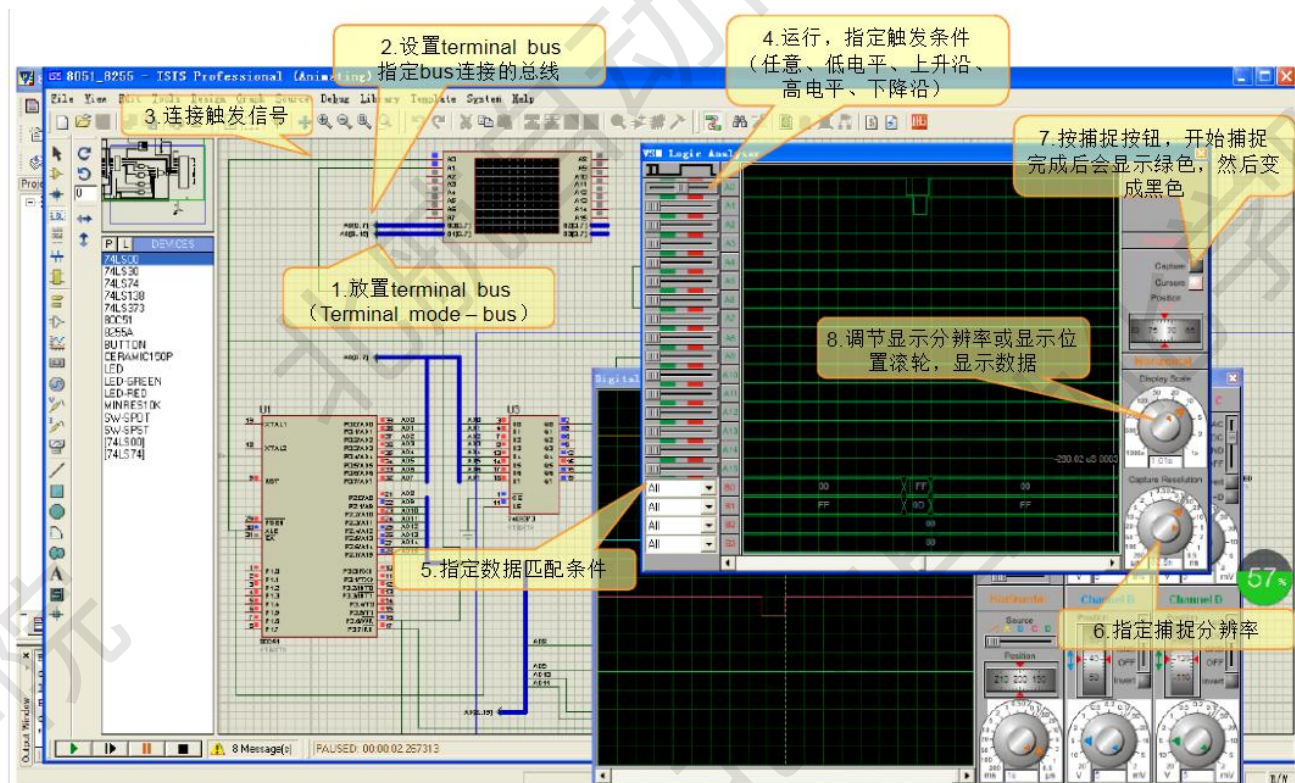


图 3.5.3 逻辑分析仪使用方法

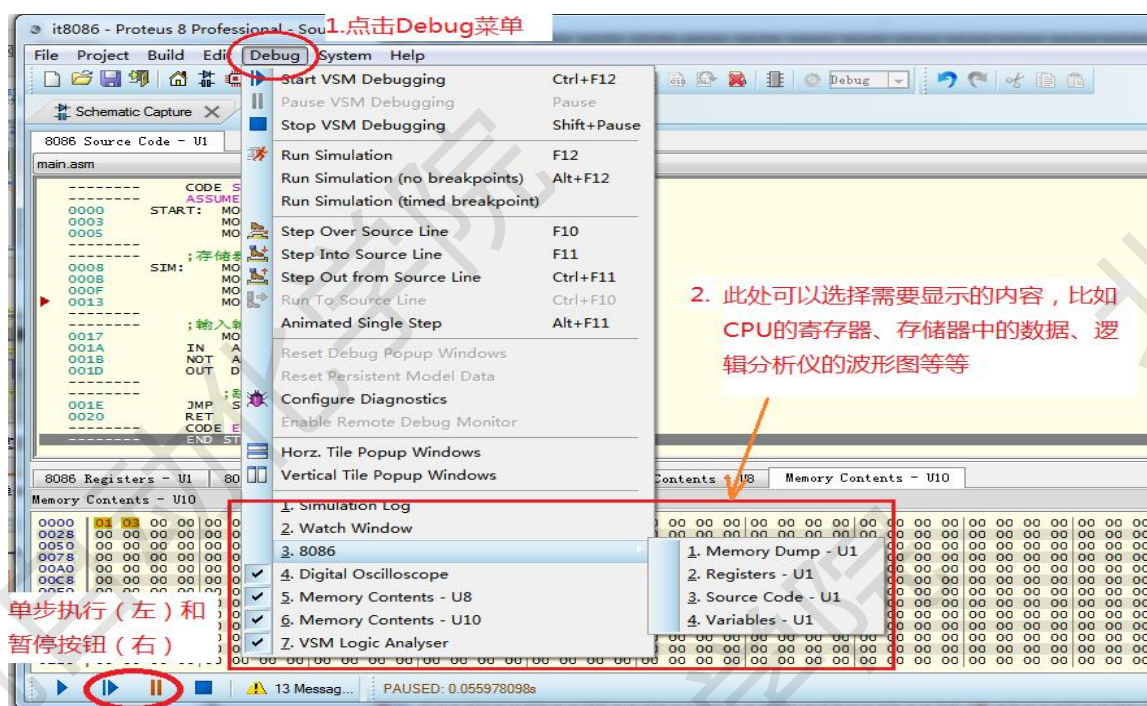


图 3.5.4 数据查看方法

3) 图 9-4 为仿真运行过程中数据查看的方法。

六、实验步骤

1. 打开 Proteus 软件，按照需求新建项目，选择 8086 微处理器和 MASM32 编译器；
2. 查找元件列表中的元件，添加到自己的元件库；
3. 根据图 9-1 搭建 8086 最小系统电路原理图。
4. 复制源程序到 main.asm（或者自己指定的源文件）中。
5. 仿真运行，观察实验现象，查看实验数据。

七、注意事项

1. 逻辑分析仪的总线输入信号最多只有 8 位，外接信号时需要注意引入信号的宽度。

八、实验报告要求

实验报告中应包括以下内容：

- 1.实验目的
- 2.实验原理
- 3.实验内容与实验过程描述
- 4.实验结果分析与实验结论
- 5.收获、体会及建议
- 6.预习思考题及实验验证
- 7.实验后思考题
- 8.实验源程序

实验报告模板可参照附件 1。

九、课后思考题

1. 地址锁存器如何锁存地址？
2. 数据缓冲器的控制信号有哪些，作用是什么？

十、参考资料

1. 微机原理及接口技术实验指导书，《微机原理及接口技术》课程教学团队，北京航空航天大学；
2. 微型计算机原理与接口技术，周荷琴，吴秀清，中国科技大学出版社；
3. 8086 芯片使用手册



实验六 并行接口设计实验（虚拟仿真实验）

一、实验目的

1. 通过设计地址译码电路，理解地址译码的工作原理和功能。
2. 学习并行接口芯片 8255A 的工作原理、编程方法

三、实验内容

在 Proteus 软件设计并行接口电路，使用 8255A 控制七段码数码管显示数字。可以在实验九“最小系统设计与仿真实验”在基础上进行修改，通过删减和添加，实现所要求的，其中并行接口电路原理图如图 10-1 所示。

1. 设计其中的地址译码电路，使得 8255A 的 4 个端口地址分别为 288H、28AH、28CH、28EH；
2. 编程在两个数码管上交替显示两个数字（固定两个数字即可，不需要从键盘输入）
3. 除了 8255A 之外；在 Proteus 中查找和添加的数码管为 7SEG-MPX2-CC，其字形码与实验内容 4 相同。

三、实验设备

1. 计算机（Windows7 32 位操作系统）
2. TPC-ZK-II 集成开发环境
3. TPC-ZK-II 实验箱

四、预习要求

1. 预习定时器 8255A 芯片工作原理、工作模式特点和编程设置。
2. 实验课之前自主准备和完成实验所用的汇编程序，以便实验课上调试和运行。
3. 预习思考题
 - 1) 8255A 的 PC 端口引脚的输出如何进行控制？
 - 2) 共阴极 7 段码数码管中发光二极管是如何排列的？试写出 0~9，A~F 的字形码。

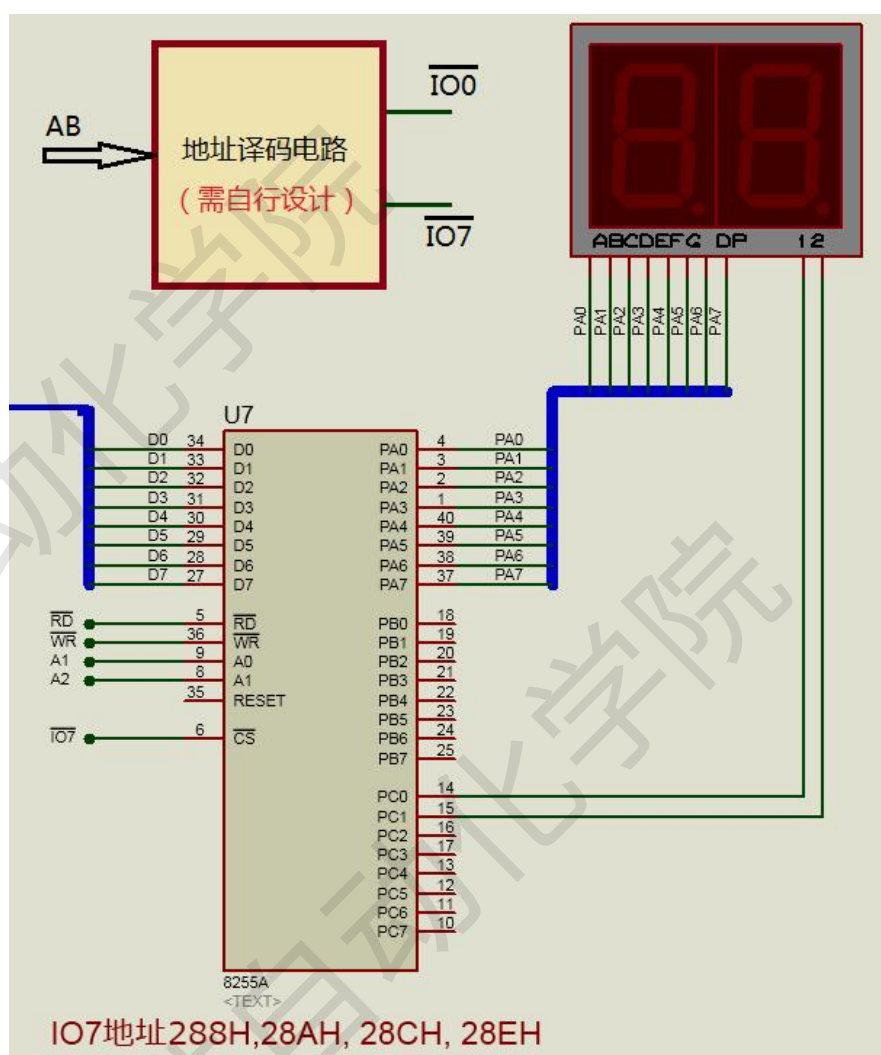


图 3.6.1 电路原理图

五、实验原理

参考实验四。

六、实验步骤

1. 打开 Proteus 软件，按照需求新建项目，选择 8086 微处理器和 MASM32 编译器；
2. 查找元件列表中的元件，添加到自己的元件库；
3. 参考图 3.5.1 和图 3.10.1 搭建系统电路原理图。
4. 编写源程序。
5. 仿真运行，观察实验现象。

七、注意事项

1. 逻辑分析仪的总线输入信号最多只有 8 位，外接信号时需要注意引入信号的宽度。

八、实验报告要求

实验报告中应包括以下内容：

1. 实验目的
2. 实验原理
3. 实验内容与实验过程描述
4. 实验结果分析与实验结论
5. 收获、体会及建议
6. 预习思考题及实验验证
7. 实验后思考题
8. 实验源程序

实验报告模板可参照附件 1。

九、课后思考题

1. 地址译码器只使用 A9~A0 是否可以，会导致什么现象？

十、参考资料

1. 微机原理及接口技术实验指导书，《微机原理及接口技术》课程教学团队，北京航空航天大学；
2. 微型计算机原理与接口技术，周荷琴，吴秀清，中国科技大学出版社；
3. 8255A 芯片使用手册

实验七 定时器与数字音频设计实验（虚拟仿真实验）

一、实验目的

1. 深入理解定时器工作原理和工作模式，掌握定时器编程技术。
2. 学习定时器电路设计。

二、实验内容

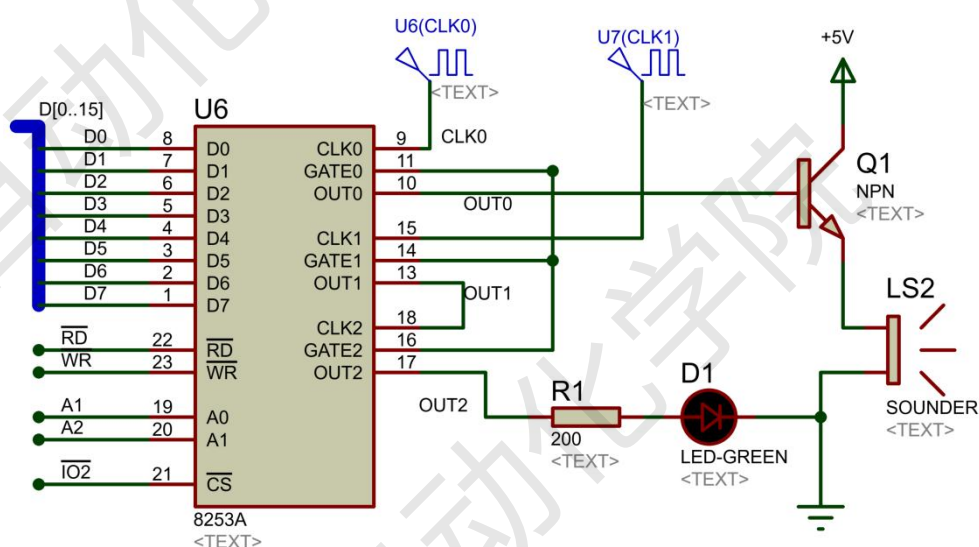


图 3.7.1 电路原理图

1. 硬件连接

图中 8253A 引脚 CLK0 接 5kHz 的时钟信号，引脚 OUT0 接 NPN 三极管的基极；引脚 CLK1 接 1kHz 时钟信号，引脚 OUT2 接电阻和发光二极管，电阻值为 $200\ \Omega$ ；引脚 $\overline{CS}$ 接地地址译码器输出的片选信号 $\overline{IO2}$ 。地址译码器和 $\overline{IO2}$ 的端口地址可自行设计。

2. 软件编程

编写程序，对计数器 0~2 分别设置工作方式，并完成以下任务：

通过程序控制 OUT0 输出不同音阶的频率信号，驱动发声器 SOUNDER 播放一段音乐；

CLK1 输入的 1kHz 时钟，经过分频后 OUT1 输出 10Hz 方波；计数器 2 设置为方式 2，计数初值设为 N ($N \leq 0FH$)，修改 N 可以改变发光二极管闪烁的频率。



*注：Proteus 仿真速度较慢，无法实时运行，CLK 引脚输入频率过高则反应时间较长。

3. （选做）添加数字键盘，通过并行接口芯片 8255A 采集数字按键状态。当按下不同按键时发出不同音阶的声音，同时在数码管上显示音阶值。。

三、实验设备

1. 计算机 （Windows7 32 位操作系统）
2. EDA 软件

四、预习要求

1. 预习定时器 8253 芯片工作原理、工作模式特点和编程设置。
2. 实验课之前自主准备和完成实验所用的汇编程序，以便实验课上调试和运行。
3. 预习思考题
 - 1) 本次实验用到哪些工作方式，简述其特点。
 - 2) 定时器各个通道的 GATE 引脚的功能是什么？

五、实验原理

Intel8253 是一种可编程计数器/定时器芯片（可编程间隔定时器，PIT：Programmable Interval Timer）。Intel8253 内部有 3 个独立的 16 位计数器通道，通过对它进行编程，每个计数器通道都可以按 6 种不同的方式工作，并且可以按 2 进制或 10 进制格式进行计数，最高计数频率 2MHZ。Intel8253 还可以作为方波发生器、分频器、程控单脉冲发生器等等。

1. 工作方式设置/控制字

在 Intel8253 工作之前，必须对它进行初始化编程，也就是向 Intel8253 的控制字寄存器写入一个控制字和向计数器赋计数初值。控制字的功能是：选择计数器，确定对计数器的读/写格式，选择计数器的工作方式以及确定计数的数制。Intel8253 控制字的格式如图所示。

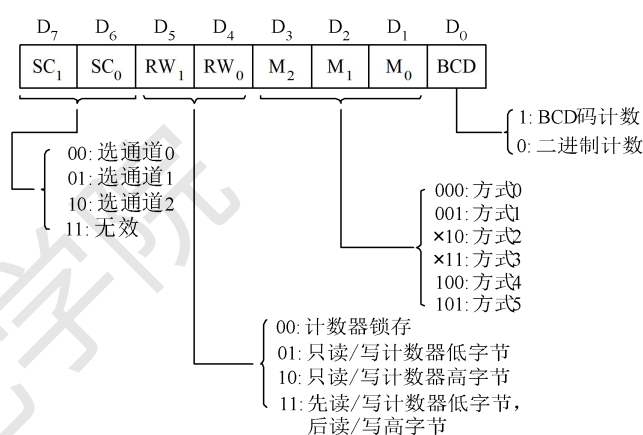


图 3.11.2 8253/ 8254 控制字

2. 读写操作

1) Intel8253 的写操作包括写控制字和写计数初值两项内容。具体要求：

- 各计数器的控制字都写到同一地址单元，而各计数初值写到各自的地址单元中。
- 对于每个计数器，必须先写控制字，后写计数初值。因为后者的格式是由前者决定的。
- 写入的计数初值必须符合控制字决定的格式。16 位数据应先写低 8 位，再写高 8 位。
- 当给多于一个的计数器写入控制字和计数初值时，其顺序有一定的灵活性，只要遵循上述要求即可。

2) Intel8253 的读操作所得到的的是当前计数值，通常用于实时检测、实时显示和数据处理。在进行读操作时需要注意以下几点：

- 读操作是通过访问对应于各计数器的地址单元来实现。
- 每个计数器的读操作必须按照控制字确定的格式。如果是 16 位计数，读操作要进行两次，先读低 8 位，后读高 8 位。

当计数器为 16 位时，为了避免在两次读出过程中计数值的变化，要求先将计数值锁存。锁存计数值的常用方法是使用计数器锁存命令：控制字的 D7D6 两位为所要锁存的计数值，D5D4 两位置为 00。Intel8253 的每个计数器都有一个输出锁存器（16 位），平时它的值跟随计数值而变化。当向计数器写入锁存命令后，现行计数值被锁存（计数器仍能继续计数）。这样 CPU 读取的就是锁存器中的值。当 CPU

读取了计数值或对计数器重新编程以后，锁存状态被解除，输出锁存器的值又随计数值变化。

3. 编程提示

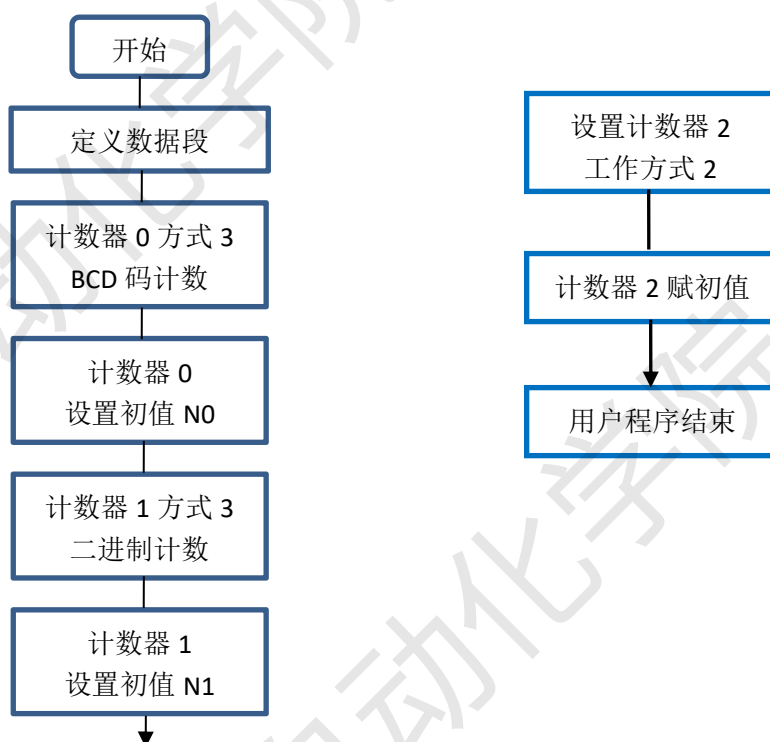


图 3.11.3 程序流程图

六、实验步骤

1. 打开 Proteus 软件，按照需求新建项目，选择 8086 微处理器和 MASM32 编译器；
2. 查找元件列表中的元件，添加到自己的元件库；
3. 参考图 3.11.1 搭建系统电路原理图。
4. 编写源程序。
5. 仿真运行，观察实验现象。

七、注意事项

1. 逻辑分析仪的总线输入信号最多只有 8 位，外接信号时需要注意引入信号的宽度。

八、实验报告要求

实验报告中应包括以下内容：

- 1.实验目的
- 2.实验原理
- 3.实验内容与实验过程描述
- 4.实验结果分析与实验结论
- 5.收获、体会及建议
- 6.预习思考题及实验验证
- 7.实验后思考题
- 8.电路原理图、实验源程序

实验报告模板可参照附件 1。

九、课后思考题

1. Intel8253 的初始化编程分几步进行？
2. Intel8253 各计数通道的 Gate 引脚作用是什么？
3. 是否可以利用 Intel8253 定时器控制 LED 闪烁来发送摩斯码？使简述编程思路。

十、参考资料

1. 微型计算机原理与接口技术，周荷琴，吴秀清，中国科技大学出版社。
2. 微机原理及接口技术实验指导书，《微机原理及接口技术》课程教学团队，北京航空航天大学。

第四部分 ARM（STM32F103R6）实验

一、 ARM（STM32F103R6）基本概念及编程简介

1. STM32 芯片介绍

STM32 就是指 ST 公司开发的 32 位 ARM 微控制器，自带了各种常用通信接口，比如 USART、I2C、SPI 等，可接非常多的传感器，可以控制很多的设备。STM32 有很多系列，从内核上分有 Cortex-M0、M3、M4 和 M7 等多种，每种内核又大概分为主流、高性能和低功耗系列。从学习的角度出发，可以选择 F1 和 F4 系列。F1 是基础型，基于 Cortex-M3 内核，主频为 72MHz；F4 是高性能系列，基于 Cortex-M4 内核，主频 180MHz。F4（429 系列以上）与 F1 系列相比，除了内核不同和主频的提升外，升级的明显特色就是带了 LCD 控制器和摄像头接口，支持 SDRAM。目前市场上资料最多，产品占有量最多的是 F1 系列的 STM32 单片机。我们这门课选择 F1 系列的 STM32F103R6 作为授课的芯片。

CPU 位数	内核	系列	描述
32	Cortex-M0	STM32-F0	入门级
		STM32-L0	低功耗
		STM32-F1	基础型，主频 72M
	Cortex-M3	STM32-F2	高性能
		STM32-L1	低功耗
		STM32-F3	混和信号
	Cortex-M4	STM32-F4	高性能，主频 180M
		STM32-L4	低功耗
8	Cortex-M7	STM32-F7	高性能
		STM8S	标准系列
	超级版 6502	STM8AF	标准系列的汽车应用
		STM8AL	低功耗的汽车应用
		STM8L	低功耗

图 4.1 STM 系列芯片

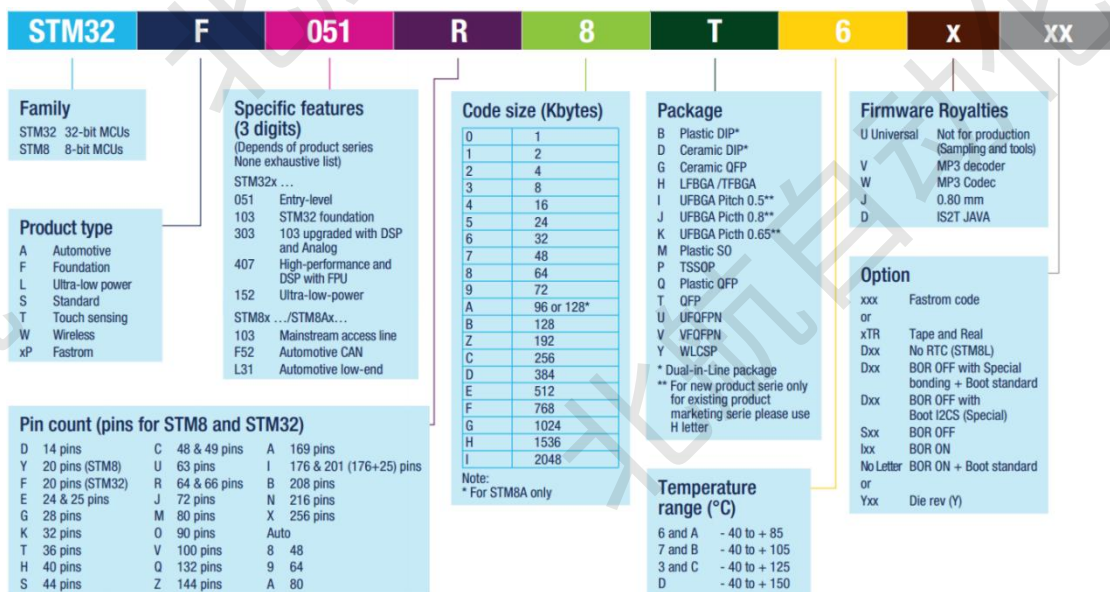


图 4.2 STM8 和 STM32 命名方法，摘自《STM8 和 STM32 选型手册》

STM32F103 采用 Cortex-M3 内核。内核由 ARM 公司设计，授权给芯片生产厂商意法半导体（STMicroelectronics）公司。ST 公司在内核之外设计部件并生产整个芯片，这些内核之外的部件被称为核外外设或片上外设,如 GPIO、USART（串口）、I2C、SPI 等等。

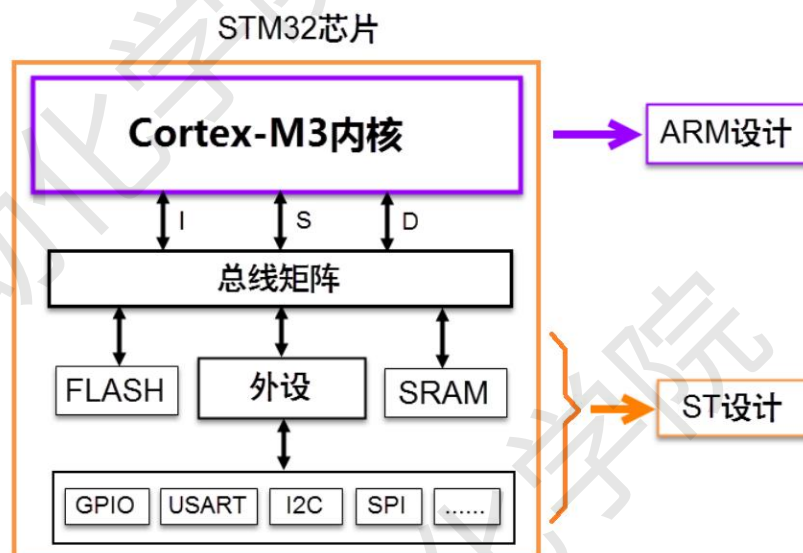


图 4.3 STM32 芯片架构简图

M3 内核和外设之间通过各种总线连接，其中驱动单元有 4 个，被动单元也有 4 个。

（1）ICode 总线

用于取指，将 Flash 中的指令传送到内核。

（2）驱动单元

包括 DCode 总线、System 总线、DMA 总线。

DCode 总线

用于存取 SRAM 中的数据。因为数据可以被 Dcode 总线和 DMA 总线访问，所以为了避免访问冲突，在取数的时候需要经过一个总线矩阵来仲裁，决定哪个总线在取数。

System 总线

系统总线主要是访问外设的寄存器。

DMA 总线

DMA 总线也主要是用来传输数据，这个数据可以是在某个外设的数据寄存器，可以在 SRAM，可以在内部的 FLASH。

（3）被动单元

包括内部 FLASH、内部 SRAM、FSMC（Flexible static memory controller，可变静态存储控制器）、AHB/APB 总线转换桥。

内部 FLASH

用于存放程序。其中的指令通过 ICode 送给 CPU。

内部 SRAM

用于存放运行过程中的数据，包括变量、堆栈等等。CPU 通过 DCode 访问内部 SRAM。

FSMC (Flexible static memory controller, 可变静态存储控制器)

用于外部存储器扩展，支持多种类型的存储器。通过对特殊功能寄存器的设置，FSMC 能够根据不同的外部存储器类型，发出相应的数据/地址/控制信号类型，从而使得 STM32 系列微控制器能够在不增加外部器件的情况下同时扩展多种不同类型的静态存储器，满足系统设计对存储容量、产品体积以及成本的综合要求。

AHB /APB 总线转换桥

AHB 总线延伸出来的两条 APB2 和 APB1 总线，上面挂载着 STM32 各种各样的外设，比如 GPIO、串口、I2C、SPI 等等，这是学习 STM32 的重点，通过对外设编程去驱动外部的各种设备。

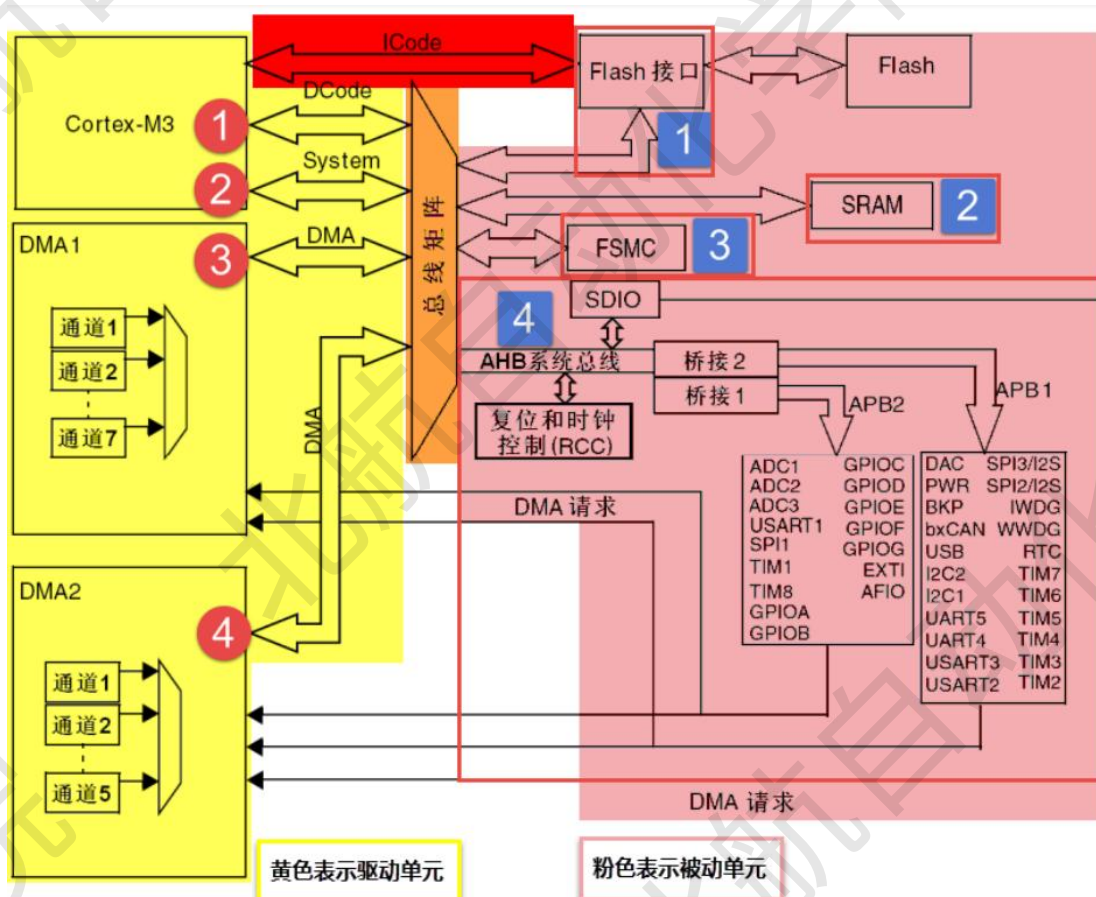


图 4.4 STM32F10xx 系统框图

2. 时钟树

STM32 为了实现低功耗，设计了复杂的时钟系统，称之为时钟树，从而使得外设的时钟可自配置。**所有的外设在使用之前都必须设置时钟信号，才可以正常工作。**因为 STM32 外设众多，而不同的项目用到的外设参差不齐，所以可控的时钟可以实现降低产品功耗。

STM32 系列芯片可以使用外部时钟源，也可以使用内部时钟源。内部 8MHz 的 RC 振荡器的误差在 1% 左右，精度通常比用 HSE（外部晶振）要低十倍以上。具体方法和电路可以查阅相关资料。

总体来说，STM32 有 5 个时钟源，分别为 HSI, HSE, LSI, LSE, PLL。

- HSI 是高速内部时钟，RC 振荡器，频率为 8MHz。
- HSE 是高速外部时钟，可接石英谐振器、陶瓷谐振器，或者直接外部时钟源，它的频率范围为 4MHz~16MHz。
- LSI 是低速内部时钟，RC 振荡器，频率为 40kHz。
- LSE 是低速外部时钟，接频率为 32.768kHz 的石英晶体。
- PLL 为锁相环倍频输出，其时钟输入源可选择为 HSI/2、HSE 或者 HSE/2。倍频可选择为 2~16 倍，但是其输出频率最大不得超过 72MHz。

STM32 芯片内部有 5 种时钟：

- SYSCLK：系统时钟，STM32 大部分器件的时钟来源。主要由 AHB 预分频器分配到各个部件。
- HCLK：由 AHB 预分频器直接输出得到，它是高速总线 AHB 的时钟信号，提供给存储器，DMA 及 Cortex 内核，是 Cortex 内核运行的时钟，CPU 主频就是这个信号，它的大小决定 STM32 运算速度、数据存取速度。
- FCLK：由 AHB 预分频器输出得到，是内核的“自由运行时钟”（free running clock）。“自由”表现在它不来自时钟 HCLK，因此在 HCLK 时钟停止时 FCLK 也继续运行。它保证在处理器休眠时，也能够采样到中断和跟踪休眠事件。它与 HCLK 互相同步。
- PCLK1：外设时钟，由 APB1 预分频器输出得到，最大频率为 36MHz，提供给挂载在 APB1 总线上的外设（低速外设）。
- PCLK2：外设时钟，由 APB2 预分频器输出得到，最大频率可为 72MHz，提供给挂载在 APB2 总线上的外设（高速外设）。
- 其它时钟：如 USB 时钟，Cortex 系统时钟，ADC 时钟，RTC 时钟、独立看门狗时钟、MCO 时钟输出等等。

STM32 库函数时钟系统时钟函数：SetSysClockTo72() 是默认的系统时钟设置函数。该函数的功能是利用 HSE 把时钟设置为：SYSCLK=HCLK=PCLK2=PLLCLK=72M，PCLK1=HCLK/2=36M，这是库函数的标准配置。

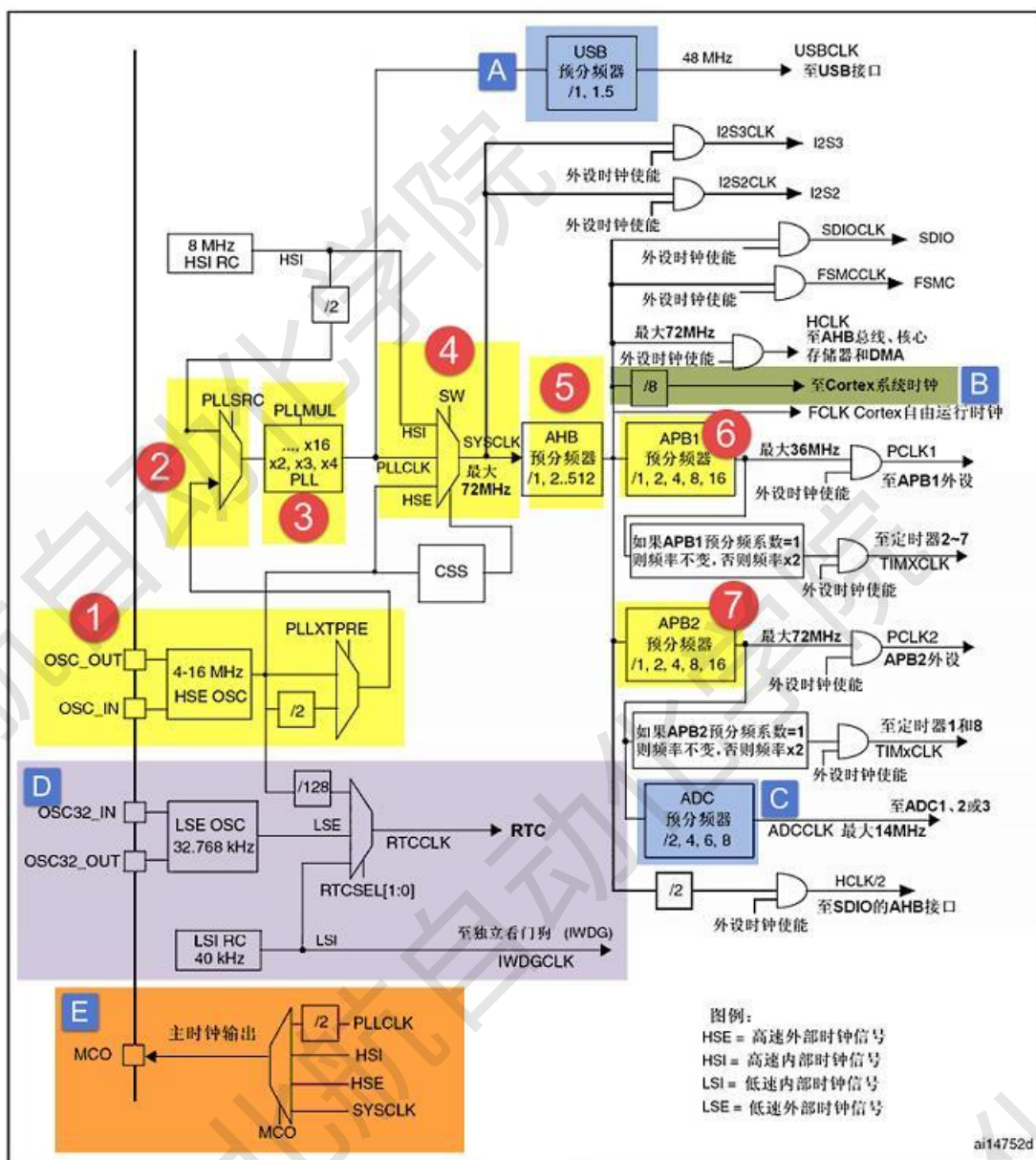


图 4.5 时钟树及时钟信号的形成

时钟输入与时钟输出之间的传播路径：①→②→③→④→⑤→⑥⑦，或者①→④→⑤→⑥⑦

在以上的时钟输出中有很多是带使能控制的，如 AHB 总线时钟、内核时钟、各种 APB1 外设、APB2 外设等。**当需要使用某模块时，必需先使能对应的时钟。**需要注意的是定时器的倍频器，当 APB 的分频为 1 时，它的倍频值为 1，否则它的倍频值就为 2。

- 连接在 APB1(低速外设)上的设备有：电源接口、备份接口、CAN、USB、I2C1、I2C2、UART2、UART3、SPI2、窗口看门狗、Timer2、Timer3、Timer4。注意 USB 模块虽然需要一个单独的 48MHz 时钟信号，但它应该不是供 USB 模块工作的时钟，而只是提供给串行接口引擎(SIE)使用的时钟。USB 模块工作的时钟应该是由 APB1 提供的。
- 连接在 APB2(高速外设)上的设备有：GPIO_A-E、USART1、ADC1、ADC2、ADC3、TIM1、TIM8、SPI1、AFIO

3. STM32 芯片编程：CMSIS 与 STM32 标准函数库

STM32 微处理的编程可以使用汇编语言或 C 语言，可以采用寄存器法或固件库法。寄存器法是使用汇编语言或 C 语言，直接对各种外设的控制寄存器进行设置和操作，从而完成指定的输入输出功能；固件库法是使用 C 语言，调用芯片厂商发布的、定义在固件库中的函数，完成指定的输入输出功能。

相对于汇编语言编程、寄存器法编程，使用 C 语言和固件库法编程，不必关注芯片硬件底层，开发效率较高。STM32 固件库是基于 CMSIS 标准定义的 C 语言函数库，也称为 STM32 标准函数库。

2.1 CMSIS 标准

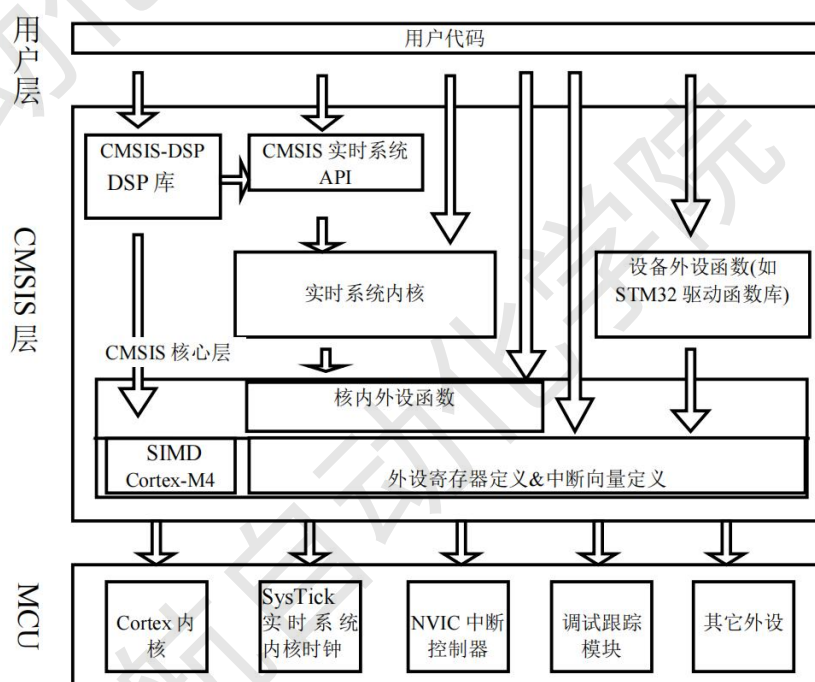


图 4.6 CMSIS 结构

不同的芯片厂商生产的 Cortex 微控制器虽然内核相同，但是外设不同。为了解决不同厂商、不同芯片开发软件的兼容性问题，ARM 与芯片厂商建立了 CMSIS 标准(Cortex MicroController Software Interface Standard)，为程序开发者提供了固件函数库和统一的编程接口（API）。用户可以无需深入掌握底层硬件细节就可以对外设进行控制。

ARM 提供独立于芯片的内核设备访问层、中间设备访问的通用方法、以及外设访问接口，芯片厂商在 CMSIS 基础上提供芯片外设接口库，如 GPIO、Timer、CAN、USART、ADC、DAC 等等，以便于二次开发，大大减少用户编程时间，降低开发成本，缩短不同芯片之间的已知时间。

CMSIS 核心层包括：

- 内核函数层：其中包含用于访问内核寄存器的名称、地址定义，主要由 ARM 公司提供；
- 设备外设访问层：提供了片上的核外外设的地址和中断定义，主要由芯片生产商提供。

CMSIS 层位于硬件层与操作系统或用户层之间，提供了与芯片生产商无关的硬件抽象层，可以为接口外设、实时操作系统提供简单的处理器软件接口，屏蔽了硬件差异，这对软件的移植是有极

大的好处的。STM32 的库，就是按照 CMSIS 标准建立的。

2.2 STM32 标准函数库

STM32 标准库可以从官网获得，本课程采用 v3.5.0 库文件。解压库文件可得到目录：“STM32F10x_StdPeriph_Lib_V3.5.0\”



图 4.7 STM32 标准函数库

固件库各文件夹的内容：

- **Libraries:** 文件夹下是驱动库的源代码及启动文件，我们使用的固件库就在这个文件夹里面。
- **Project :** 文件夹下是用驱动库写的例子和工程模板，包括为每个外设写好的例程，非常有用，在学习的时候就可以参考。
- **Utilities:** 包含了基于 ST 官方实验板的例程，不需要用到，可略过。
- **stm32f10x_stdperiph_lib_um.chm:** 库帮助文档，一个已经编译好的 HTML 文件，可以查询每个外设的及相关函数的说明，非常详细。

在使用库开发时，需要把 **libraries** 目录下的库函数文件添加到工程中，并查阅库帮助文档来了解 ST 提供的库函数。

2.3 函数库结构和文件分析

进入 **Libraries** 文件夹看到，关于内核与外设的库文件分别存放在 **CMSIS** 和 **STM32F10x_StdPeriph_Driver** 文件夹中。

2.3.1 函数库 CMSIS 目录

STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\CMSIS\文件夹展开内容见图，其中黄色部分的是我们需要用到的内容。

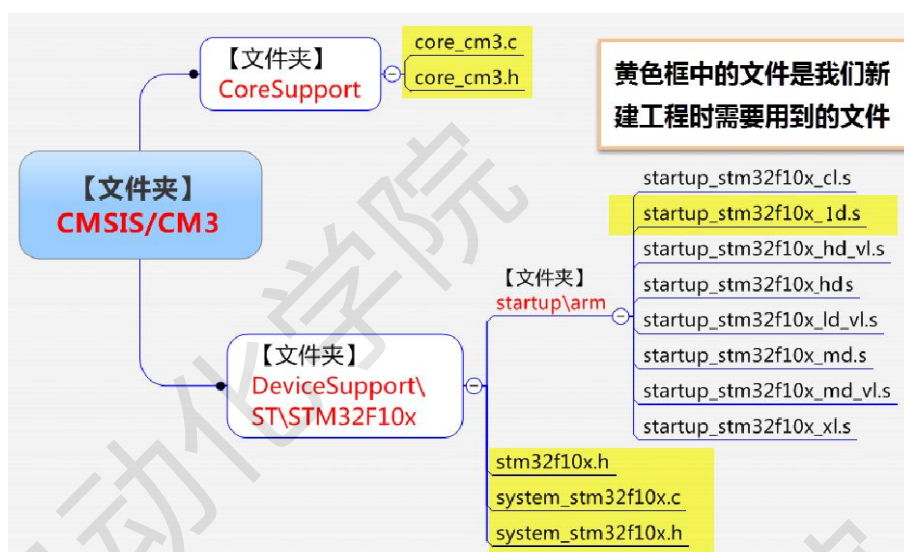


图 4.7 函数库 CMSIS 目录组成

(1) 内核相关文件：oreSupport 目录中的 core_cm3.c 和 core_cm3.h

Core_cm3.h 头文件里面实现了内核的寄存器映射，对应外设头文件 stm32f10x.h，区别就是一个针对内核的外设，一个针对片上（内核之外）的外设。core_cm3.c 文件实现了一下操作内核外设寄存器的函数，用的比较少。

(2) 启动文件：startup/arm 目录中的 startup_stm32f10x_xx.s

启动文件放在 startup/arm 这个文件夹下面。启动文件有很多个，不同型号的单片机用的启动文件不一样，有关每个启动文件的详细说明见表

启动文件	区别
startup_stm32f10x_ld.s	ld: low-density 小容量， FLASH 容量在 16-32K 之间
startup_stm32f10x_md.s	md: medium-density 中容量， FLASH 容量在 64-128K 之间
startup_stm32f10x_hd.s	hd: high-density 中容量， FLASH 容量在 256-512K 之间
startup_stm32f10x_xl.s	xl: 超大容量， FLASH 容量在 512-1024K 之间

以上四种都属于基本型，包括 STM32F101xx、STM32F102xx、STM32F103xx 系列。

(3) STM32 系统文件：stm32f10x.h、system_stm32f10x.c、system_stm32f10x.h

头文件 Stm32f10x.h 实现了片上外设的所以寄存器的映射，是一个非常重要的头文件，在内核中与之想对应的头文件是 core_cm3.h。

system_stm32f10x.c/.h 文件实现了 STM32 的时钟配置，操作的是片上的外设 RCC。系统在上电之后，首选会执行由汇编编写的启动文件，启动文件中的复位函数中调用的 SystemInit 函数就在这个文件里面定义。调用完之后，系统的时钟就被初始化成 72M。如果后面需要重新配置系统时钟，可以参考这个函数重写。为了维持库的完整性，我们不会直接在这个文件里面修改时钟配置函数。

2.3.2 函数库 STM32F10x_StdPeriph_Driver 目录

STM32F10x_StdPeriph_Driver 文件夹下有 inc (include 的缩写) 和 src (source 的简写) 两个子目录, 是 ST 标准库的主要内容。src 里面是每个 STM32 芯片外设的驱动源程序, inc 则是相对应的头文件。每个外设对应一个 .c 和 .h 后缀的文件。如针对模数转换(ADC)外设, 在 src 文件夹下有一个 stm32f10x_adc.c 源文件, 在 inc 文件夹下有一个 stm32f10x_adc.h 头文件。

这两个文件夹中, 还有一个很特别的 misc.c 文件, 这个文件提供了外设对内核中的 NVIC(中断向量控制器)的访问函数, 在配置中断时必须把这个文件添加到工程中。

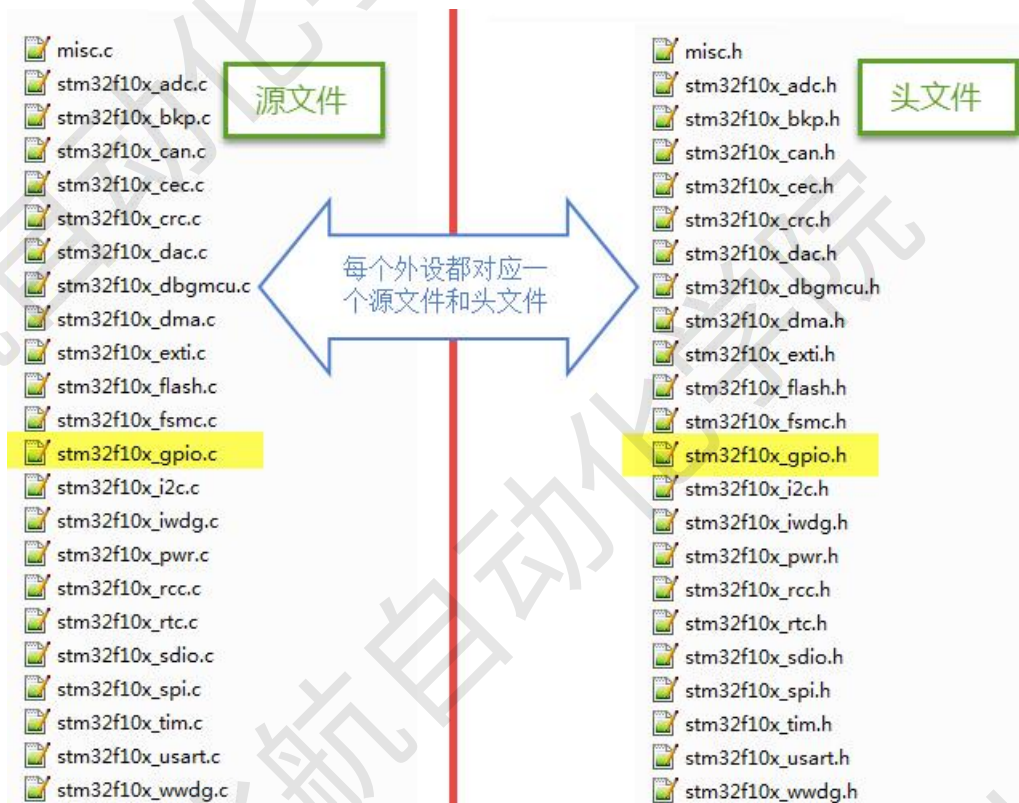


图 4.9 STM32 外设驱动源程序

2.3.3 函数库\Project\STM32F10x_StdPeriph_Template 目录

这个文件目录存放了官方的库工程模板。在用库建立一个完整的工程时, 还需要添加这个目录下的 stm32f10x_it.c、stm32f10x_it.h、stm32f10x_conf.h 和 system_stm32f10x.c 这四个文件。

stm32f10x_it.c: 用于编写中断服务函数。这个文件中已经预定义了一些系统异常(特殊中断)的接口函数, 但是中断处理的具体代码需要用户自行编写。另外还有一些其它普通中断服务函数需要自行添加。

system_stm32f10x.c: 包含了 STM32 芯片上电后初始化系统时钟、扩展外部存储器用的函数。例如, STM32F103 系列的芯片, 调用库中的 SystemInit 函数后, 系统时钟被初始化为 72MHz, 如有需要可以修改这个文件的内容, 设置成自己所需的时钟频率, 但鉴于保持库的完整性, 在做系统时钟配置的时候会另外重写时钟配置函数。

stm32f10x_conf.h: 包含了所有 STM32 外设定义的头文件, 而这个文件又被包含进 stm32f10x.h

文件。因此在应用程序中只包含 `stm32f10x.h` 即可。不必要的外设也可以在 `stm32f10x_conf.h` 中注释掉。

代码清单 `stm32f10x_conf.h` 文件配置软件库

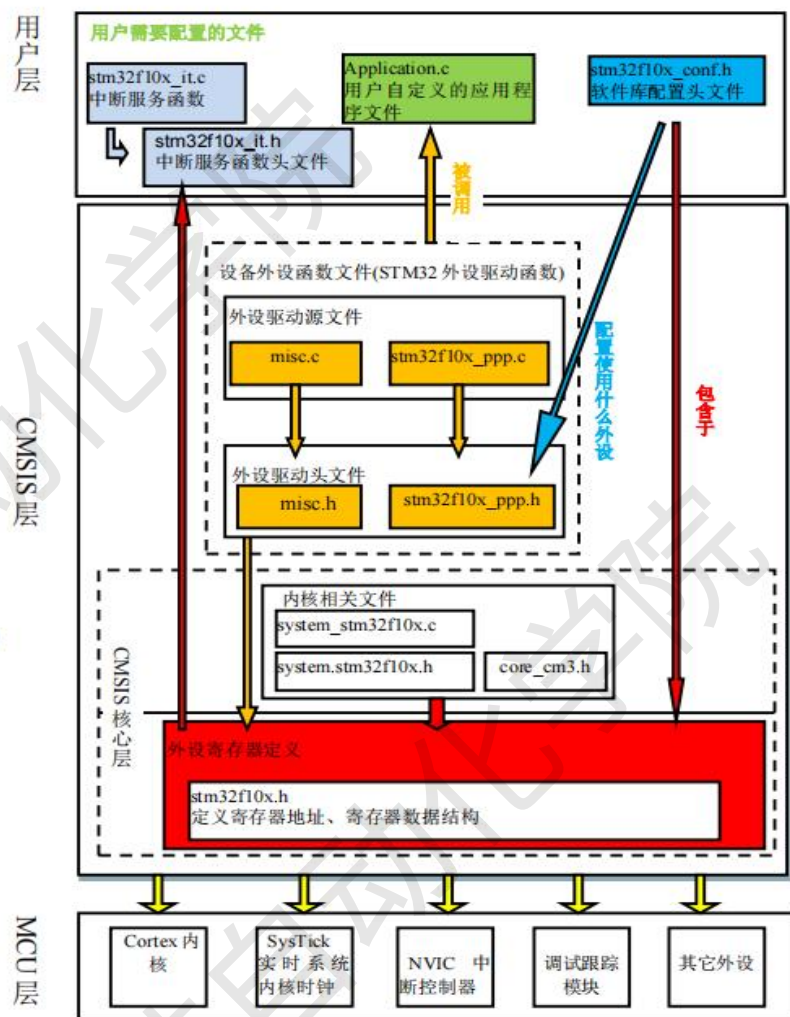
```
1 #include "stm32f10x_adc.h"
2 #include "stm32f10x_bkp.h"
3 #include "stm32f10x_can.h"
4 #include "stm32f10x_cec.h"
5 #include "stm32f10x_crc.h"
6 #include "stm32f10x_dac.h"
7 #include "stm32f10x_dbgmcu.h"
8 #include "stm32f10x_dma.h"
9 #include "stm32f10x_exti.h"
10 #include "stm32f10x_flash.h"
11 #include "stm32f10x_fsmc.h"
```

```
12 #include "stm32f10x_gpio.h"
13 #include "stm32f10x_i2c.h"
14 #include "stm32f10x_iwdg.h"
15 #include "stm32f10x_pwr.h"
16 #include "stm32f10x_rcc.h"
17 #include "stm32f10x_rtc.h"
18 #include "stm32f10x_sdio.h"
19 #include "stm32f10x_spi.h"
20 #include "stm32f10x_tim.h"
21 #include "stm32f10x_usart.h"
22 #include "stm32f10x_wwdg.h"
23 #include "misc.h"
```

图 4.10 配置头文件

2.4 函数库文件之间的关系

上述文件对应到 CMSIS 标准架构上，如图 4.11 所示。位于 CMSIS 层的文件包含进工程，除了特殊系统时钟需要修改 `system_stm32f10x.c`，其它文件丝毫不用修改，也不建议修改。对于位于用户层的几个文件，针对不同的应用需要进行增删（用条件编译的方法增删）和改动。



4.11 函数库文件在 CMMIS 架构中的位置

2.4 参考资料和帮助文档

《STM32F10X-中文参考手册》：全方位介绍了 STM32 芯片的各种片上外设，它把 STM32 的时钟、存储器架构、及各种外设、寄存器等

《STM32 规格书》：相当于 STM32 的 datasheet，包含了 STM32 芯片所有的引脚功能说明及存储器架构、芯片外设架构说明。

《Cortex-M3 权威指南》：由 ARM 公司提供，详细讲解了 Cortex 内核的架构和特性，深入了解 Cortex-M 内核的首选。这个手册也被翻译成中文出版成书。

《stm32f10x_stdperiph_lib_um.chm》：固件库自带的电子文档，可以查阅解标准库提供了哪些外设、函数原型或库函数的调用的方法。

二、ARM/STM32F103 实验

实验八 通用输入输出 (GPIO)/流水灯实验

一、实验目的

1. 学习使用 EDA 软件搭建 STM32 微处理器系统电路原理图；
2. 学习使用 Keil MDK 软件开发 STM32 微处理器软件；
3. 学习和理解 STM32 微处理器 GPIO 的原理和使用方法。

四、实验内容

4. 在 Proteus 软件中搭建电路原理图, 如图 4.12.1 所示。

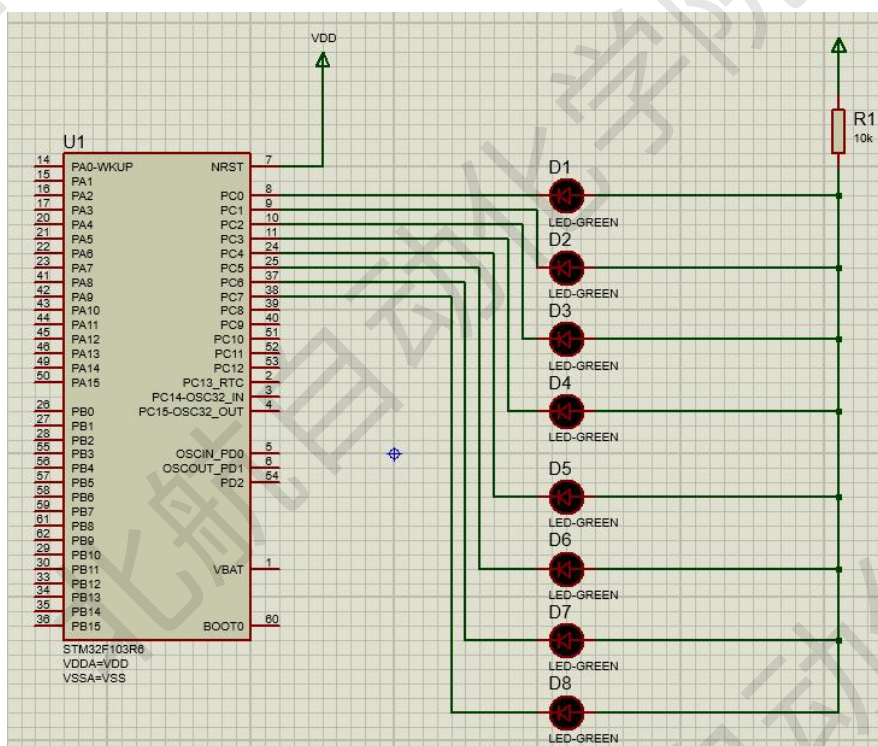


图 4.12.1 STM32F103R6 电路原理图

元器件列表

元件名称	所属类	所属子类	功能说明
STM32F103R6	Microprocessor ICs	Stellaris Family	微处理器
RES	Resistors		电阻
LED-GREEN	Optoelectronics	LEDs	绿色发光二极管

5. 在 Keil MDK 中编写系统软件,编译并加载到 Proteus 中的 STM32 微处理模型中。
- 系统软件程序如下:


```
#include "stm32f10x.h"
GPIO_InitTypeDef GPIO_InitStructure;
void delay_ms(uint32_t ms)
{
    uint32_t i_cnt,j_cnt;
    for(i_cnt=0;i_cnt<3000;i_cnt++);
    for(j_cnt=0;j_cnt<ms;j_cnt++);
}
int main(void)
{
    u16 gpio_pin;
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1|GPIO_Pin_2|GPIO_Pin_3|
    GPIO_Pin_4|GPIO_Pin_5|GPIO_Pin_6|GPIO_Pin_7;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_Init(GPIOC, &GPIO_InitStructure);

    gpio_pin=GPIO_Pin_0 | GPIO_Pin_1|GPIO_Pin_2|GPIO_Pin_3|GPIO_Pin_4|GPIO_Pin_5|
    GPIO_Pin_6|GPIO_Pin_7;
    GPIO_SetBits(GPIOC, gpio_pin);

    while (1)
    {
        uint32_t i;
        for(i=0;i<8;i++)
        {
            switch(i)
            {
                case 0:  gpio_pin=GPIO_Pin_0;break;
                case 1:  gpio_pin=GPIO_Pin_1;break;
                case 2:  gpio_pin=GPIO_Pin_2;break;
                case 3:  gpio_pin=GPIO_Pin_3;break;
                case 4:  gpio_pin=GPIO_Pin_4;break;
                case 5:  gpio_pin=GPIO_Pin_5;break;
                case 6:  gpio_pin=GPIO_Pin_6;break;
                case 7:  gpio_pin=GPIO_Pin_7;break;
                default: break;
            }
            GPIO_ResetBits(GPIOC, gpio_pin);
            delay_ms(199000);
            GPIO_SetBits(GPIOC, gpio_pin);
            delay_ms(199000);
        }
    }
}
```

三、实验设备

1. 计算机（Windows7/10 32/64 位操作系统）
2. 电子设计自动化（EDA）软件 Proteus
3. 源程序集成开发软件 Keil MDK

四、预习要求

1. 学习 ARM、STM32F103 微处理器基本概念；
2. 安装 Proteus 和 Keil MDK 软件，了解使用方法；
3. 预习思考题
 - 1) STM32 芯片中的 AHB、APB 总线的作用是什么？
 - 2) 使用 Keil MDK 和固件库法对 STM32F103R6 芯片进行编程时，需要从固件库中复制哪些文件到工程项目中？
 - 3) GPIO 引脚可以设置为哪几种输入输出方式？

五、实验原理

1. 通用输入输出（GPIO）基本概念

通用输入输出 GPIO 是嵌入式开发中最基本、最常用的硬件端口。STM32F103 系列芯片最多可提供 8 组端口（PORT，编号 A~H），每组 16 个 IO 引脚，共 128 个。具体到某个型号，则配备的端口数量、IO 引脚数量略有不同。每个 IO 端口控制器含有多个寄存器，用于配置和支持 IO 端口/引脚的工作模式。在使用固件库编程方法时可以不必了解控制寄存器。

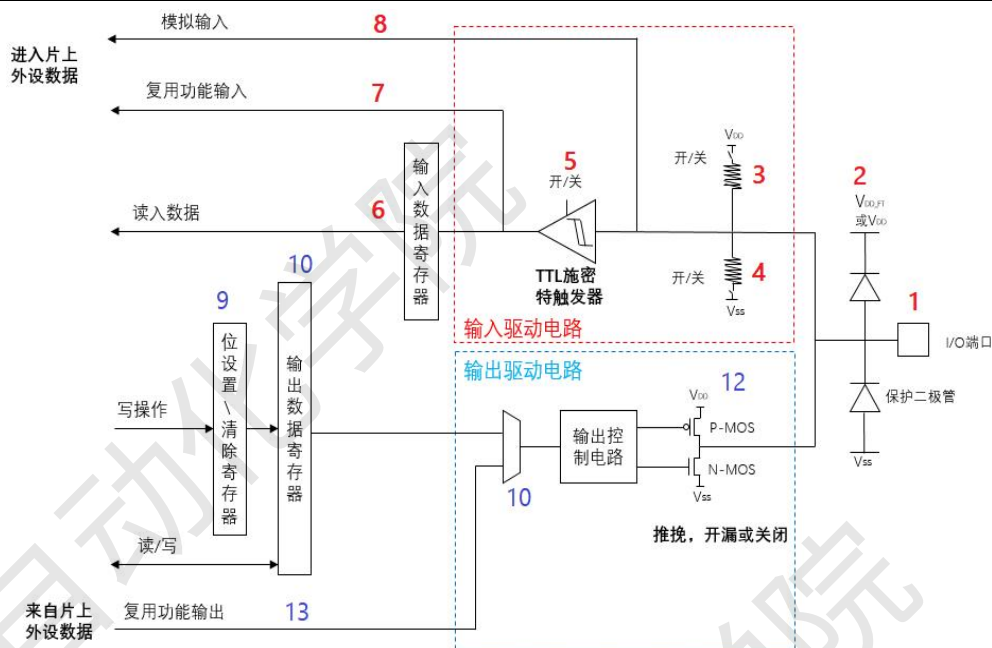


图 4.12.2 STM32F103 系列芯片的 GPIO 结构

STM32F103R6 芯片只有 4 组端口 51 个 IO 引脚：PA0~15、PB0~15、PC0~15、PD0~2。

每个 IO 引脚单独配置为输入、输出，另外也可以配置为复用功能。

输入模式四种：

- 上拉：引脚悬空时为 1
- 下拉：引脚悬空时为 0
- 浮空：引脚悬空则电平不确定
- 模拟：用于 AD 采集

输出模式两种（加上复用方式则为四种）：

- 推挽：P 管、N 管交替工作，各自负责正负半周的波形放大任务
- 开漏：输出 0：N 管导通，1：N 管关闭；P 管总是关闭。因此需要在引脚外接上拉电阻才能输出逻辑 1。一般来说，开漏是用来连接不同电平的器件，匹配电平用的。通过改变上拉电源的电压，便可以改变传输电平。
- 复用功能：将 IO 引脚映射到某个片上外设，引脚的输出信号来自片上外设。分为复用推挽输出、复用开漏输出。

2. GPIO 固件库编程所使用的函数

(1) GPIO 初始化函数

```
void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct)
```

- 功能： 根据 GPIO_InitStruct 中指定的参数初始化外设 GPIOx 寄存器
- GPIOx: 用来选择待操作的 GPIO, x 可以是 A, B, C, D 等等
- GPIO_InitStruct 指向 GPIO_InitTypeDef 结构的指针, 包含了待操作 GPIO 的配置信息
- GPIO_InitTypeDef 定义于文件 “stm32f10x_gpio.h”

```
typedef struct {  
    u16 GPIO_Pin;  
    GPIOSpeed_TypeDef GPIO_Speed;  
    GPIOMode_TypeDef GPIO_Mode;  
} GPIO_InitTypeDef;
```

- GPIO_Pin 为待设置的 GPIO 管脚, 使用操作符 “|” 可以一次选中多个管脚值

GPIO_Pin_None 无管脚被选中

GPIO_Pin_0 选中管脚 0

GPIO_Pin_1 选中管脚 1

GPIO_Pin_2 选中管脚 2

... ..

GPIO_Pin_4 选中管脚 15

GPIO_Pin_All 选中全部管脚

- GPIO_Speed 值, 据不同 MCU 型号, 有些有 3 个, 有些有 4 个

GPIO_Speed_10MHz 最高输出速率 10MHz

GPIO_Speed_2MHz 最高输出速率 2MHz

GPIO_Speed_50MHz 最高输出速率 50MHz

- GPIO_Mode 值

GPIO_Mode_AIN 模拟输入

GPIO_Mode_IN_FLOATING 浮空输入

GPIO_Mode_IPD 下拉输入

GPIO_Mode_IPU 上拉输入

GPIO_Mode_Out_OD 开漏输出

GPIO_Mode_Out_PP 推挽输出

GPIO_Mode_AF_OD 复用开漏输出

GPIO_Mode_AF_PP 复用推挽输出

(2) 设置 GPIO 时钟

void RCC_APB2PeriphClockCmd(u32 RCC_APB2Periph, FunctionalState NewState)

- 功能：使能或者禁止 APB2 外设时钟
- RCC_APB2Periph: 指定所控制的 APB2 外设

RCC_APB2Periph 的值：

RCC_APB2Periph_AFIO 功能复用 IO 时钟

RCC_APB2Periph_GPIOA GPIOA 时钟

RCC_APB2Periph_GPIOB GPIOB 时钟

RCC_APB2Periph_GPIOC GPIOC 时钟

RCC_APB2Periph_GPIOD GPIOD 时钟

RCC_APB2Periph_ADC1 ADC1 时钟

RCC_APB2Periph_ADC2 ADC2 时钟

RCC_APB2Periph_TIM1 TIM1 时钟

RCC_APB2Periph_SPI1 SPI1 时钟

RCC_APB2Periph_USART1 USART1 时钟

RCC_APB2Periph_ALL 全部 APB2 外设时钟

- NewState: 指定外设时钟的新状态，取值为 ENABLE 或者 DISABLE

(3) 从 GPIO 引脚读入数据

u8 GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, u16 GPIO_Pin)

- 功能：读取指定端口管脚的输入
- 返回值：输入端口管脚值
- 参数定义：见“(1) GPIO 初始化函数”部分

(4) 将 GPIO 引脚的输出置 1

`void GPIO_SetBits(GPIO_TypeDef* GPIOx, u16 GPIO_Pin)`

- 功能： 设置指定的数据端口位
- 参数定义： 见“(1) GPIO 初始化函数”部分

(5) 将 GPIO 引脚的输出清 0

`void GPIO_ResetBits(GPIO_TypeDef* GPIOx, u16 GPIO_Pin)`

- 功能： 清除指定的数据端口位
- 参数定义： 见“(1) GPIO 初始化函数”部分

(6) 设置 GPIOx 所有引脚的输出

`void GPIO_Write(GPIO_TypeDef* GPIOx, uint16_t PortVal)`

- 功能： 向指定的数据端口写入 16 位的数据 PortVal。PortVal 的每个二进制位对应一个引脚，如 D0 对应引脚 Pin0。
- 参数定义： 见“(1) GPIO 初始化函数”部分

(7) 读入 GPIOx 指定的引脚的状态

`uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)`

- 功能： 返回 GPIOx 引脚 GPIO_Pin 的状态，类型是 8 位整型数据，引脚输入为低电平时为 0，高电平时为 1。
- 参数定义： 见“(1) GPIO 初始化函数”部分

(8) 读入 GPIOx 所有引脚的状态

`uint16_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx)`

- 功能： 返回指定 GPIOx 所有引脚的状态，返回数据类型为 16 位整型，每位对应一个引脚。如 D0 位对应 Pin0。
- 参数定义： 见“(1) GPIO 初始化函数”部分

(9) 其它 GPIO 函数可参考 stm32f10x_gpio.c 源文件。

六、实验步骤

安装 Proteus 和 Keil MDK 软件之后，安装以下步骤完成实验。

1. 软件设计

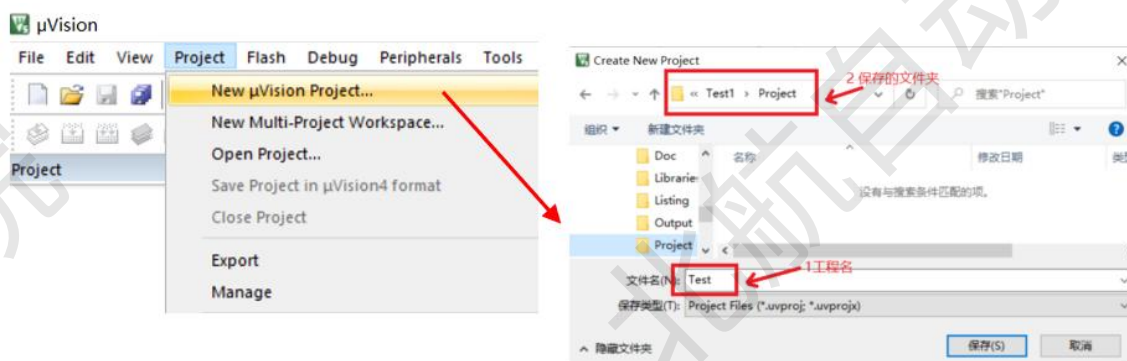
(1) 在硬盘上新建一个文件夹，用来存放所开发的程序和工程。这里为了方便，文件夹命名为 test1。在文件夹中再建几个子文件夹，名称和作用如下图所示

rogram > Arm > 流水灯 > Test1		
名称	名称	作用
Doc	Doc	用来存放程序说明的文件，由写程序的人添加
Libraries	Libraries	存放的是库文件
Listing	Listing	存放编译器编译时候产生的 C/汇编/链接的列表清单
Output	Output	存放编译产生的调试信息、hex 文件、预览信息、封装库等
Project	Project	用来存放工程
User	User	用户编写的驱动文件

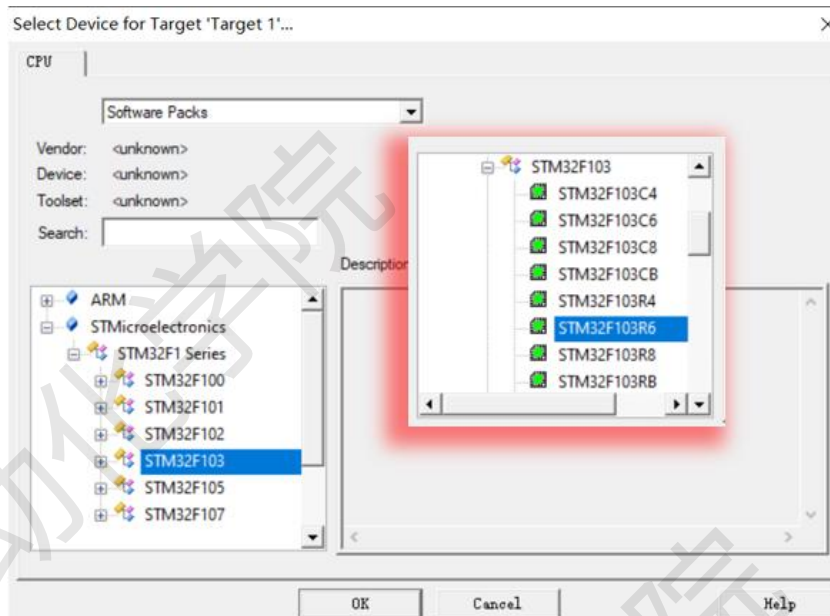
(2) 工程文件夹做以下操作

文件夹	操作
Doc	根据需要编写文档：工程说明、调试和改错记录
Libraries	将 固件库Libraries 文件夹中所有内容复制到此处
Listing	无
Output	无
Project	无
User	将 固件库Project\STM32F10x_StdPeriph_Template 文件夹中的 stm32f10x_conf.h 、 stm32f10x_it.h 、 stm32f10x_it.c 、 main.c 复制到此处

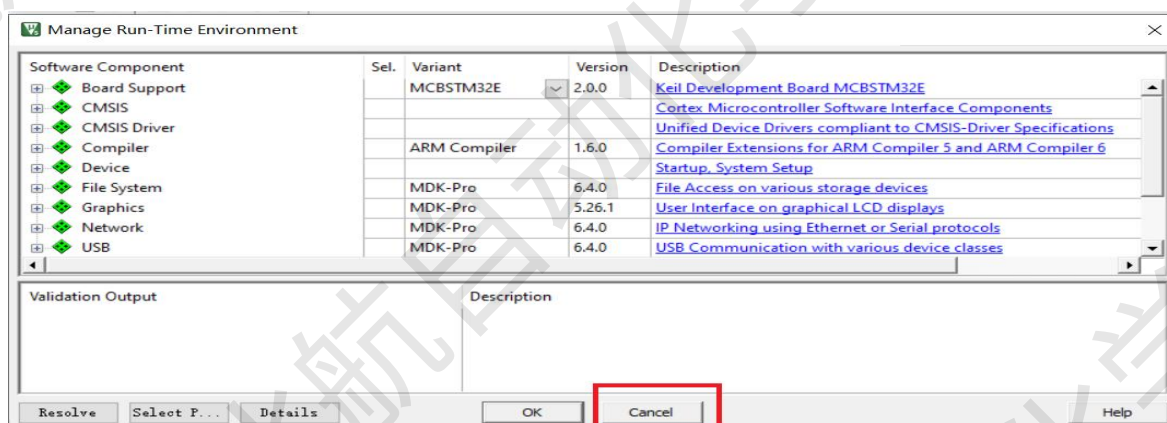
(3) 打开 KEIL MDK，新建一个工程，工程名根据喜好命名。这里取名 Test



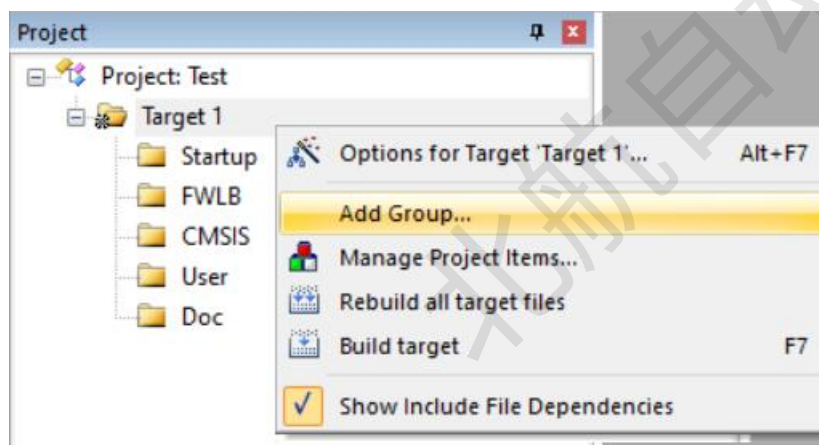
(4) 然后进入下一步，选择芯片型号。如图所示，点开“STM32F103”后，选择 STM32F103R6，然后点击 OK 键。



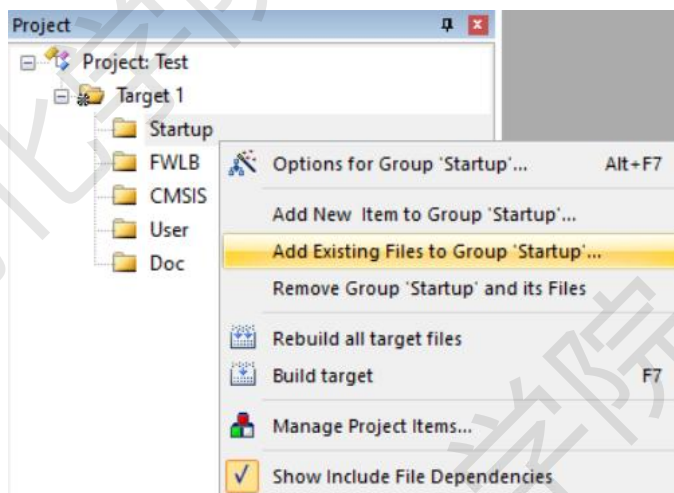
(5) 选择库文件。因为前面我们已经手动复制了库文件，所以这里选择 **cancel** 键，然后就进入工程界面。



(6) 在 Project 窗口，鼠标右键点击 Target1，在弹出菜单中点击 Add Group，新建 Startup、FWLB 等目录



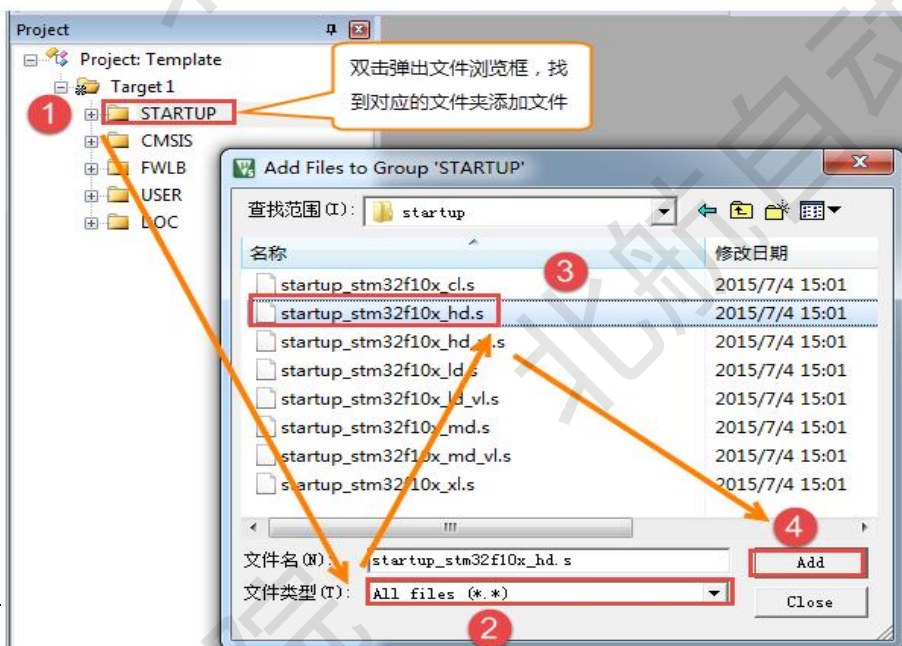
(7) 在建好的文件夹上点击鼠标右键，然后点击弹出菜单中的选项，向文件夹中添加文件。由于文件已经复制到文件夹中。因此选择 Add Existing Files to...选项，把文件添加进去即可。



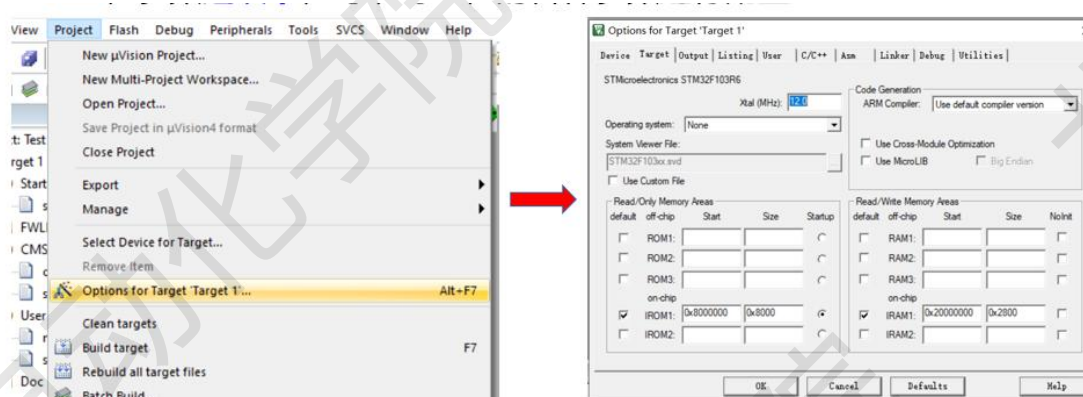
具体添加的文件见下图

名称	存放的文件
STARTUP	startup_stm32f10x_hd.s
CMSIS	core_cm3.c、system_stm32f10x.c
FWLB	STM32F10x_StdPeriph_Driver\src 文件夹下的全部 C 文件，即固件库
USER	用户编写的文件： main.c: main 函数文件，暂时为空 stm32f10x_it.c: 跟中断有关的函数都放这个文件，暂时为空
DOC	工程说明.txt: 程序说明文件，用于说明程序的功能和注意事项等

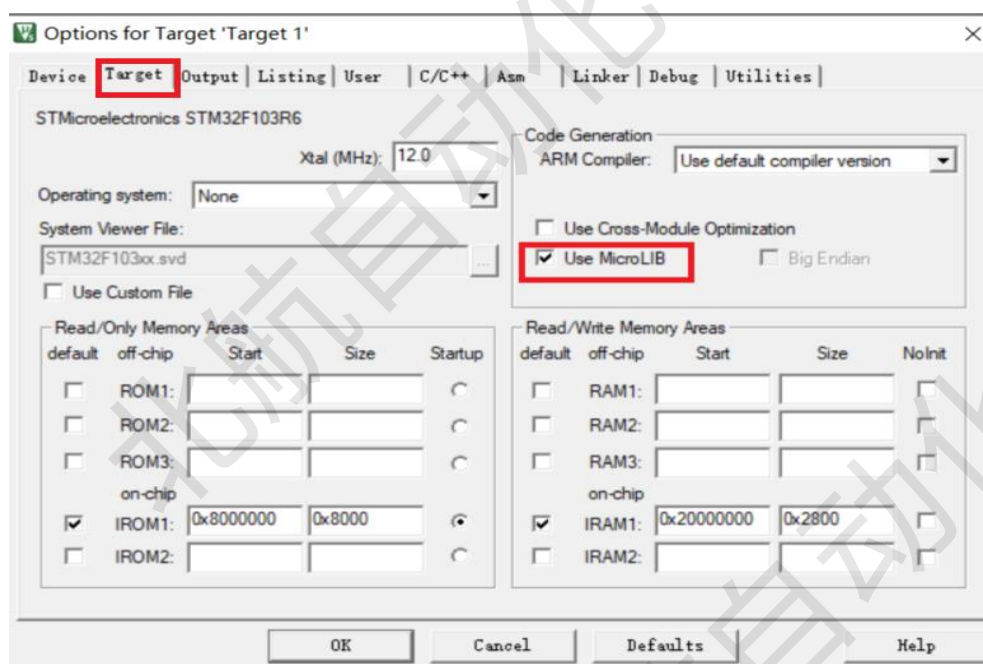
示例：添加文件过程。此处的例子是在文件夹上直接双击鼠标，效果等同于鼠标右键+Add Existing Files



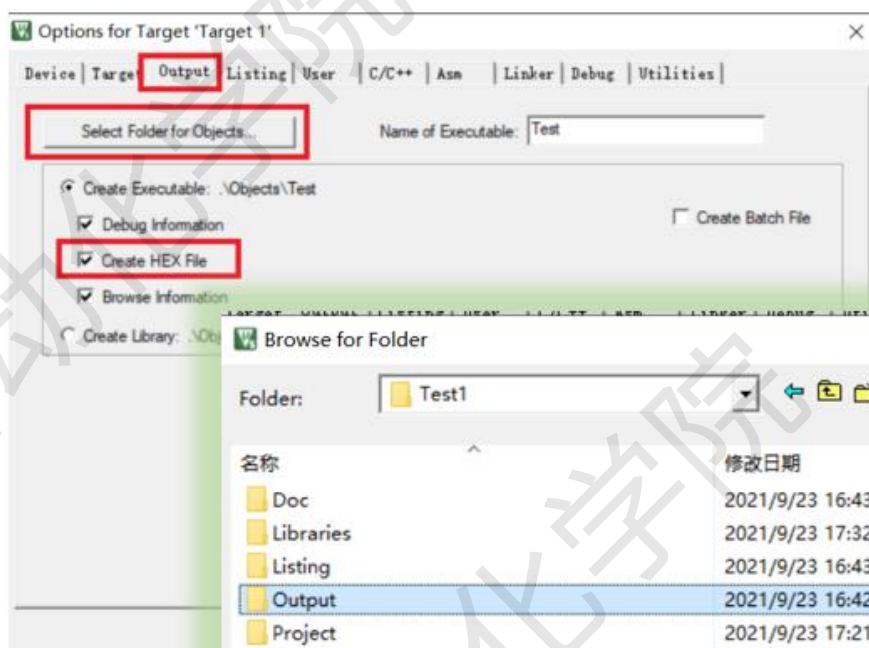
(8) 配置工程参数。点击 Project->Options for Target..., 弹出工程参数选项卡, 可以对工程的各种参数进行配置。



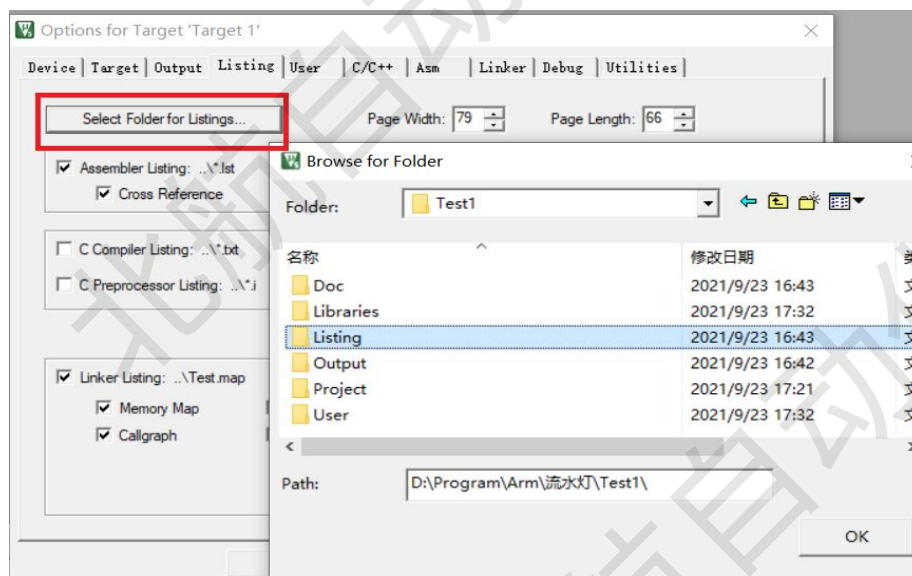
(8.1) Target 中选中微库 “Use MicroLib”, 为的是在日后编写串口驱动的时候可以使用 printf 函数



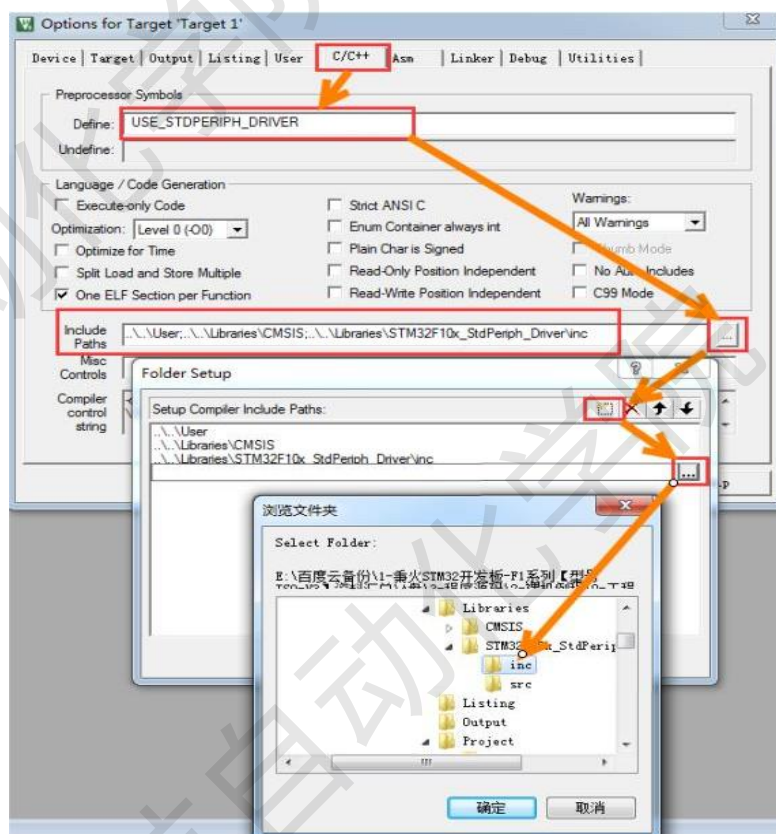
(8.2) Output 选项卡中把 Create HEX File 选项勾上, 以便生成 Hex 可行性文件;把输出文件夹定位到工程目录下的“Output”文件夹, 编译生成的 Hex 文件将保存在这里



(8.3) 在 Listing 选项卡中把输出文件夹定位到工程目录下的“Listing”文件夹。



(8.4) 在 C/C++选项卡中添加处理宏及编译器编译的时候查找的头文件路径。如果头文件路径添加有误，则编译的时候会报错找不到头文件。添加宏 USE_STDPERIPH_DRIVER，相当于在文件中使用“#define”语句定义宏。编译器中添加宏的好处是不用在源文件中修改代码。

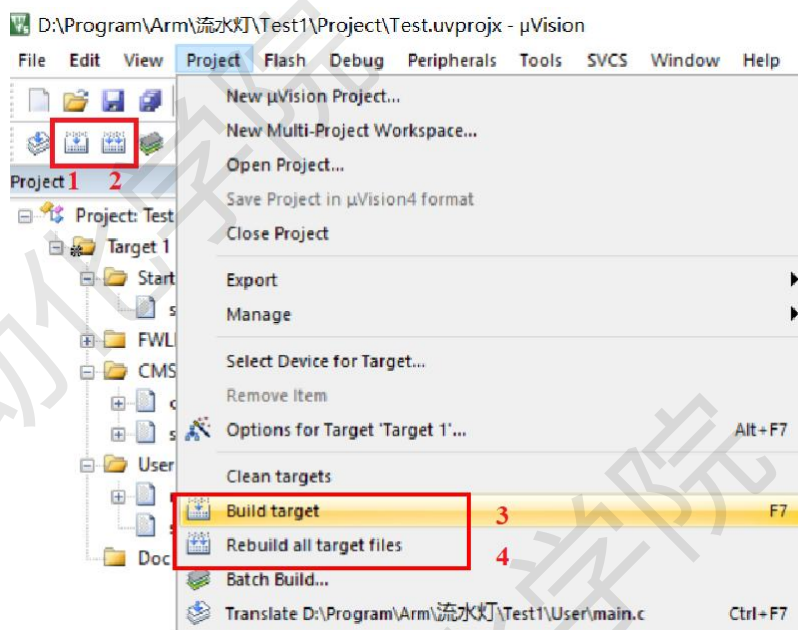


注意：Include Path 设置应当如下图所示，否则编译报错



(9) 还可以在 Debug 和 Utility 选项中设置仿真器。仿真器用于和硬件开发板连接。使用时可以参考相关说明书。因课程学习是使用 Proteus 仿真，因此 Debug 和 Utility 的设置略过。

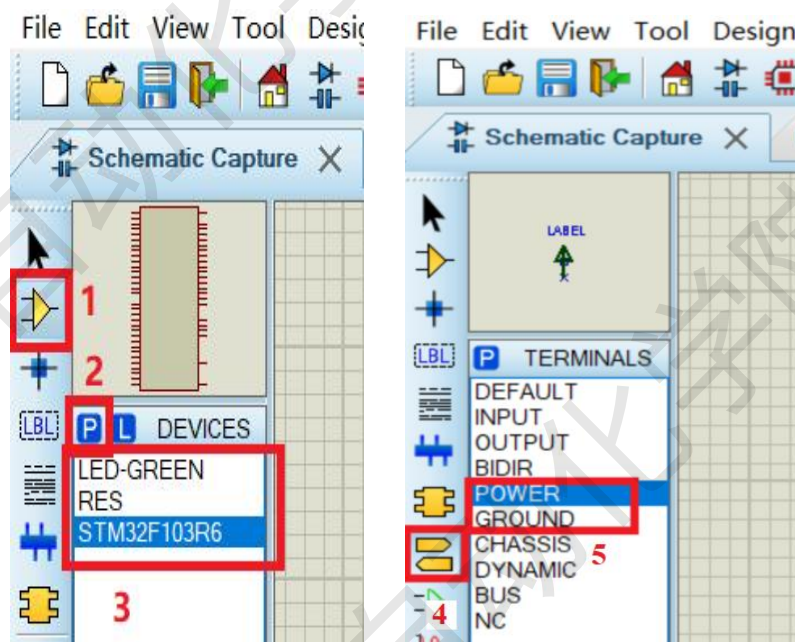
(10) 将源程序复制到 **main.c**，即可进行编译。编译方法：如图，点击图标 1、2 中到任何一个，或者选择 3、4 中的任一选项。



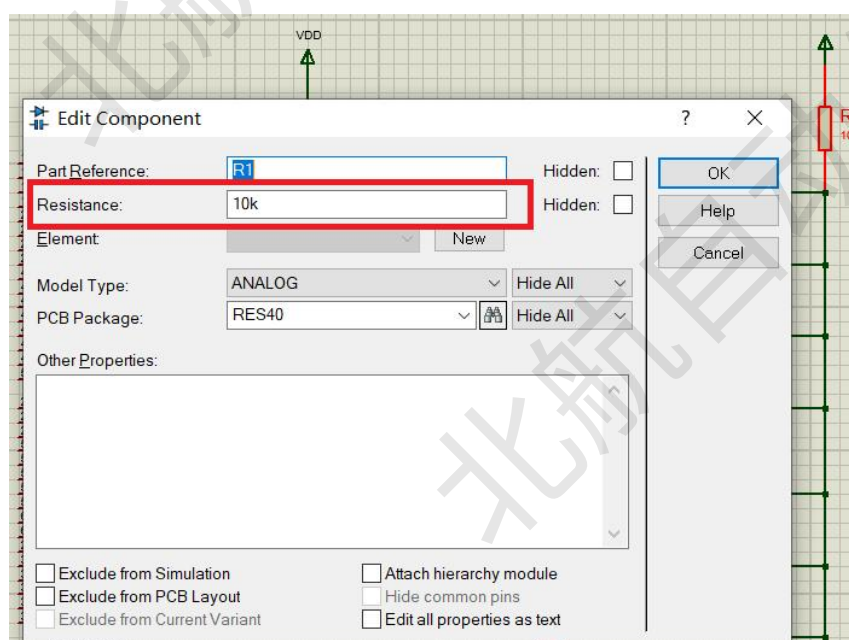
(11) 编译结束后，可以看到在 **Output** 文件夹中，生成了可执行文件 **Test.hex**。这个文件可以下载到 STM32 开发板运行，也可以提供给 Proteus 仿真运行

2. 硬件设计

(1) 按照 Proteus 的规则新建一个工程。新建工程的过程中注意选择 STM32F103R6 芯片。系统使用的器件较少，可以按照下图的顺序点击 1、再点击 2，然后在弹出的对话框里搜索并选择以下器件：RES（电阻）、LED-GREEN。然后将上述器件拖放到工作区并连线。注意 RES 电阻需要修改阻值为 10K。电源的选择如图中 4、5 所示

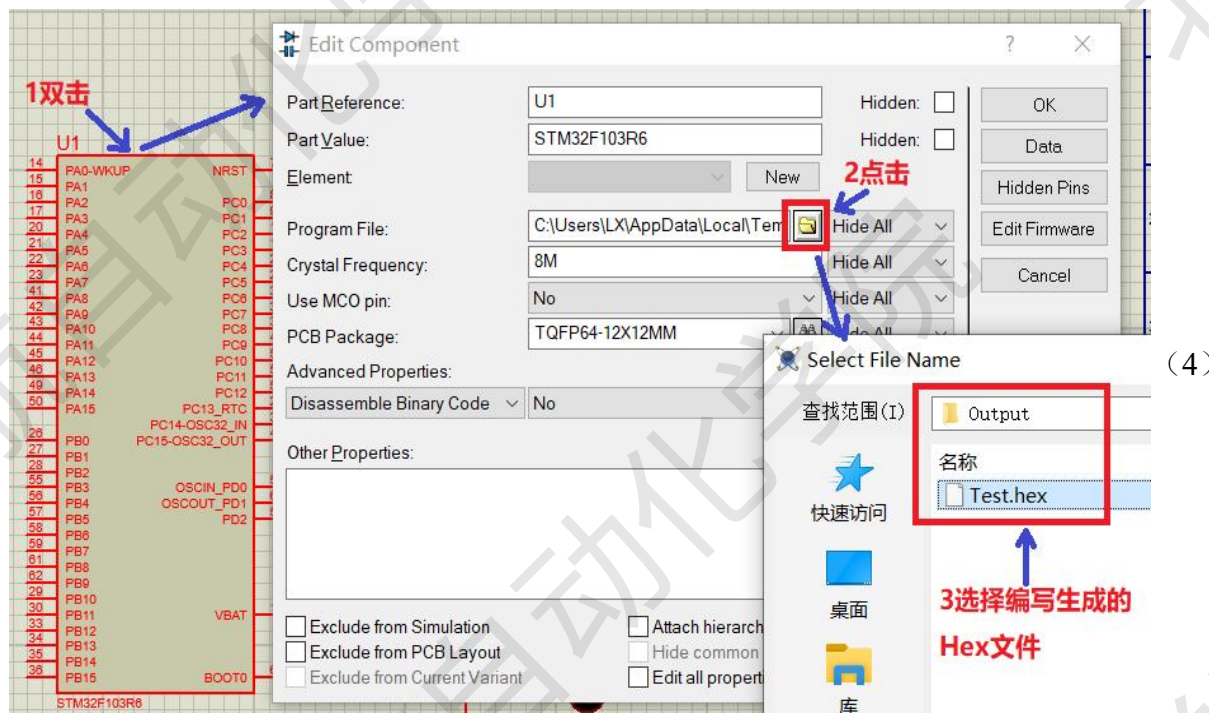


(2) 电阻值修改方法：双击电阻，弹出对话框。其中红框内即为电阻值。对话框中也可以修改电阻的名称。

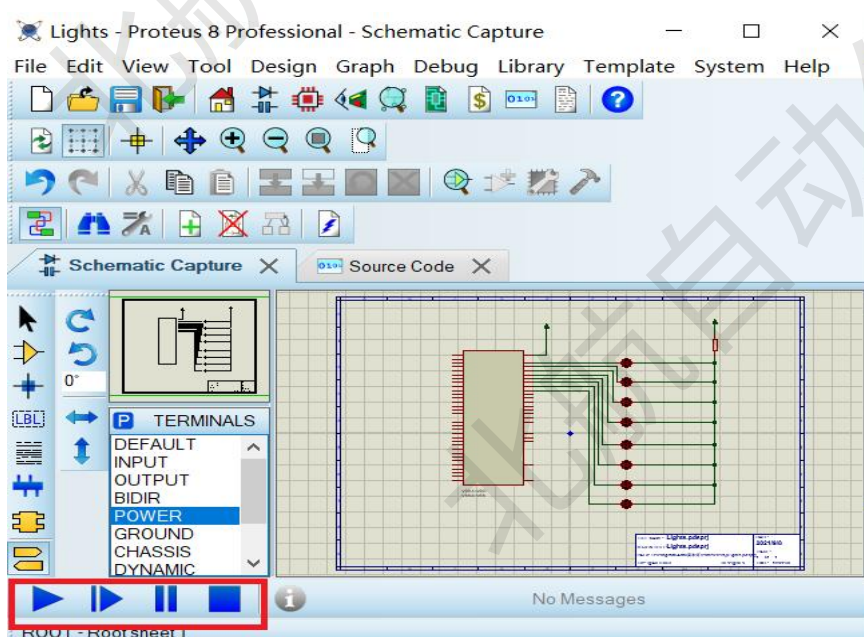


(3) 为了能够仿真运行，需要设置 STM32F103R6 模型的参数。双击 STM32F103R6 模块，在弹出的对话框中点击 Program File 选择框，改变目录至刚才建立的 Keil 工程的 Output 文件夹，选择 Test.hex。

*此处也可以人为设置芯片的工作频率，使得芯片在没有时钟电路的情况下仍然能够运行



点击 Proteus 左下角的图标，即可运行程序，控制系统仿真的过程



七、注意事项

八、实验报告要求

实验报告中应包括以下内容：

- 1.实验目的
- 2.实验原理
- 3.实验内容与实验过程描述
- 4.实验结果分析与实验结论
- 5.收获、体会及建议
- 6.预习思考题及实验验证
- 7.实验后思考题
- 8.实验源程序

实验报告模板可参照附件 1。

九、课后思考题

1. 程序中使用了 `GPIO_ResetBits()`和 `GPIO_SetBits()`函数对引脚输出进行设置。参考 `GPIO` 的相关函数，是否有更简洁的代码实现流水灯功能？

十、参考资料

1. 《STM32F10X-中文参考手册》：全方位介绍了 STM32 芯片的各种片上外设，它把 STM32 的时钟、存储器架构、及各种外设、寄存器等
2. 《STM32 规格书》：相当于 STM32 的 `datasheet`，包含了 STM32 芯片所有的引脚功能说明及存储器架构、芯片外设架构说明。
3. 《Cortex-M3 权威指南》：由 ARM 公司提供，详细讲解了 Cortex 内核的架构和特性，深入了解 Cortex-M 内核的首选。这个手册也被翻译成中文出版成书。
4. 《stm32f10x_stdperiph_lib_um.chm》：固件库自带的电子文档，可以查阅解标准库提供了哪些外设、函数原型或库函数的调用的方法。

实验九 SysTick 时钟/数码管显示实验

一、实验目的

1. 了解 ARM Cortex-M3 内核系统定时器 SysTick
2. 学习使用固件库法对 SysTick 编程

五、实验内容

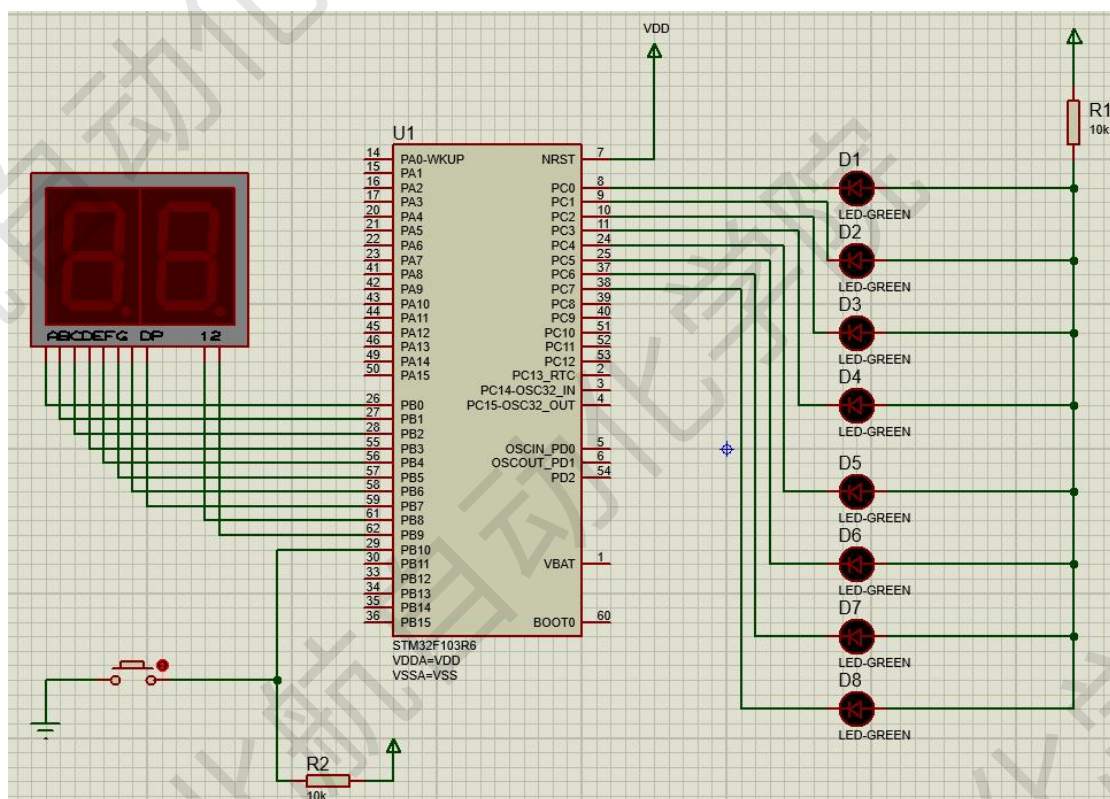


图 4.13.1 电路原理图

1. 在 Proteus 软件中实现图 4.13.1 中所示的电路原理图。元器件列表如下所示。

元件名称	所属类	所属子类	功能说明
STM32F103R6	Microprocessor ICs	Stellaris Family	微处理器
RES	Resistors		电阻
LED-GREEN	Optoelectronics	LEDs	绿色发光二极管
7SEG-MPX2-CC	Optoelectronics	7-Segment Displays	共阴极 7 段数码管

2. 在 Keil MDK 中编写系统软件，编译并加载到 Proteus 中的 STM32 微处理模型中。

系统软件程序如下：

(1) main.c 中的源程序

```
uint32_t i; //用于循环
uint32_t TimingCount; //每次 SysTick 中断减 1，用于流水灯延时
GPIO_InitTypeDef GPIO_InitStructure; //GPIO 设置结构，用于初始化
int main(void) //主函数
{
    int oldTC ;
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);
    GPIO_InitStructure.GPIO_Pin=GPIO_Pin_0 | GPIO_Pin_1 |
    GPIO_Pin_2 | GPIO_Pin_3 | GPIO_Pin_4 | GPIO_Pin_5 | GPIO_Pin_6 | GPIO_Pin_7;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_Init(GPIOC, &GPIO_InitStructure); //GPIOC 初始化设置
    GPIO_Write(GPIOC, 0x00ff); //GPIOC 低 8 位输出 FF,流水灯灭
    Init_SysTick(); //初始化 SysTick 时钟，其实现在后面
    i=0;
    while (1) //TimingCount 每计数 40 次，流水灯变化一次
    {
        if(((TimingCount%40)==0) && (oldTC != TimingCount))
        {
            oldTC = TimingCount;
            if(i<8)
                GPIO_Write(GPIOC, ~(1<<i));
            else if(i==8)
                GPIO_Write(GPIOC, 0x0000);
            else
                GPIO_Write(GPIOC, 0x00ff);
            i++;
            if(i>10){i=0; TimingCount=10;}
        }
    }
}

void Init_SysTick(void) //初始化 SysTick 时钟
{
    if(SysTick_Config(SystemCoreClock/1000))
        while(1);
}

void incCounter(void) //被 SysTick 时钟中断服务函数调用
{
    TimingCount++; //统计 SysTick 时钟中断调用的次数
}
```

(2) stm32f10x_it.c 中的源程序

这个 c 语言源程序文件主要包含了中断服务函数。中断服务函数名称已经被预定义，需要用户自己编写函数内部的代码实现中断处理。这里我们使用了 SysTick 时钟中断，因此需要完善其相应的中断服务函数 SysTick_Handler。代码如下：

```
extern void incCounter(void);
void SysTick_Handler(void) //SysTick 时钟中断处理函数
{ incCounter();
}
```

这个中断处理函数比较简单。为了程序结构比较清晰，只是调用了 main.c 中的函数 incCounter()。

3. （选做）在实验内容 2 的基础上，修改和添加代码，设置 GPIOB 引脚，其中 PB0~PB9 为输出，PB10 为输入；编写程序，使得数码管能够显示数字（两个数码管显示内容相同），并且每按下一次按键，两个数码管显示的数字加 1（一直到显示 FF 为止，再从 00 开始显示。同时流水灯也在流转。其中数码管的七段码分别为：

0	1	2	3	4	5	6	7
0x3F	0x06	0x5B	0x4F	0x66	0x6D	0x7D	0x07
8	9	A	B	C	D	E	F
0x7F	0x6F	0x77	0x7C	0x39	0x5E	0x79	0x71

4. （选做）在实验内容 3 的基础上，修改和添加代码，使得数码管能够循环显示 0~99（或 FF）之间的数字，每按下一次按键，数码管显示的数字加 1（比如 00, 01, 02...98, 99）。

三、实验设备

1. 计算机（Windows 7/10 32/64 位操作系统）
2. 电子设计自动化（EDA）软件 Proteus
3. 源程序集成开发软件 Keil MDK

四、预习要求

1. 了解 ARM- CM3 内核中的 SysTick 工作原理；
2. 预习 STM 32F103R6 固件库中 SysTick 相关函数及其使用方法。

3. 预习思考题

1) STM32F103R6 芯片有哪几种时钟信号, 使用芯片厂商默认的设置函数对这些时钟进行初始化, 其频率各为多少?

2) 简述 SysTick 时钟的特点。

五、实验原理

Cortex - M3 处理器内部包含了一个简单的定时器 SysTick。所有的 CM3 芯片都带有 SysTick 定时器, 因此 SysTick 定时使得在不同 CM3 芯片之间的移植工作得以简化。可以对 SysTick 设置计数值, 当减 1 计数至 0 时触发中断, 从而实现定时功能。

SMT32 固件库对 SysTick 编程进行了封装, 编程时可以不必关注和了解 SysTick 寄存器设置、中断及其优先级设置等工作。步骤主要有:

(1) 在 main 函数中调用 SysTick_Config(uint32_t ticks) 进行初始化。其中参数 ticks 为计数初值。其计算方法为:

计数初值 ticks = 系统频率 * 定时时间

例如, 若系统时钟为 72MHz, 则 SysTick_Config(1) = 1/72000000s;

又如, SysTick_Config(SystemCoreClock/1000), 其中参数 SystemCoreClock 为固件库中预定义的系统时钟频率, 默认值为 72MHz; 除以 1000 表示计数初值为 72000, 因此定时为 72000/72Mhz = 1ms。

*注意: 在 Proteus 中进行仿真时, 由于计算机系统运算能力有限, 无法按照 72MHz 的频率对微处理器进行仿真运行, 实际工作频率要低得多。

(2) 在 stm32f10x_it.c 中实现中断服务函数 void SysTick_Handler(void)

函数 SysTick_Handler() 是固件库中预定义的 SysTick 中断处理函数, 但其内部处理代码需要由用户根据需要自行实现。

(3) GPIO 设置、引脚输入、输出等操作相关的函数, 参考实验十二 第五节“实验原理”部分。

六、实验步骤

参考实验十二所示的步骤。



七、注意事项

八、实验报告要求

实验报告中应包括以下内容：

- 1.实验目的
- 2.实验原理
- 3.实验内容与实验过程描述
- 4.实验结果分析与实验结论
- 5.收获、体会及建议
- 6.预习思考题及实验验证
- 7.实验后思考题
- 8.实验源程序

实验报告模板可参照附件 1。

九、课后思考题

试查找资料回答以下问题：

1. SysTick 时钟中断服务程序的中断向量、及其在中断向量表中的位置是多少？
2. 除了 SysTick 时钟，STM32F103R6 还有哪些具有定时功能的模块？

十、参考资料

1. 《STM32F10X-中文参考手册》：全方位介绍了 STM32 芯片的各种片上外设，它把 STM32 的时钟、存储器架构、及各种外设、寄存器等
2. 《STM32 规格书》：相当于 STM32 的 datasheet，包含了 STM32 芯片所有的引脚功能说明及存储器架构、芯片外设架构说明。
3. 《Cortex-M3 权威指南》：由 ARM 公司提供，详细讲解了 Cortex 内核的架构和特性，深入了解 Cortex-M 内核的首选。这个手册也被翻译成中文出版成书。
4. 《stm32f10x_stdperiph_lib_um.chm》：固件库自带的电子文档，可以查阅解标准库提供了哪些外设、函数原型或库函数的调用的方法。

附件 1：（见下页）



成绩 _____

北京航空航天大学
BEIHANG UNIVERSITY

微机原理及接口技术 实验报告

院（系）名称 自动化科学与电气工程学院

专业名称 自动化

学生学号 XXXX

学生姓名 XXXX

指导教师 XXXX

2023 年 11 月

实验三 字符串排序及 BCD 码加法

(三号, 黑体; 居中; 单倍行距; 段前、段后各 0.5 行)

实验时间_____ 实验编号_____ 同组同学_____

(“实验编号”填写实验所用计算机编号; 若有同组实验同学, 需在“同组同学”处填写其姓名, 没有则填无)

一、实验目的 (四号黑体; 左对齐; 单倍行距; 段前、段后 0.5 行)

(以上均为小四号, 宋体, 英文及字母 Times New Roman 体; 首行缩进 2 字符; 两端对齐; 1.5 倍行距)

二、实验原理

1. 说明实验中涉及到的相关理论、概念、算法或者原理

2. 附上电路原理图 (实验 4、5、6、7、8)、流程图 (实验 3、4、5、6、7、8)、

(图按顺序从 1 编号如: 图 1. 图名, 五号, 宋体, 英文及字母 Times New Roman 体, 居中)

(正文小四号, 宋体, 英文及字母 Times New Roman 体; 首行缩进 2 字符; 两端对齐; 1.5 倍行距)

三、实验内容与实验过程描述

陈述实验内容、步骤和实验结果

(小四号, 宋体, 英文及字母 Times New Roman 体; 首行缩进 2 字符; 两端对齐; 1.5 倍行距)

四、实验结果分析与实验结论

1. 对实验所得结果进行分析处理, 总结实验结论。

2. 实验过程中遇到的问题分析总结。

(以上均为小四号, 宋体, 英文及字母 Times New Roman 体; 首行缩进 2 字符; 两



端对齐；1.5 倍行距）

五、收获、体会及建议

1. 总结实验收获与个人体会；欢迎同学提出改善实验课程的建议。

（以上均为小四号，宋体，英文及字母 Times New Roman 体；首行缩进 2 字符；两端对齐；1.5 倍行距）

注意：括号（）内为说明事项，最终实验报告中须删除！

附录 1

1、预习思考题及实验验证

（小四号，宋体，英文及字母 Times New Roman 体；首行缩进 2 字符；两端对齐；1.5 倍行距）

2、实验后思考题

- 1.
- 2.

（小四号，宋体，英文及字母 Times New Roman 体；首行缩进 2 字符；两端对齐；1.5 倍行距）

附录 2：实验源程序（）

（小四号，宋体，英文及字母 Times New Roman 体；首行缩进 2 字符；两端对齐；1.5 倍行距，分两栏书写）